

Faculté des Sciences et Ingénierie
Master 1 Mention Mathématiques et Applications
Parcours Interactions de l'informatique et des mathématiques pour l'intelligence artificielle

RAPPORT DE STAGE

Détection des deepfakes générés par des modèles de diffusion

AXCEL MUGISHA

Responsables de stage

JULIETTE CHEVALLIER, Maîtresse de Conférences (INSA Toulouse, IMT)
EMMANUELLE CLAEYS, Maîtresse de Conférences (Université de Toulouse, IRIT)

Stage effectué

À l'Institut de Mathématiques de Toulouse
Au sein de l'équipe Statistique et Optimisation
Et en collaboration avec l'équipe Machine Learning Integrity & Safety Fairness Impact Trust (MISFIT)
Du département Intelligence Artificielle de l'Institut de Recherche en Informatique de Toulouse (IRIT)
Du 27 avril au 28 août 2026

Année universitaire

2025 – 2026

Résumé

Ce stage a été réalisé en collaboration avec Victoria Pédoussat, stagiaire en première année du Master Mathématiques Appliquées à l'Ingénierie, à l'Industrie et à l'Innovation (MAPI3). Les différentes étapes du projet ont fait l'objet d'un travail commun et d'échanges réguliers, même si certaines tâches expérimentales ont été réparties entre nous.

Ce rapport porte sur la détection d'images synthétiques, en particulier celles générées par des modèles de diffusion. Les progrès récents de l'intelligence artificielle générative permettent de produire des images de plus en plus réalistes, rendant leur détection automatique particulièrement difficile.

Nous présentons dans un premier temps le fonctionnement des modèles de diffusion ainsi que plusieurs méthodes récentes de l'état de l'art de détection d'images générées. Le benchmark développé par le Pôle d'Expertise de la Régulation Numérique (PEReN) et VIGINUM est ensuite étudié afin de disposer d'un cadre de référence pour l'évaluation des détecteurs et l'analyse de leur robustesse. Les résultats publiés dans ce benchmark sont utilisés comme références externes pour situer les performances obtenues dans notre étude.

Le cœur de ce travail repose sur une approche de détection fondée sur la reconstruction d'une image à l'aide d'un modèle de diffusion pré-entraîné. À partir des quantités produites au cours de cette reconstruction, cinq représentations de l'image avant et après reconstruction sont étudiées : pixel, latent, bruit, fréquence et ondelettes. Pour chacune d'elles, plusieurs architectures de classification sont comparées, puis les modèles retenus sont évalués pour différents niveaux de bruitage α , plusieurs graines aléatoires et différents générateurs d'images synthétiques.

Les résultats montrent que les erreurs de reconstruction contiennent une information discriminante exploitable. Parmi les couples représentation–architecture évalués, l'approche pixel obtient les meilleures performances globales, avec une Balanced Accuracy moyenne de 94,7 % et une AUC de 98,7 %. Les expériences mettent également en évidence une sensibilité différente au niveau de bruitage et au générateur selon la représentation considérée.

Table des matières

Résumé	2
Organisme d'accueil	5
Introduction	7
1 Modèles de diffusion et état de l'art sur les méthodes de détection	8
1.1 Modèles de diffusion	8
1.2 Méthodes de détection d'images générées	13
2 Benchmark PEReN-VIGINUM	18
2.1 Jeu de données et détecteurs évalués	18
2.2 Performances, robustesse et généralisation	20
3 Protocole expérimental et résultats obtenus	23
3.1 Mise en œuvre et protocole expérimental	23
3.1.1 Environnement de calcul	23
3.1.2 Entraînement et évaluation	24
3.2 Extraction des cartes d'erreur	24
3.3 Choix des architectures	26
3.4 Résultats obtenus	30
3.4.1 Sélection des modèles	30
3.4.2 Influence du niveau de bruitage α	31
3.4.3 Comparaison globale des représentations	32
3.4.4 Performances selon les générateurs	32
3.4.5 Synthèse et discussion	34
Conclusion	36
Bibliographie	42
A Autres modèles génératifs	43
A.1 Autoencodeurs variationnels	43
A.2 Réseaux antagonistes génératifs	44
B Métriques de classification et réseaux de neurones	46
B.1 Métriques d'évaluation pour la classification binaire	46
B.2 Réseaux de neurones et réseaux convolutifs	47
B.3 Détail des architectures utilisées dans le rapport	49
B.3.1 Architectures associées à la <i>feature</i> pixel	49
B.3.2 Architectures associées à la <i>feature</i> latente	51
B.3.3 Architectures associées à la <i>feature</i> bruit	55
B.3.4 Architectures associées à la <i>feature</i> fréquence	56

B.3.5	Architecture associée à la <i>feature</i> ondelettes	59
B.4	Comparaison détaillée des architectures	61
C	Analyse fréquentielle et décomposition en ondelettes	65
C.1	Analyse dans le domaine fréquentiel	65
C.2	Décomposition en ondelettes	66

Organisme d'accueil

L'[Institut de Mathématiques de Toulouse](#) (IMT) est une Unité Mixte de Recherche (UMR 5219) constituant l'un des principaux pôles de recherche en mathématiques du site toulousain. Il rassemble environ 240 enseignants-chercheurs, chercheurs permanents, ingénieurs, techniciens et personnels administratifs, auxquels s'ajoutent près de 120 doctorants et une trentaine de chercheurs postdoctoraux. L'institut est placé sous la tutelle de six établissements et organismes : l'[Université de Toulouse](#), le [Centre National de la Recherche Scientifique](#) (CNRS), l'[INSA Toulouse](#), l'[Université Toulouse – Jean Jaurès](#), la [Toulouse School of Economics](#) et l'[Institut National Universitaire Champollion](#).

Les activités scientifiques de l'IMT couvrent un large spectre des mathématiques, depuis leurs aspects les plus théoriques jusqu'à leurs applications. Le laboratoire développe également de nombreuses interactions avec les autres disciplines scientifiques ainsi qu'avec le monde socio-économique. Il participe ainsi activement aux recherches à l'interface entre mathématiques, sciences des données, modélisation, optimisation et intelligence artificielle. L'IMT est notamment partenaire fondateur de l'[Artificial and Natural Intelligence Toulouse Institute](#) (ANITI) et contribue, aux côtés notamment de l'IRIT et du LAAS-CNRS, au développement de collaborations interdisciplinaires autour des mathématiques, de l'informatique et de l'intelligence artificielle.

D'un point de vue organisationnel, l'IMT est structuré en six équipes de recherche correspondant à différents grands domaines des mathématiques : *Analyse, Dynamique et Géométrie Complexe, Équations aux Dérivées Partielles, Géométrie Topologie Algèbre, Probabilités et Statistique et Optimisation*. Cette organisation permet de couvrir aussi bien les mathématiques fondamentales que les problématiques contemporaines liées à la modélisation, au traitement des données et à l'apprentissage automatique.

L'équipe [Statistique et Optimisation](#), au sein de laquelle s'inscrit ce stage, développe ses recherches autour de deux domaines fortement liés : l'estimation statistique et les méthodes d'optimisation. Leur interaction joue notamment un rôle central dans le développement et l'analyse des méthodes modernes d'apprentissage automatique.

Les travaux de l'équipe couvrent un ensemble étendu de problématiques comprenant notamment l'estimation et les tests statistiques, l'apprentissage statistique, l'inférence post-sélection, l'analyse de sensibilité, les processus gaussiens, le transport optimal ainsi que les problèmes inverses. Une partie importante des recherches concerne également l'optimisation continue et stochastique ainsi que l'étude des méthodes d'optimisation utilisées pour l'entraînement des réseaux de neurones. L'équipe s'intéresse par ailleurs aux questions de biais, d'équité et d'interprétabilité des modèles d'apprentissage, ainsi qu'à leurs applications dans différents domaines scientifiques.

Ces travaux s'inscrivent dans un environnement fortement interdisciplinaire. Les membres de l'équipe collaborent avec d'autres équipes de l'IMT ainsi qu'avec plusieurs acteurs de l'écosystème scientifique toulousain, notamment dans le cadre de l'institut ANITI. Cette proximité entre statistique, optimisation et intelligence artificielle permet de confronter les développements méthodologiques et théoriques aux problématiques rencontrées dans les systèmes modernes d'apprentissage automatique.

Dans le cadre de ce stage, j'ai été accueilli au sein de l'équipe *Statistique et Optimisation* de l'Institut de Mathématiques de Toulouse (IMT), où est rattachée Juliette Chevallier, l'une de mes responsables de stage. Les travaux ont également été menés en collaboration avec l'équipe *Machine Learning Integrity Safety Fairness Impact Trust* (MISFIT), rattachée au département Intelligence Artificielle de l'[Institut de Recherche en Informatique de Toulouse](#) (IRIT), au sein duquel travaille Emmanuelle Claeys, ma seconde responsable de stage.

Cette collaboration entre l'IMT et l'IRIT permet d'associer les compétences de l'équipe Statistique et Optimisation en statistique, optimisation et apprentissage automatique aux problématiques étudiées par MISFIT autour de l'intégrité, de la robustesse, de la sécurité et de la confiance dans les systèmes d'intelligence artificielle. Cette complémentarité est particulièrement adaptée au sujet du stage, consacré à l'étude et à l'évaluation de méthodes de détection d'images synthétiques.

Introduction

Depuis plusieurs années, l’intelligence artificielle occupe une place croissante dans la production de contenus numériques. Elle permet notamment de générer ou de modifier automatiquement des images, des vidéos ou des enregistrements audio. Les progrès récents de l’intelligence artificielle générative, en particulier des modèles de diffusion, ont fortement amélioré la qualité de ces contenus, au point de rendre certaines images synthétiques difficiles à distinguer visuellement d’images réelles.

Cette évolution soulève des enjeux importants concernant l’authenticité des contenus diffusés en ligne. Des images artificielles peuvent notamment être utilisées pour représenter des événements n’ayant jamais eu lieu ou diffuser de fausses informations. En mai 2023, une image représentant une prétendue explosion à proximité du Pentagone a ainsi largement circulé sur les réseaux sociaux. Bien que l’événement ait été fictif, sa diffusion a provoqué une brève perturbation des marchés financiers avant que l’information ne soit démentie (Marcelo, 2023). Cet exemple illustre l’intérêt de disposer de méthodes capables d’identifier automatiquement les contenus synthétiques.

La détection de telles images reste cependant difficile. Les modèles génératifs évoluent rapidement et les signatures laissées par un générateur peuvent différer de celles produites par un autre. Les performances d’un détecteur peuvent également diminuer lorsque les images subissent des transformations telles que la compression, le redimensionnement ou l’ajout de contenu (Corvi et al., 2022). Une méthode efficace doit donc distinguer les images réelles des images synthétiques tout en conservant des performances satisfaisantes dans des conditions variées.

Dans ce contexte, le premier objectif de ce stage a été d’étudier le fonctionnement des modèles de diffusion ainsi que les principales méthodes de détection d’images générées proposées dans la littérature. Nous nous sommes également appuyés sur le benchmark développé conjointement par le Pôle d’Expertise de la Régulation Numérique (PEReN) et VIGINUM (Bazin et al., 2025), qui fournit un cadre de référence pour comparer plusieurs détecteurs sur un ensemble varié d’images réelles et synthétiques. Ses résultats sont utilisés dans ce rapport comme références externes.

Le principal objectif expérimental consiste à étudier une approche de détection fondée sur la reconstruction d’une image à l’aide de CAD-I (Degeorge et al., 2025), un modèle de diffusion latente pré-entraîné sur ImageNet et utilisant une architecture de type Recurrent Interface Network (RIN) pour le débruitage. L’hypothèse étudiée est que les erreurs produites lors de cette reconstruction présentent des structures ou des signatures différentes selon que l’image initiale est réelle ou synthétique. Afin d’explorer cette hypothèse, cinq représentations sont considérées : pixel, latent, bruit, fréquence et ondelettes.

Pour chacune de ces représentations, plusieurs architectures de classification sont comparées. Les modèles retenus sont ensuite évalués pour plusieurs niveaux de bruitage α , plusieurs graines aléatoires et différents générateurs, afin de comparer différentes manières d’exploiter les erreurs de reconstruction dans un protocole expérimental commun.

Le rapport est organisé en trois chapitres. Le premier présente les modèles de diffusion et un état de l’art des méthodes de détection. Le deuxième étudie le benchmark PEReN–VIGINUM, notamment sous l’angle de la robustesse aux transformations et de la dépendance au générateur. Enfin, le troisième présente le protocole expérimental, les représentations extraites, les architectures étudiées et les résultats obtenus.

Chapitre 1

Modèles de diffusion et état de l’art sur les méthodes de détection

Afin de comprendre les informations exploitées pour détecter les images produites par les modèles de diffusion, il est nécessaire d’étudier au préalable leur mécanisme de génération. Ce chapitre présente donc leur fonctionnement ainsi que les propriétés qui en font aujourd’hui des modèles de référence pour la génération d’images. Nous introduirons ensuite plusieurs méthodes de détection proposées dans la littérature, regroupées selon la nature des informations qu’elles exploitent.

1.1 Modèles de diffusion

Avant l’essor des modèles de diffusion, les autoencodeurs variationnels (VAE) (Kingma and Welling, 2022) et les réseaux antagonistes génératifs (GAN) (Goodfellow et al., 2014) constituaient deux des principales familles de modèles utilisées pour la génération d’images. Les VAE apprennent une représentation latente probabiliste à partir de laquelle les données peuvent être reconstruites ou générées, tandis que les GAN reposent sur un apprentissage adversarial entre un générateur et un discriminateur. Ces deux familles de modèles sont présentées plus précisément dans l’annexe A.

Les modèles de diffusion reposent sur un principe différent. Une donnée est progressivement dégradée par l’ajout de bruit jusqu’à se rapprocher d’une distribution simple, généralement gaussienne. Un réseau de neurones apprend ensuite à inverser ce processus afin de reconstruire progressivement une donnée structurée à partir du bruit (Sohl-Dickstein et al., 2015; Ho et al., 2020). Contrairement aux GAN, leur entraînement ne repose donc pas sur la compétition entre deux réseaux. Ils sont également moins sujets à l’effondrement des modes, c’est-à-dire à la génération d’une diversité limitée d’échantillons.

Les modèles de diffusion ont été introduits par Sohl-Dickstein et al. (2015), puis popularisés pour la génération d’images par les travaux de Ho et al. (2020) sur les *Denoising Diffusion Probabilistic Models* (DDPM). Dhariwal and Nichol (2021) montrent notamment que des modèles de diffusion correctement dimensionnés et guidés peuvent atteindre une meilleure qualité d’image que certains GAN, tout en assurant une meilleure couverture de la distribution des données. Cette amélioration s’accompagne toutefois d’un coût de génération généralement plus important, le processus de débruitage étant réalisé de manière itérative.

Un modèle de diffusion probabiliste repose sur deux chaînes de Markov complémentaires (Weng, 2021) : un processus de diffusion direct, également appelé processus de bruitage, qui dégrade progressivement la donnée initiale, et un processus de diffusion inverse, ou processus de débruitage, qui apprend à retrouver progressivement une donnée structurée à partir du bruit.

Processus de diffusion direct

Soit

$$x_0 \in \mathbb{R}^{C \times H \times W}$$

une image, où C désigne le nombre de canaux et H et W ses dimensions spatiales.

Nous définissons le processus de diffusion direct, au cours duquel une faible quantité de bruit gaussien est ajoutée à l'image x_0 à chacune des T étapes, produisant une suite d'échantillons bruités $x_1, \dots, x_T$. Les échantillons obtenus perdent alors progressivement les caractéristiques de l'image initiale à mesure que le pas de temps $t \in \{1, \dots, T\}$ augmente (Weng, 2021). Ce processus est défini comme une chaîne de Markov fixe :

$$q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$$

où chaque transition ajoute une faible quantité de bruit gaussien :

$$q(x_t | x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}\right)$$

La notation $\mathcal{N}(z; \mu, \Sigma)$ désigne la densité d'une loi normale évaluée en z , de moyenne μ et de matrice de covariance Σ .

Les coefficients $\beta_1, \dots, \beta_T$, avec $\beta_t \in]0, 1[$, définissent le *noise schedule*, ou programme de variance, qui détermine la quantité de bruit ajoutée à chaque pas du processus de diffusion. Le facteur $\sqrt{1 - \beta_t}$ réduit la contribution de l'état précédent x_{t-1} afin que la variance du processus reste contrôlée (Ho et al., 2020).

En posant $\alpha_t = 1 - \beta_t$ et $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, il est possible d'exprimer directement la distribution de x_t conditionnellement à x_0 :

$$q(x_t | x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I}\right) \quad (1.1)$$

Cette propriété permet d'échantillonner directement n'importe quel état x_t sans calculer successivement les états $x_1, \dots, x_{t-1}$.

L'échantillonnage de x_t est une opération aléatoire, puisque x_t suit la loi conditionnelle définie dans l'équation (1.1). La technique de reparamétrisation consiste à isoler cette source d'aléa dans une variable auxiliaire $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$, indépendante des autres quantités du modèle. L'échantillon aléatoire x_t peut alors être écrit comme une fonction déterministe de x_0 , de $\bar{\alpha}_t$ et de ε :

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \quad (1.2)$$

De manière générale, cette réécriture permet de calculer les gradients nécessaires à l'entraînement du réseau. Dans le processus direct des DDPM, cette réécriture permet surtout de construire directement un état bruité x_t à partir de l'image initiale x_0 , sans simuler toutes les étapes intermédiaires (Weng, 2021).

Le premier terme correspond à l'information issue de l'image initiale, tandis que le second correspond au bruit ajouté. Lorsque t augmente, $\bar{\alpha}_t$ diminue. L'influence de x_0 devient donc progressivement plus faible, tandis que celle du bruit augmente (Ho et al., 2020).

Pour un nombre d'étapes T suffisamment grand et un programme de variance bien choisi, la distribution de x_T devient approximativement une loi gaussienne isotrope standard (Weng, 2021).

Le processus direct est entièrement fixé par le programme de variance. Il ne contient aucun paramètre appris ; son rôle est uniquement de définir les différents niveaux de dégradation auxquels le réseau de débruitage sera confronté pendant l'entraînement (Ho et al., 2020).

Processus de diffusion inverse

La génération d'une nouvelle image consiste à inverser le processus précédent. Le modèle part d'un bruit gaussien $x_T \sim \mathcal{N}(0, \mathbf{I})$, puis construit progressivement les états $x_{T-1}, \dots, x_0$.

Le modèle génératif est défini par (Weng, 2021) :

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t), \quad p(x_T) = \mathcal{N}(0, \mathbf{I})$$

où θ représente l'ensemble des paramètres appris du modèle.

Pour des pas de diffusion suffisamment faibles, chaque transition inverse peut être modélisée par une distribution gaussienne (Ho et al., 2020; Sohl-Dickstein et al., 2015) :

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 \mathbf{I})$$

où $\mu_\theta(x_t, t)$ désigne la moyenne paramétrée par le modèle et σ_t^2 la variance fixée pour la transition inverse.

La distribution inverse exacte $q(x_{t-1} | x_t)$ est difficile à calculer, car elle dépend de la distribution inconnue des données réelles. En revanche, lorsque l'image initiale x_0 est connue pendant l'entraînement, le postérieur $q(x_{t-1} | x_t, x_0)$ possède une expression analytique gaussienne (Weng, 2021) :

$$q(x_{t-1} | x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

avec :

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t, \quad \tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

L'apprentissage vise alors à rapprocher les transitions inverses prédites par p_θ de ces distributions postérieures connues pendant l'entraînement (Ho et al., 2020).

Les DDPM sont initialement entraînés en minimisant une borne variationnelle de la négative log-vraisemblance des données, comme montré dans la figure 1.1, notée L_{VLB} :

$$L_{\text{VLB}} = \mathbb{E}_q \left[D_{\text{KL}}(q(x_T | x_0) \| p(x_T)) + \sum_{t=2}^T D_{\text{KL}}(q(x_{t-1} | x_t, x_0) \| p_\theta(x_{t-1} | x_t)) - \log p_\theta(x_0 | x_1) \right]$$

où $D_{\text{KL}}(P \| Q)$ désigne la divergence de Kullback-Leibler¹ entre deux distributions P et Q .

Le premier terme mesure l'écart entre la distribution finale du processus direct et la loi gaussienne utilisée pour initialiser la génération. Les termes intermédiaires comparent les transitions inverses prédites aux postérieurs du processus direct. Enfin, le dernier terme correspond à la reconstruction de l'image à partir du premier état bruité.

1. La divergence de Kullback-Leibler est une mesure permettant de comparer deux distributions de probabilité P et Q , de densités respectives p et q . Elle vaut : $D_{\text{KL}}(P \| Q) = \int_{\mathcal{X}} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$ avec $\mathcal{X}$ l'espace où réside la variable x .

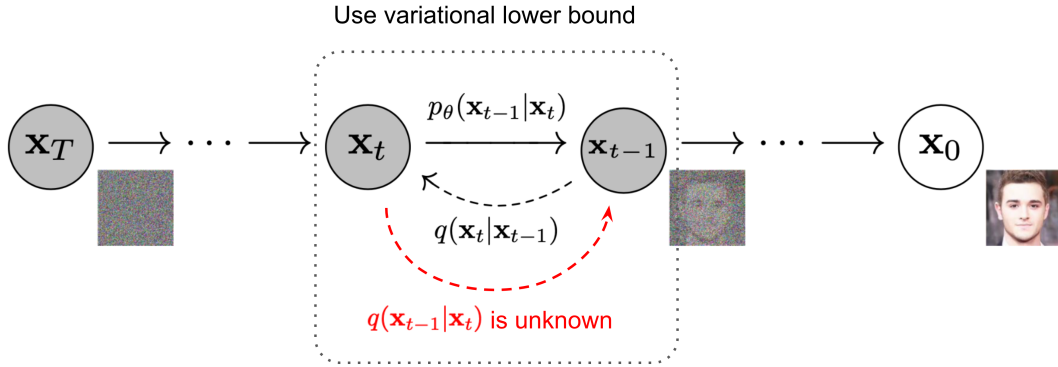


FIGURE 1.1 – Processus de diffusion inverse (Weng, 2021).

Ho et al. (2020) montrent qu'il n'est pas nécessaire de faire prédire directement au réseau l'image initiale x_0 ou la moyenne μ_θ . Une paramétrisation particulièrement efficace consiste à prédire le bruit ε ayant permis de construire x_t dans l'équation (1.2).

Le réseau, noté ε_θ , reçoit donc l'image bruitée x_t ainsi que le pas de temps t et produit une estimation du bruit, notée $\hat{\varepsilon} = \varepsilon_\theta(x_t, t)$. Généralement, le réseau utilisé ici est un U-Net (Ronneberger et al., 2015), détaillé sur la figure 1.2. Il repose sur une structure encodeur-décodeur. L'encodeur réduit progressivement les dimensions spatiales de l'entrée afin d'en extraire des cartes de caractéristiques. Le décodeur augmente ensuite progressivement ces dimensions pour produire la sortie. Des connexions directes, appelées *skip connections*, relient certaines couches de l'encodeur aux couches correspondantes du décodeur. Elles permettent au décodeur de récupérer des informations issues de l'encodeur, afin de préserver davantage de détails dans la sortie produite par le réseau.

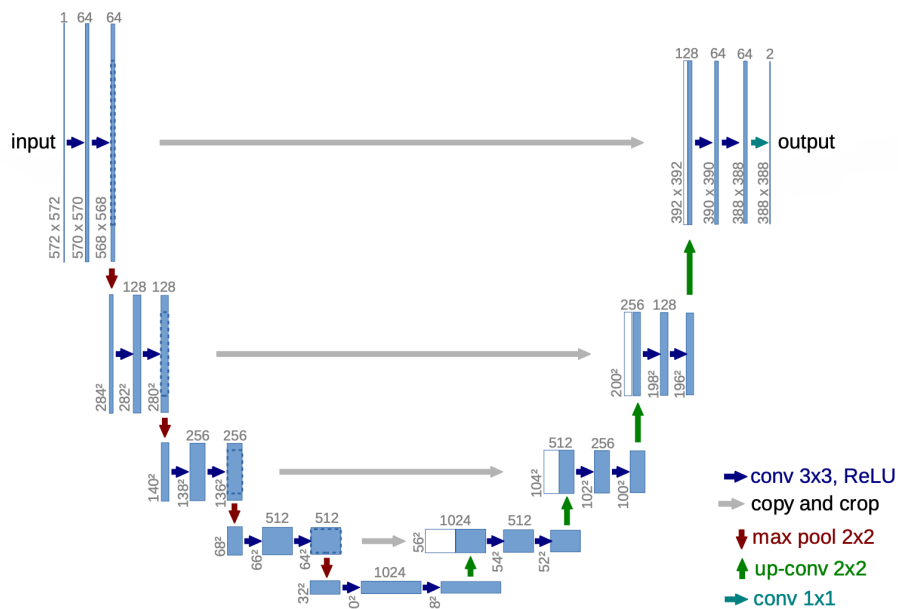


FIGURE 1.2 – Architecture d'un réseau U-Net : chaque rectangle bleu représente une carte de caractéristiques, avec le nombre de canaux indiqué en haut et ses dimensions (hauteur $\times$ largeur) en bas à gauche. Les flèches grises indiquent les *skip connexions* (Ronneberger et al., 2015).

La fonction objectif d'entraînement à minimiser devient donc (Weng, 2021) :

$$L^{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon} \left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2 \right]$$

Pour chaque itération d'entraînement, une image x_0 , un pas de temps t et un bruit ε sont échantillonnés. L'état bruité x_t est ensuite calculé directement à partir de l'équation (1.2). Le réseau est alors entraîné à prédire le bruit ε utilisé pour construire x_t .

À partir de la prédiction $\varepsilon_\theta(x_t, t)$, une estimation de l'image non bruitée peut être obtenue :

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

La moyenne du processus inverse peut également être paramétrée sous la forme :

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right)$$

Pendant la génération, chaque état x_{t-1} est obtenu à partir de l'état bruité x_t selon :

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t \xi$$

avec :

$$\xi \sim \mathcal{N}(0, \mathbf{I}) \quad \text{si } t > 1, \quad \xi = 0 \quad \text{si } t = 1$$

En répétant ce calcul de $t = T$ jusqu'à $t = 1$, le modèle transforme progressivement le bruit initial x_T en une image synthétique.

Modèles de diffusion latente

Les DDPM classiques appliquent la diffusion directement dans l'espace image, où chaque état x_t conserve les dimensions $C \times H \times W$ de l'image initiale. Cette approche étant coûteuse, notamment à haute résolution, les modèles de diffusion latente réalisent plutôt le processus dans un espace compressé de plus faible dimension, appelé espace latent, représenté par une variable latente z conservant les informations essentielles de l'image.

Les *Latent Diffusion Models* (LDM), introduits par [Rombach et al. \(2022\)](#), visent alors à réduire le coût des modèles de diffusion tout en préservant la qualité perceptuelle des images générées. Leur principe consiste à séparer deux tâches : la compression et la reconstruction perceptuelle des images, ainsi que la modélisation générative de leur contenu dans un espace latent.

Dans une première étape, un autoencodeur est entraîné à projeter une image x dans une représentation latente plus compacte $z_0 = E(x)$, où E désigne l'encodeur. Une approximation de l'image originale est ensuite reconstruite à partir de ce latent :

$$\hat{x} = D(z_0) = D(E(x))$$

où D désigne le décodeur.

L'objectif n'est pas de reconstruire exactement chaque valeur de pixel, mais de conserver les informations importantes de l'image. La compression élimine ainsi une partie des détails de haute fréquence peu perceptibles, tandis que la structure principale de l'image est conservée ([Rombach et al., 2022](#)).

Dans une seconde étape, le processus de diffusion est appliqué à la représentation latente z_0 . Le processus direct devient :

$$q(z_t | z_{t-1}) = \mathcal{N} \left(z_t; \sqrt{1 - \beta_t} z_{t-1}, \beta_t \mathbf{I} \right)$$

Comme dans l’espace des images, un état latent bruité peut être obtenu directement :

$$z_t = \sqrt{\alpha_t} z_0 + \sqrt{1 - \alpha_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

Le réseau U-Net apprend alors à prédire le bruit à partir de z_t :

$$L_{\text{LDM}} = \mathbb{E}_{t, z_0, \varepsilon} \left[\|\varepsilon - \varepsilon_\theta(z_t, t)\|^2 \right]$$

Pour générer une image, le processus inverse commence à partir d’un bruit latent $z_T \sim \mathcal{N}(0, \mathbf{I})$. Le réseau de diffusion transforme progressivement z_T en un latent z_0 . Une seule passe dans le décodeur D est ensuite nécessaire pour produire l’image finale.

La réduction de la résolution spatiale dans l’espace latent diminue fortement la quantité de calcul et la mémoire nécessaires au réseau de diffusion. Elle permet ainsi de travailler plus efficacement avec des images de haute résolution, tout en conservant une qualité visuelle élevée (Rombach et al., 2022).

1.2 Méthodes de détection d’images générées

La détection automatique de *deepfakes* peut être formulée comme un problème de classification binaire consistant à déterminer si une image d’entrée est réelle ou synthétique. Un détecteur doit donc être capable de classer correctement et d’obtenir de bonnes performances, mais également de reconnaître des images issues de générateurs absents de ses données d’entraînement.

Pour répondre à ces difficultés, plusieurs travaux récents utilisent les modèles de diffusion eux-mêmes comme outils d’analyse. Ces méthodes sont regroupées dans cette section selon la nature de l’information principalement exploitée : les erreurs calculées dans l’espace image, les représentations latentes et les prédictions du bruit fournies par le modèle de diffusion.

Nous présentons également d’autres approches complémentaires qui ne s’inscrivent pas directement dans les familles précédentes. Certaines exploitent encore un modèle de diffusion, tandis que d’autres reposent sur des informations forensiques différentes.

Méthodes basées sur les erreurs de reconstruction dans l’espace image

Une première famille de méthodes exploite la capacité de reconstruction des modèles génératifs. Elle repose sur l’hypothèse qu’un modèle de diffusion reconstruit mieux les images synthétiques que les images naturelles capturées par une caméra. Cette différence peut notamment s’expliquer par le fait que les images réelles contiennent des traces liées au processus d’acquisition, telles que le bruit du capteur ou certaines textures naturelles, qui peuvent être difficilement reproduites par un modèle génératif.

DIRE. La méthode DIRE (Wang et al., 2023), pour *Diffusion Reconstruction Error*, applique ce principe à la détection d’images générées par diffusion. Elle repose sur les *Denoising Diffusion Implicit Models* (DDIM) (Song et al., 2022). Un DDIM définit une trajectoire de diffusion qui peut être rendue déterministe, contrairement au processus stochastique généralement utilisé dans les DDPM. À partir de l’image d’entrée x , le DDIM ajoute progressivement du bruit afin d’obtenir un état fortement bruité x_T . À partir de cet état x_T , le processus est ensuite appliqué dans le sens inverse pour reconstruire une image $\hat{x}$.

DIRE calcule alors la différence absolue entre l’image initiale et sa reconstruction, puis fournit la carte d’erreur obtenue à un classifieur. Les auteurs observent que les images réelles tendent à présenter des erreurs de reconstruction plus importantes que les images générées par diffusion.

Cette méthode reste toutefois coûteuse, car DIRE nécessite plusieurs étapes successives d’inversion puis de reconstruction pour chaque image. Cette contrainte est propre à son utilisation d’une trajectoire DDIM complète : d’autres approches fondées sur les modèles de diffusion, dont

celle étudiée dans ce rapport, peuvent exploiter une étape de bruitage et de prédiction sans parcourir l’ensemble de la trajectoire de diffusion.

AEROBLADE. Une autre méthode, AEROBLADE (Ricker et al., 2024b), repose sur un principe similaire, mais utilise uniquement l’autoencodeur des modèles de diffusion latente. L’image d’entrée est encodée dans l’espace latent, puis reconstruite par le décodeur. La détection repose ensuite sur l’erreur entre l’image initiale et sa reconstruction : les images générées par un modèle de diffusion latente tendent à être mieux reconstruites que les images réelles.

Contrairement à DIRE, AEROBLADE n’utilise pas le processus de diffusion et ne nécessite aucun entraînement supplémentaire. Elle constitue ainsi une approche moins coûteuse, mais reste dépendante de l’autoencodeur utilisé et s’applique principalement aux images produites par des modèles de diffusion latente.

FIRE. Certaines différences entre les images réelles et synthétiques peuvent être peu visibles dans le domaine spatial, mais apparaître plus clairement dans le domaine fréquentiel. C’est précisément l’hypothèse exploitée par FIRE (Chu et al., 2025), pour *Frequency-guided Reconstruction Error*, qui combine l’analyse fréquentielle avec l’erreur de reconstruction.

Les auteurs observent que les fréquences intermédiaires des images réelles contiennent des informations particulièrement difficiles à reconstruire par un modèle de diffusion. FIRE cherche donc à identifier ces composantes fréquentielles, puis à les retirer de l’image afin d’obtenir une version filtrée. L’erreur de reconstruction est calculée à la fois sur l’image originale et sur cette version filtrée. Les deux cartes d’erreur obtenues sont ensuite transmises à un classifieur afin de distinguer les images réelles des images générées.

Méthodes exploitant les représentations latentes

Comme vu précédemment, les modèles de diffusion latente réalisent directement le processus de bruitage et de débruitage dans l’espace latent. Certaines méthodes de détection exploitent donc directement les représentations obtenues dans cet espace.

LaRE². LaRE² (Luo et al., 2025), pour *Latent Reconstruction Error*, cherche à réduire le coût des méthodes fondées sur une reconstruction complète comme DIRE. Pour une image x , l’encodeur produit une représentation latente z . Une perturbation de diffusion est ensuite appliquée afin d’obtenir un latent bruité :

$$z_t = \sqrt{\bar{\alpha}_t}z + \sqrt{1 - \bar{\alpha}_t}\varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

Le U-Net du modèle de diffusion prédit alors le bruit présent dans z_t . L’erreur entre le bruit réellement ajouté et celui prédit par le réseau est calculée :

$$L_\varepsilon = \varepsilon - \varepsilon_\theta(z_t, t)$$

La carte d’erreur latente est obtenue à partir du carré de cette erreur, calculé élément par élément. Dans LaRE², cette opération peut être répétée plusieurs fois avec différents bruits, puis les cartes obtenues sont moyennées.

Contrairement à DIRE, chaque erreur est ainsi obtenue à partir d’une seule étape de débruitage dans l’espace latent, ce qui réduit fortement le coût de calcul. La carte d’erreur obtenue n’est toutefois pas utilisée seule : elle sert à guider le raffinement de caractéristiques visuelles extraites par un réseau, à la fois spatialement et selon les canaux.

LATTE. La méthode LATTE (Vasilcoiu et al., 2025), pour *Latent Trajectory Embedding*, remet en question l'utilisation d'une représentation latente calculée à une étape unique. Le processus de diffusion étant séquentiel, les auteurs proposent d'exploiter des représentations latentes obtenues à plusieurs étapes du débruitage. Pour une image donnée, plusieurs pas de temps répartis le long du processus sont sélectionnés. À chacun de ces pas, un latent bruité est construit, puis une étape de débruitage est appliquée afin d'obtenir une représentation latente associée à ce niveau de bruit. L'ensemble de ces représentations forme alors une trajectoire latente.

Grâce à un mécanisme d'attention croisée², chaque représentation latente est ensuite combinée avec des caractéristiques visuelles extraites de l'image. Les différentes représentations obtenues sont finalement regroupées, puis combinées à une représentation globale de l'image avant d'être transmises à un classifieur.

LATTE exploite ainsi l'évolution des représentations latentes à différents niveaux du processus de débruitage, plutôt qu'une information provenant d'une seule étape. Cette approche nécessite cependant de traiter plusieurs pas de temps, contrairement aux méthodes fondées sur une unique étape de débruitage.

Méthodes fondées sur les prédictions de bruit

Une autre famille de méthodes s'intéresse au bruit prédit par le modèle de diffusion. Lorsqu'une image est progressivement bruitée, le U-Net estime le bruit associé à chaque état. L'idée est que ces prédictions diffèrent selon que l'image de départ est réelle ou synthétique, et peuvent donc être utilisées pour les distinguer.

DNF. DNF (Zhang and Xu, 2025), pour *Diffusion Noise Feature*, exploite ces prédictions à plusieurs pas de temps $t_1, \dots, t_K$ et obtient ainsi une séquence de bruits estimés :

$$\{\hat{\epsilon}_{t_1}, \hat{\epsilon}_{t_2}, \dots, \hat{\epsilon}_{t_K}\}$$

Plusieurs stratégies sont étudiées pour construire la représentation DNF à partir de cette séquence : utiliser le premier bruit estimé, le dernier ou la moyenne de l'ensemble des bruits. La représentation obtenue est ensuite transmise à un classifieur.

ANL. La méthode ANL (Qi et al., 2026), pour *Attention-guided Noise Learning*, cherche à améliorer la détection d'images produites par des modèles génératifs inconnus. Elle exploite pour cela le bruit prédit par un modèle de diffusion pré-entraîné.

Pour une image donnée, le modèle de diffusion produit une estimation du bruit. Cette estimation permet de construire une carte indiquant les zones de l'image où le bruit prédit est le plus important. Cette carte est ensuite utilisée pour modifier les caractéristiques extraites par le classifieur, afin que celui-ci accorde davantage d'importance à ces zones lors de la classification. L'objectif est ainsi de détecter des différences liées au processus de génération plutôt que de se concentrer uniquement sur le contenu visuel de l'image.

Autres méthodes

Les approches présentées précédemment se distinguent principalement par la nature de l'information extraite à l'aide d'un modèle de diffusion. D'autres travaux cherchent plutôt à améliorer la généralisation des détecteurs en agissant sur les données d'entraînement et sur la stratégie d'apprentissage.

2. L'attention croisée est un mécanisme permettant de mettre en relation deux ensembles de représentations différents afin d'identifier les informations les plus pertinentes de l'un par rapport à l'autre.

DRCT. DRCT (Chen et al., 2024), pour *Diffusion Reconstruction Contrastive Training*, est une stratégie d’entraînement conçue pour améliorer la généralisation à des modèles génératifs inconnus. Pour cela, les auteurs reconstruisent des images réelles et synthétiques à l’aide de modèles de diffusion. Les reconstructions d’images réelles conservent un contenu visuel proche de l’image d’origine, mais présentent également des traces introduites par le modèle de diffusion. Elles présentent ainsi à la fois des caractéristiques d’images réelles et synthétiques, ce qui les rend plus difficiles à classer.

DRCT repose sur un apprentissage contrastif, dont le principe est de rapprocher dans l’espace de représentation les images appartenant à une même classe et d’éloigner celles appartenant à des classes différentes. Cet apprentissage est réalisé à l’aide d’une perte contrastive, combinée à une perte de classification binaire permettant d’apprendre directement à distinguer les images réelles des images synthétiques. Cette méthode ne constitue donc pas un détecteur indépendant, mais une stratégie d’entraînement pouvant être appliquée à différentes architectures.

Une autre approche, proposée par Corvi et al. (2025), cherche également à améliorer la généralisation des détecteurs, mais dans le contexte des vidéos entièrement générées. Les auteurs cherchent à orienter le détecteur vers des traces de bas niveau laissées par les architectures génératives, plutôt que vers des différences liées au contenu des vidéos.

Pour cela, des vidéos réelles sont d’abord reconstruites à l’aide d’un autoencodeur. Leur contenu reste proche de celui des vidéos originales, mais des traces liées à l’architecture générative sont introduites. Les auteurs utilisent ensuite une augmentation de données fondée sur une décomposition en ondelettes : certaines bandes fréquentielles des vidéos synthétiques sont remplacées par celles des vidéos réelles, tandis que les composantes diagonales de moyennes à hautes fréquences sont conservées. Le détecteur est ainsi encouragé à exploiter ces dernières, qui se révèlent discriminantes et relativement résistantes à la compression vidéo.

Bien que cette méthode soit développée pour la vidéo, elle montre l’importance du choix des données et des transformations utilisées pendant l’entraînement pour améliorer la généralisation des détecteurs.

Méthode	Information exploitée	Principe	Principales limites
DIRE	Erreur de reconstruction pixel	Inversion puis reconstruction DDIM ; classification à partir de l'erreur entre l'image et sa reconstruction.	Coût élevé lié aux multiples étapes de diffusion.
AEROBLADE	Erreur de reconstruction pixel	Reconstruction uniquement avec l'autoencodeur d'un modèle de diffusion latente.	Dépendance à l'autoencodeur et aux modèles de diffusion latente.
FIRE	Pixel et fréquence	Compare les erreurs de reconstruction de l'image originale et d'une version filtrée fréquemment.	Nécessite une étape supplémentaire d'analyse fréquentielle.
LaRE ²	Erreur latente / bruit	Exploite l'erreur de prédiction du bruit dans l'espace latent pour guider des caractéristiques visuelles.	Dépendance au modèle de diffusion utilisé.
LATTE	Trajectoire latente	Exploite plusieurs représentations latentes obtenues à différents niveaux de bruitage.	Plusieurs pas de diffusion doivent être considérés.
DNF	Bruit prédit	Construit une représentation à partir des prédictions de bruit obtenues à plusieurs pas de temps.	Performances dépendantes des niveaux de bruitage sélectionnés.
ANL	Bruit prédit	Utilise le bruit prédit pour guider l'attention du classifieur vers les zones jugées discriminantes.	Ajoute un mécanisme d'attention et un classifieur appris.
DRCT	Données reconstruites	Utilise des reconstructions d'images réelles et synthétiques dans un apprentissage contrastif.	Nécessite la construction de nouvelles données d'entraînement.

TABLE 1.1 – Synthèse des principales méthodes de détection fondées sur l'exploitation de modèles de diffusion présentées dans cette section.

Chapitre 2

Benchmark PEReN-VIGINUM

Ce chapitre est basé sur l'article ([Pôle d'expertise de la régulation numérique \(PEReN\), 2025](#)).

2.1 Jeu de données et détecteurs évalués

Les méthodes présentées précédemment sont généralement évaluées à partir de jeux de données, de générateurs et de protocoles expérimentaux différents. Les performances rapportées dans les articles ne sont donc pas toujours directement comparables. Par ailleurs, certaines méthodes sont principalement évaluées dans des conditions idéales, sur des images peu ou pas transformées. Or, les contenus diffusés sur les plateformes en ligne peuvent subir une compression, un redimensionnement, un recadrage ou d'autres opérations de post-traitement. Ces transformations peuvent atténuer, voire supprimer, certaines traces forensiques exploitées par les détecteurs et ainsi dégrader leurs performances ([Corvi et al., 2022](#)).

Afin de mieux caractériser les performances des détecteurs de *deepfakes* dans des conditions proches de celles rencontrées sur les plateformes en ligne, le PEReN et VIGINUM ont développé une interface standardisée open-source. Celle-ci permet d'intégrer plusieurs détecteurs dans un cadre commun, d'uniformiser leur exécution et de mesurer leurs performances sur un même ensemble de données.

Le benchmark repose sur un jeu de 20 000 images contenant à la fois des images réelles et des images synthétiques provenant de différents GAN et modèles de diffusion. Les auteurs distinguent les portraits des images généralistes. Les images réelles ainsi qu'une partie des images synthétiques proviennent de différents jeux de données académiques, tandis qu'une autre partie a été directement générée dans le cadre du projet. Le détail du jeu de données est présenté dans le tableau [2.1](#).

Le benchmark intègre également différents détecteurs issus de la littérature. Leurs codes sont conservés autant que possible dans leur forme originale, avec seulement certaines adaptations techniques nécessaires à leur intégration dans l'interface commune.

Le tableau [2.2](#) résume les données et les modèles génératifs utilisés pour leur entraînement. Ces informations sont particulièrement importantes pour l'analyse de la généralisation, car elles permettent de distinguer les générateurs déjà rencontrés par un détecteur de ceux qui lui sont inconnus.

Les détecteurs présentent ainsi des conditions d'entraînement très différentes. Certains ont principalement été confrontés à des images générées par des GAN, tandis que CoDE a été entraîné sur plusieurs modèles de diffusion. Cette diversité doit être prise en compte lors de l'interprétation de leurs performances sur le jeu de données du benchmark, qui contient une proportion importante d'images générées par des modèles de diffusion.

Dataset	Nb	Type	Nature	Modèles génératifs
Conceptual Captions (Sharma et al., 2018)	6000	Image généraliste	Réelle	–
FFHQ (Karras et al., 2019)	5000	Portrait	Réelle	–
DatasetForDream (CYANEX 1702, 2024)	500	Image généraliste	Synthétique	Midjourney (< v5) (Midjourney, 2026)
midjourney-images (ehristoforu, 2024b)	500	Image généraliste	Synthétique	Midjourney ($\geq$ v5)
dalle-3-images (ehristoforu, 2024a)	500	Image généraliste	Synthétique	DALL-E 3 (Betker et al., 2023)
SFHQ-T2I (Beniaguev, 2024)	1500	Portrait	Synthétique	DALL-E 3 ; Stable Diffusion XL (Podell et al., 2024) ; FLUX.1-dev (Black Forest Labs, 2024)
ELSA-D3 (Baraldi et al., 2024)	1500	Image généraliste	Synthétique	Stable Diffusion XL ; Stable Diffusion 2.1 (Rombach et al., 2022) ; DeepFloyd IF (DeepFloyd, 2023)
DIMD (Corvi et al., 2022)	334	Portrait	Synthétique	StyleGAN3 (Karras et al., 2021)
DIMD	166	Image généraliste	Synthétique	StyleGAN3
AIS-4SD (Pôle d'expertise de la régulation numérique (PEReN), 2025)	2000	Portrait	Synthétique	Stable Diffusion XL ; Stable Diffusion 2.1 ; Stable Diffusion 3 Medium (Esser et al., 2024) ; Stable Diffusion 3.5 Large (Stability AI, 2024)
AIS-4SD	2000	Image généraliste	Synthétique	Stable Diffusion XL ; Stable Diffusion 2.1 ; Stable Diffusion 3 Medium ; Stable Diffusion 3.5 Large

TABLE 2.1 – Jeux de données utilisés dans l'étude.

Détecteur	Données d'entraînement	Générateurs présents à l'entraînement
CNNDetect (Wang et al., 2020)	LSUN	ProGAN
FreqDetect (Frank et al., 2020)	FFHQ	StyleGAN2
DIMD (Corvi et al., 2022)	COCO, ImageNet, UCID	Latent Diffusion
ViT-L/14 (Ojha et al., 2024)	LSUN	ProGAN
gragnaniello_progan (Ricker et al., 2024a)	LSUN	ProGAN
gragnaniello_stylegan2 (Ricker et al., 2024a)	LSUN	StyleGAN2
mandelli (Ricker et al., 2024a)	FFHQ, MetFaces, AFHQ2	StyleGAN2, StarGAN, Taming Transformers, FaceVid2Vid
CoDE (Baraldi et al., 2024)	LAION-400M	Stable Diffusion XL, 1.4, 2.1 ; DeepFloyd
Xception, UCF, SPSL, Recce (Yan et al., 2023)	FaceForensics++	Deepfakes, Face2Face, FaceSwap, NeuralTextures

TABLE 2.2 – Principales données d'entraînement des détecteurs intégrés au benchmark PEReN-VIGINUM.

2.2 Performances, robustesse et généralisation

Les auteurs évaluent tout d’abord les détecteurs sur les images originales selon différentes métriques : la *balanced accuracy*, l’AUC, la précision, le rappel et le score F1. Les définitions de ces métriques sont présentées dans l’annexe B.1.

Les résultats présentés dans la figure 2.1 montrent que CoDE et DIMD obtiennent les meilleures performances globales, tandis que plusieurs autres détecteurs présentent des performances proches du hasard, avec notamment une AUC proche de 0,50.

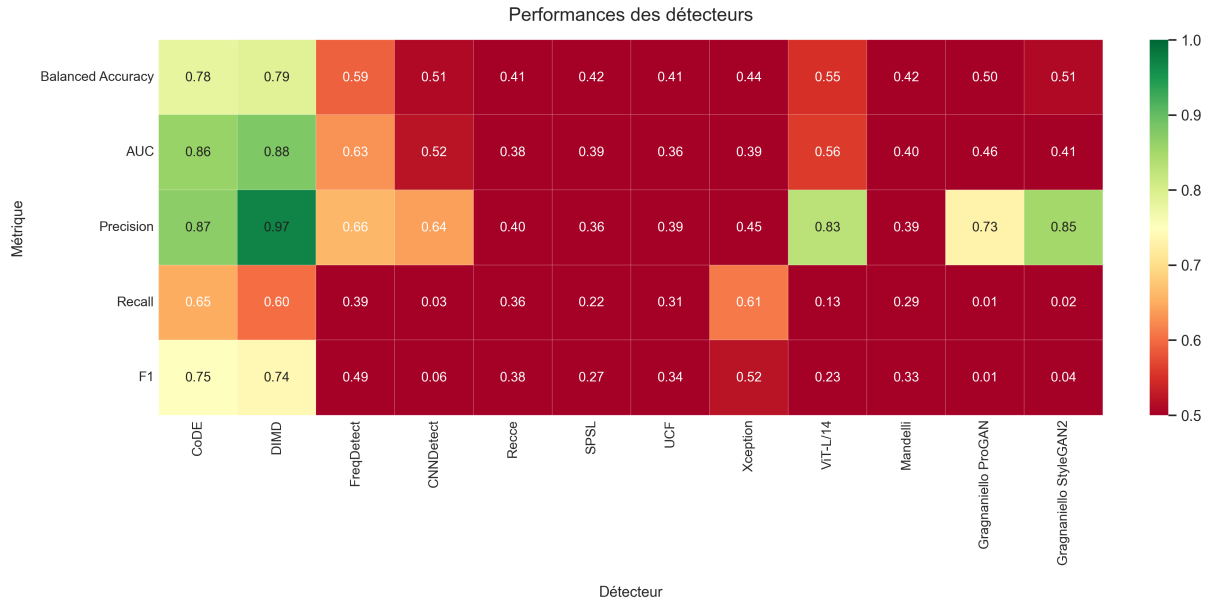


FIGURE 2.1 – Performances des détecteurs évalués.

Ces différences peuvent notamment être mises en relation avec les données d’entraînement présentées dans le tableau 2.2. Plusieurs détecteurs ont été entraînés uniquement ou principalement sur des GAN ou sur des méthodes de manipulation de visages, alors que le jeu de test contient de nombreux modèles de diffusion. Une partie de la baisse de performance peut donc être liée au changement de distribution entre les données d’entraînement et celles utilisées pour l’évaluation.

Parmi les méthodes obtenant les meilleurs résultats, **CoDE** (Baraldi et al., 2024) repose sur un apprentissage contrastif visant à construire un espace de représentation séparant les images réelles des images synthétiques. **DIMD** (Corvi et al., 2022) exploite quant à lui des traces laissées par le processus de génération, notamment dans les résidus de bruit et dans les composantes fréquentielles.

Le benchmark étudie ensuite la robustesse des détecteurs aux transformations couramment rencontrées lors de la diffusion d’images en ligne. Les auteurs appliquent notamment une compression JPEG, un ajout de contenu textuel, des filtres esthétiques tels que le flou, le niveau de gris ou le sépia, ainsi qu’un redimensionnement. Une transformation « réseaux sociaux » combine plusieurs de ces opérations.

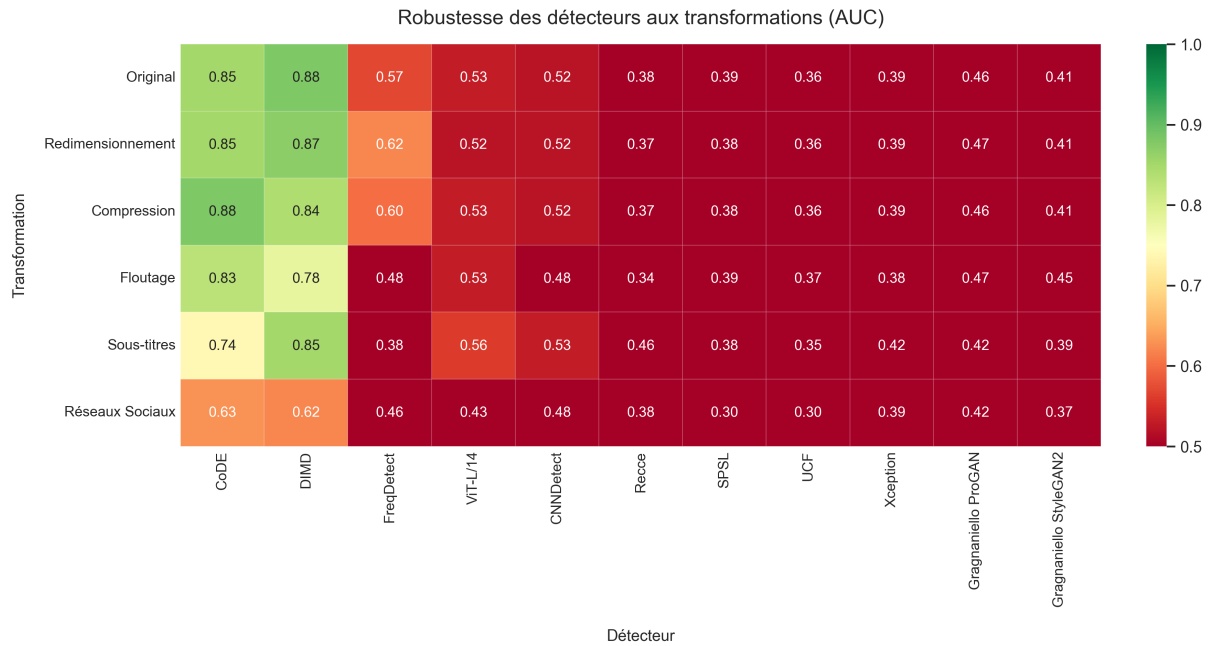


FIGURE 2.2 – Robustesse des détecteurs à différentes transformations.

Les performances diminuent généralement après transformation des images. **La transformation « réseaux sociaux », qui combine plusieurs altérations, fait partie des cas les plus défavorables.** Ces résultats montrent qu'un détecteur efficace sur les images originales ne conserve pas nécessairement cette efficacité après leur mise en ligne ou leur recompression.

Enfin, les performances sont analysées selon le modèle ayant généré chaque image. Cette comparaison est particulièrement importante pour évaluer la capacité de généralisation d'un détecteur.

Les figures 2.3a et 2.3b présentent les performances de CoDE et DIMD selon les différents jeux de données et modèles génératifs.

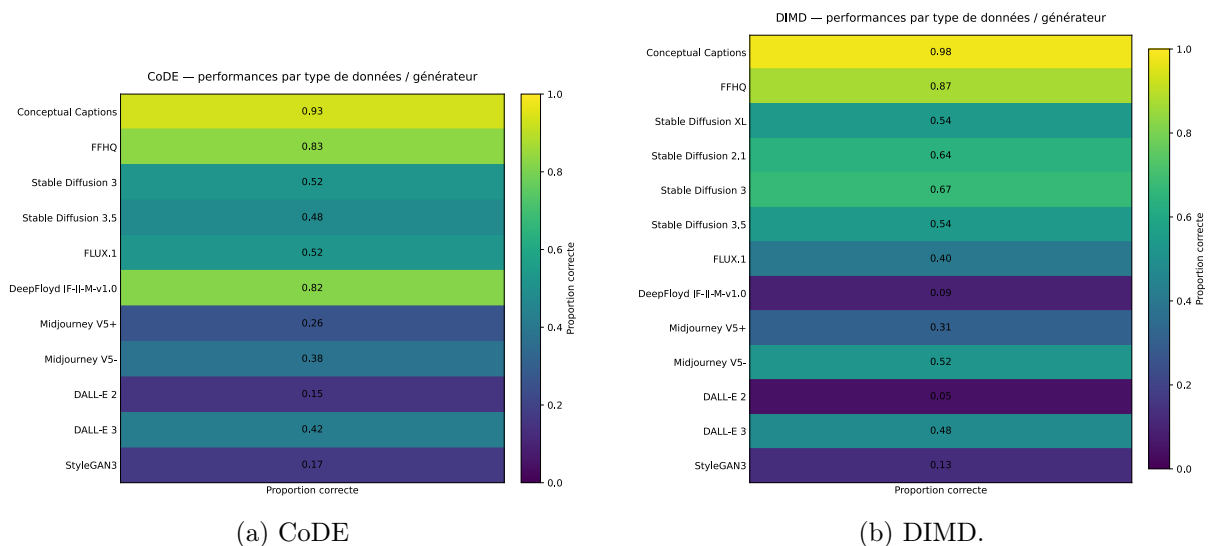


FIGURE 2.3 – Performances de CoDE et DIMD selon la nature des données et le modèle génératif, d'après les résultats du PEReN et VIGINUM.

Les résultats de CoDE mettent en évidence une dépendance importante au générateur. Le tableau 2.2 montre que son entraînement comprend notamment Stable Diffusion XL, Stable

Diffusion 1.4, Stable Diffusion 2.1 et DeepFloyd. Or, plusieurs de ses meilleures performances sont précisément obtenues sur des modèles appartenant à cet ensemble, tandis qu’elles diminuent sur d’autres générateurs.

Cette observation est particulièrement intéressante, car elle montre qu’une bonne performance globale peut en partie refléter la capacité du détecteur à reconnaître les traces des modèles déjà rencontrés pendant son entraînement, plutôt qu’une propriété commune à toutes les images synthétiques.

DIMD présente un comportement différent. D’après le tableau 2.2, son entraînement repose sur des images générées par un modèle de type Latent Diffusion. Il conserve néanmoins des capacités de détection sur plusieurs autres générateurs qui ne sont pas explicitement présents dans ses données d’entraînement. Ses performances restent variables et diminuent fortement pour certains modèles, mais cette observation suggère une capacité de généralisation plus importante que celle de plusieurs autres détecteurs évalués.

La comparaison avec les autres méthodes renforce également ce constat. CNNDetect, ViT-L/14 et gragnaniello_progan ont par exemple été entraînés sur ProGAN, tandis que FreqDetect et gragnaniello_stylegan2 reposent sur StyleGAN2. De même, Xception, UCF, SPSL et Recce ont été entraînés sur des manipulations de visages issues de FaceForensics++. Leur faible performance sur un benchmark largement composé d’images issues de modèles de diffusion est donc cohérente avec une spécialisation aux familles de générateurs rencontrées durant l’apprentissage. Elle ne permet cependant pas, à elle seule, d’établir que le changement de générateur en est l’unique cause.

Ces résultats soulignent ainsi une difficulté centrale de la détection d’images synthétiques : **un détecteur performant sur les générateurs vus pendant son entraînement ne généralise pas nécessairement à des générateurs inconnus**. L’évaluation d’une méthode uniquement sur des modèles déjà représentés dans ses données d’apprentissage peut donc donner une vision trop optimiste de ses performances.

Le benchmark PEReN–VIGINUM met finalement en évidence deux difficultés principales : **la robustesse aux transformations et la généralisation à de nouveaux générateurs**. Il constitue ainsi un point de référence pour notre étude. Nous reprendrons notamment l’analyse des performances globales ainsi que leur comparaison selon les différents générateurs, afin d’étudier dans quelle mesure les performances de notre approche dépendent du modèle génératif considéré.

Les résultats publiés par le PEReN et VIGINUM sont toutefois utilisés comme référence externe. Les détecteurs du benchmark n’ayant pas été réexécutés dans notre protocole, les comparaisons avec nos résultats ne constituent pas une reproduction expérimentale directe.

Chapitre 3

Protocole expérimental et résultats obtenus

Les méthodes de détection de deepfakes présentées dans les chapitres précédents montrent que les détecteurs peuvent perdre fortement en performance lorsqu'ils sont confrontés à de nouveaux générateurs ou à des images transformées. Nous avons donc cherché à identifier des informations qui dépendent moins d'un générateur particulier. Notre hypothèse est que la reconstruction d'une image par un modèle de diffusion produit des erreurs dont la structure diffère selon que l'image initiale est réelle ou synthétique.

Cependant, nous ne savions pas initialement dans quel espace ces différences seraient les plus visibles, ni quelle intensité de bruit ou quelle architecture de classification serait la plus adaptée. Nous avons donc choisi d'étudier plusieurs *features* : pixel, latent, bruit, fréquence et ondelettes. Pour chacune d'elles, différentes architectures ont été comparées afin d'identifier les choix les plus pertinents pour la détection des *deepfakes*.

Ce chapitre présente d'abord la mise en œuvre et le protocole expérimental, puis le processus d'extraction des différentes cartes d'erreur. Nous exposerons ensuite les architectures choisies, avant de présenter les résultats obtenus. Afin de faciliter la compréhension de ce chapitre, l'annexe [B.2](#) rappelle les principales notions relatives aux réseaux de neurones, notamment aux réseaux convolutionnels.

3.1 Mise en œuvre et protocole expérimental

3.1.1 Environnement de calcul

Pour nos expériences, nous avons utilisé conjointement les environnements Google Colab et Jean Zay. Google Colab a servi à réaliser la majorité des développements et des expérimentations, notamment les premiers tests de fonctionnement, les analyses sur des sous-échantillons, ainsi que les entraînements sur le jeu de données complet. Cette plateforme permet d'accéder simplement à un GPU et offre ainsi un environnement adapté au développement et à l'exécution de différentes expériences.

L'extraction des différentes cartes sur l'ensemble du jeu de données a été réalisée sur la plateforme de calcul Jean Zay. Les programmes y sont exécutés sur des serveurs équipés de GPU performants et lancés à l'aide de Slurm, le gestionnaire de tâches de la plateforme. Celui-ci attribue à chaque exécution les ressources demandées, telles que le nombre de GPU, de processeurs ainsi que la durée maximale d'exécution. L'ensemble des scripts a été développé en Python à l'aide de la bibliothèque PyTorch.

3.1.2 Entraînement et évaluation

Les 20 000 images sont réparties en trois sous-ensembles : 14 000 images pour l'entraînement, 3 000 pour la validation et 3 000 pour le test. Cette partition est conservée pour l'ensemble des représentations et des architectures afin de garantir des comparaisons homogènes. Chaque configuration est entraînée avec trois graines aléatoires (42, 123 et 3407), de manière à mesurer la variabilité liée à l'initialisation et au processus d'entraînement. Pour chaque représentation, les paramètres de normalisation sont estimés uniquement à partir du jeu d'entraînement, puis appliqués sans modification aux jeux de validation et de test, afin d'éviter toute fuite d'information.

Les modèles sont entraînés avec l'optimiseur AdamWScheduleFree (Defazio et al., 2024). Un *scheduler* est un mécanisme qui fait évoluer le taux d'apprentissage au cours de l'entraînement selon une règle prédéfinie ou en fonction des performances observées. L'approche *Schedule-Free* permet de s'affranchir de l'utilisation d'un tel mécanisme explicite. Une alternative aurait par exemple consisté à utiliser AdamW avec un *scheduler* classique, tel qu'une décroissance cosinus du taux d'apprentissage ou une réduction déclenchée par la stagnation des performances sur le jeu de validation. La fonction de perte utilisée est l'entropie croisée binaire appliquée aux logits. Pour une image i de label $y_i \in \{0, 1\}$ et un logit s_i produit par le réseau, elle s'écrit :

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\sigma(s_i)) + (1 - y_i) \log(1 - \sigma(s_i))]$$

où σ désigne la fonction sigmoïde.

Afin de limiter le surapprentissage, un mécanisme d'*early stopping* interrompt l'entraînement lorsque la performance sur le jeu de validation ne s'améliore plus pendant cinq époques. L'entraînement peut également être arrêté lorsqu'un écart supérieur à 0,1 entre les Balanced Accuracy d'entraînement et de validation persiste.

La sélection des architectures et des hyperparamètres est réalisée exclusivement à partir du jeu de validation. Le seuil de décision est lui aussi déterminé sur ce jeu, puis conservé sans modification pour l'évaluation finale. Le jeu de test n'intervient donc à aucun moment dans la sélection des modèles et est réservé à l'estimation finale de leurs performances.

3.2 Extraction des cartes d'erreur

L'objectif de l'étape d'extraction est de construire, pour chaque image du jeu de données VIGINUM, différentes représentations obtenues à partir d'une reconstruction par un modèle de diffusion. Ces représentations exploitent l'espace image, l'espace latent ainsi que le bruit prédit par le modèle.

L'extraction repose sur le modèle préentraîné CAD-I (Degeorge et al., 2025), initialement conçu pour la génération d'images à partir de descriptions textuelles. Entraîné sur ImageNet (Deng et al., 2009), il utilise une architecture de type *Recurrent Interface Network* (RIN) (Jabri et al., 2023) comme réseau de débruitage.

CAD-I fonctionne dans un espace latent. Une image de dimensions $3 \times 256 \times 256$ est encodée par le VAE associé à Stable Diffusion 1.5 (Rombach et al., 2022) en un tenseur de dimensions $4 \times 32 \times 32$. Un bruit gaussien est ensuite ajouté à cette représentation, puis le réseau RIN estime le bruit présent dans le latent bruité.

La prédiction est également conditionnée par une représentation textuelle obtenue à l'aide de FLAN-T5 XL (Chung et al., 2022). Dans nos expériences, le *prompt* générique **a photo** est utilisé pour toutes les images. Le conditionnement textuel est donc identique pour l'ensemble du jeu de données et ne contient aucune information sur la classe réelle ou synthétique.

Les paramètres de CAD-I et du VAE restent gelés pendant toute l'extraction. Le modèle est uniquement utilisé pour produire les quantités intermédiaires nécessaires à la construction des différentes représentations. CAD-I a été retenu car il est pré-entraîné et publiquement disponible.

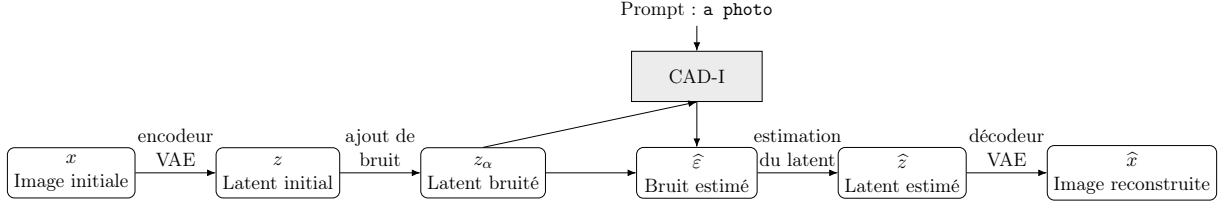


FIGURE 3.1 – Pipeline de reconstruction utilisé pour extraire les différentes quantités étudiées.

Son utilisation permet de séparer l'entraînement du modèle de diffusion de celui du détecteur, tout en donnant accès aux représentations latentes et aux prédictions de bruit nécessaires à notre étude, pour un coût de calcul raisonnable. Ce choix reste néanmoins expérimental et les résultats peuvent dépendre du modèle de diffusion utilisé.

Soit une image initiale x . Sa représentation latente est définie par

$$z(x) := E(x)$$

où E désigne l'encodeur du VAE.

Une réalisation de bruit gaussien ε est ensuite ajoutée au latent selon

$$z_\alpha(x, \varepsilon) := \sqrt{\alpha} z(x) + \sqrt{1 - \alpha} \varepsilon$$

où α contrôle l'intensité de la perturbation. Une valeur proche de 1 conserve davantage le latent initial, tandis qu'une valeur plus faible augmente la contribution du bruit.

En notant C_θ le réseau CAD-I et c le conditionnement textuel, le bruit estimé est donné par

$$\hat{\varepsilon}_\alpha(x, \varepsilon) := C_\theta(z_\alpha(x, \varepsilon), \alpha, c)$$

Cette prédiction permet d'estimer le latent initial :

$$\hat{z}_\alpha(x, \varepsilon) := \frac{z_\alpha(x, \varepsilon) - \sqrt{1 - \alpha} \hat{\varepsilon}_\alpha(x, \varepsilon)}{\sqrt{\alpha}}$$

L'image reconstruite est finalement obtenue à l'aide du décodeur D du VAE :

$$\hat{x}_\alpha(x, \varepsilon) := D(\hat{z}_\alpha(x, \varepsilon))$$

Les dépendances à x , α et ε pourront être omises par la suite lorsqu'elles ne présentent aucune ambiguïté. Le pipeline complet est résumé dans la figure 3.1.

Pour chaque image x et chaque valeur de α , une seule réalisation du bruit ε est utilisée. Les différentes représentations sont ainsi construites à partir des mêmes représentations intermédiaires : z , ε , $\hat{\varepsilon}_\alpha$, $\hat{z}_\alpha$ et $\hat{x}_\alpha$.

Pour chacune des trois représentations considérées ici : pixel, latente et bruit, l'étape d'extraction conserve systématiquement trois éléments : la représentation initiale, la représentation reconstruite ou prédite, et leur différence signée.

Ces trois éléments constituent les données disponibles pour l'étape d'apprentissage. Leur utilisation dépend ensuite de l'architecture considérée : certains modèles utilisent uniquement la différence, tandis que d'autres exploitent conjointement la représentation initiale et sa reconstruction, ou encore les trois représentations.

Carte pixel : Dans l'espace image, on conserve le triplet $(x, \hat{x}_\alpha, x - \hat{x}_\alpha)$, correspondant respectivement à l'image originale, à l'image reconstruite et à leur différence signée.

Carte latente : Dans l'espace latent, on conserve le triplet $(z, \hat{z}_\alpha, z - \hat{z}_\alpha)$, correspondant respectivement au latent initial, au latent reconstruit et à leur différence signée.

Carte de bruit : Dans l'espace du bruit, on conserve le triplet $(\varepsilon, \hat{\varepsilon}_\alpha, \varepsilon - \hat{\varepsilon}_\alpha)$, correspondant respectivement au bruit ajouté au latent, au bruit prédit par CAD-I et à leur différence signée.

La figure 3.2 présente un exemple des différentes représentations obtenues.

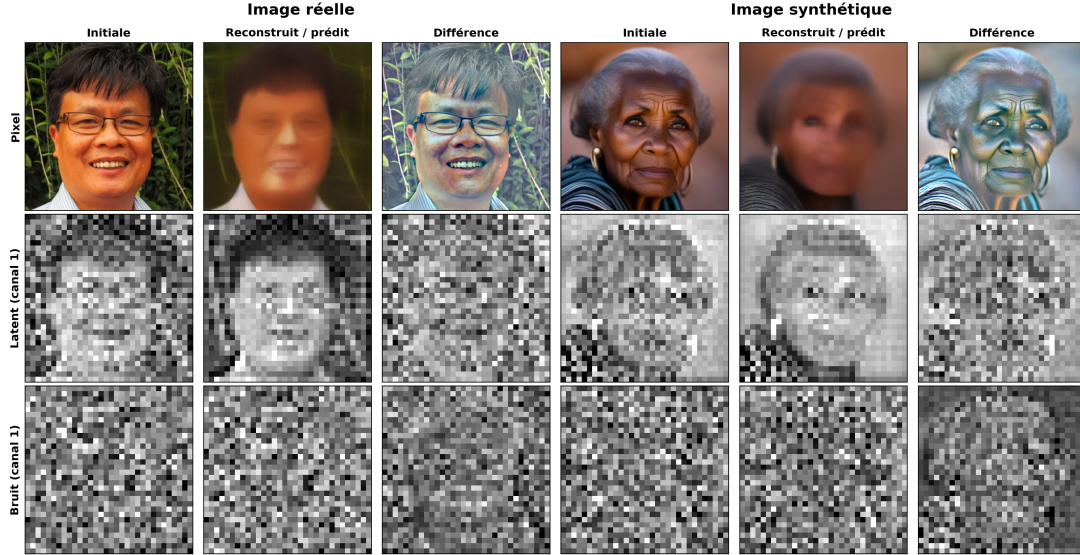


FIGURE 3.2 – Comparaison des représentations extraites à partir d’une image réelle (colonnes de gauche) et d’une image synthétique (colonnes de droite) pour $\alpha = 0,55$. Pour chaque image, les lignes présentent successivement la représentation pixel, la représentation latente et la représentation bruit. Les colonnes correspondent respectivement à la représentation initiale, à sa reconstruction ou prédiction par le modèle de diffusion, puis à leur différence. Pour les représentations latente et bruit, seul le premier canal est affiché. Les cartes sont normalisées uniquement à des fins de visualisation ; leur intensité ne doit donc pas être interprétée comme une comparaison directe de l’amplitude des erreurs entre les deux images.

3.3 Choix des architectures

À partir des représentations extraites, différentes stratégies de classification ont été étudiées. Pour les *features* pixel, latent et bruit, les architectures exploitent selon les cas la différence seule, la représentation initiale et sa reconstruction, ou l’ensemble des trois éléments extraits. Les *features* fréquence et ondelettes sont quant à elles obtenues par des transformations supplémentaires appliquées à Δ_{pixel} . Chaque architecture est ainsi adaptée à la nature de la représentation considérée.

Les schémas présentés dans cette section sont volontairement simplifiés afin de mettre en évidence le cheminement de l’information et les principales différences entre les architectures. Une description plus détaillée de leur structure et de leurs différentes couches est fournie en annexe B.3.

Approches basées sur la *feature* pixel

Pour la *feature* pixel, la classification repose sur la différence signée Δ_{pixel} . Sa structure spatiale étant proche de celle d’une image, des architectures convolutionnelles ont été privilégiées afin d’exploiter les motifs locaux présents dans les erreurs de reconstruction.

Deux approches complémentaires ont été comparées, comme illustré sur la figure 3.3. Le CNN-D1 constitue une approche entraînée entièrement à partir des cartes pixel et permet d'évaluer directement l'information discriminante contenue dans Δ_{pixel} . À l'inverse, le ResNet18 (He et al., 2016) pré-entraîné sur ImageNet permet d'étudier si des caractéristiques visuelles apprises sur des images naturelles peuvent être transférées à l'analyse des erreurs de reconstruction.

Deux niveaux d'adaptation du ResNet18 sont considérés : un *backbone* entièrement gelé, afin d'évaluer la pertinence des représentations pré-entraînées sans les modifier, et un *fine-tuning* partiel du dernier bloc résiduel, permettant d'adapter les caractéristiques de haut niveau à la distribution particulière des cartes pixel. Cette comparaison permet ainsi d'opposer une représentation apprise spécifiquement pour la tâche à l'utilisation de caractéristiques pré-entraînées, avec ou sans adaptation.

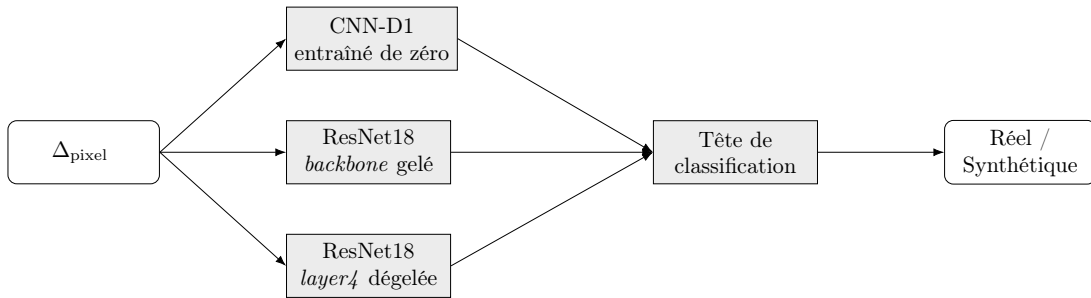


FIGURE 3.3 – Architectures étudiées pour exploiter la différence pixel : CNN-D1, ResNet18 avec *backbone* gelé et ResNet18 avec dégel du dernier bloc résiduel.

Approches basées sur la *feature* latente

Les représentations latentes étant plus compactes spatialement que les cartes pixel, les architectures étudiées cherchent principalement à exploiter la relation entre le latent initial z et sa reconstruction $\hat{z}_\alpha$. Deux stratégies complémentaires ont ainsi été considérées, comme illustré sur la figure 3.4.

La première repose sur une fusion précoce des informations. Les tenseurs z , $\hat{z}_\alpha$ et leur différence absolue sont concaténés :

$$X_{\text{latent}} = \text{concat}(z, \hat{z}_\alpha, |z - \hat{z}_\alpha|)$$

Deux variantes inspirées du U-Net sont comparées. *LatentUNetSmall*, limité à l'encodeur, permet d'évaluer si une extraction hiérarchique compacte suffit à caractériser les écarts latents. *LatentUNetLarge*, qui conserve l'encodeur et le décodeur, permet d'étudier l'intérêt de préserver davantage d'information spatiale avant la classification.

La seconde stratégie utilise une architecture siamoise dans laquelle z et $\hat{z}_\alpha$ sont traités séparément par deux branches partageant les mêmes paramètres. Cette approche vise à apprendre une représentation commune des deux latents avant de comparer explicitement leurs caractéristiques. Les représentations obtenues, leur différence absolue et leur produit terme à terme sont ensuite combinés avant la classification.

Approche basée sur la *feature* bruit

Pour la *feature* bruit, deux stratégies complémentaires ont été étudiées afin d'exploiter la relation entre le bruit injecté ε et le bruit prédit $\hat{\varepsilon}_\alpha$.

La première repose sur la différence signée $\Delta_{\text{bruit}} = \varepsilon - \hat{\varepsilon}_\alpha$. Les écarts entre les deux bruits pouvant être faibles et localisés, l'architecture NoiseSRMResidualCNN combine des filtres SRM (*Spatial Rich Model*) à un CNN résiduel afin de renforcer les variations locales et les composantes de haute fréquence susceptibles d'être discriminantes. Trois filtres SRM fixes sont appliqués aux

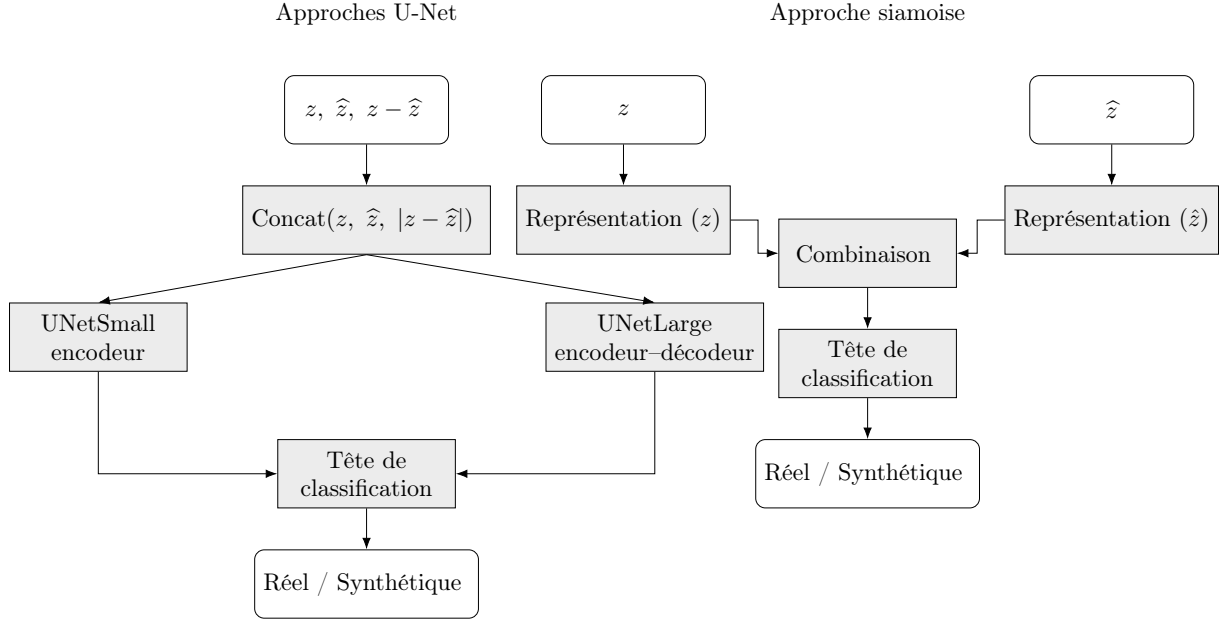


FIGURE 3.4 – Architectures étudiées pour exploiter l'information latente.

quatre canaux de Δ_{bruit} , puis leurs réponses sont concaténées aux canaux d'origine avant d'être traitées par le réseau résiduel (Fridrich and Kodovsky, 2012).

La seconde stratégie utilise un réseau convolutionnel siamois. Contrairement à l'approche précédente, ε et $\hat{\varepsilon}_\alpha$ sont traités séparément par deux branches partageant les mêmes paramètres. Cette architecture permet ainsi d'apprendre une représentation commune des deux bruits avant de comparer explicitement leurs caractéristiques. Elle suit le même principe que l'architecture siamoise utilisée pour les représentations latentes.

Ces deux approches permettent donc de comparer l'exploitation directe de l'erreur de prédiction du bruit à une comparaison apprise entre le bruit injecté et le bruit prédit.

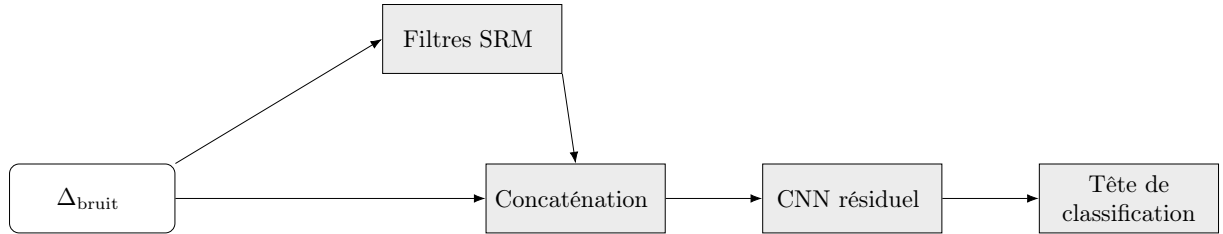


FIGURE 3.5 – Architecture NoiseSRMResidualCNN utilisée pour exploiter la différence de bruit.

Approches basées sur la *feature* fréquence

Les informations fréquentielles sont extraites à partir de la différence signée Δ_{pixel} . Deux stratégies complémentaires ont été étudiées afin d'exploiter les signatures spectrales potentiellement présentes dans les erreurs de reconstruction.

La première, *AmpPhaseSiamese*, sépare explicitement l'amplitude et la phase du spectre obtenu par transformée de Fourier 2D. L'amplitude décrit la répartition de l'énergie fréquentielle, tandis que la phase apporte une information complémentaire sur l'organisation des structures. La phase est représentée par son sinus et son cosinus afin de respecter sa nature périodique, puis pondérée selon l'amplitude. Les deux représentations sont ensuite traitées par des branches convolutionnelles partageant les mêmes paramètres avant d'être combinées pour la classification.

La seconde approche est une adaptation de FreqNet (Tan et al., 2024). Elle vise à intégrer

l'information fréquentielle directement dans le processus d'extraction des caractéristiques, et non uniquement au niveau de l'entrée. Le module HFRI (*High-Frequency Representation of Image*) met en évidence les composantes haute fréquence de la carte d'entrée. Les modules HFRFS et HFRFC poursuivent cette analyse sur les représentations intermédiaires, respectivement selon les dimensions spatiales et selon la dimension des canaux. Enfin, les couches FCL (*Frequency Convolution Layer*) réalisent des transformations apprises dans le domaine fréquentiel avant de revenir dans le domaine spatial.

Ces deux architectures permettent ainsi de comparer une décomposition explicite en amplitude et phase à une exploitation des hautes fréquences intégrée tout au long du réseau.

Les deux stratégies sont résumées sur la figure 3.6.

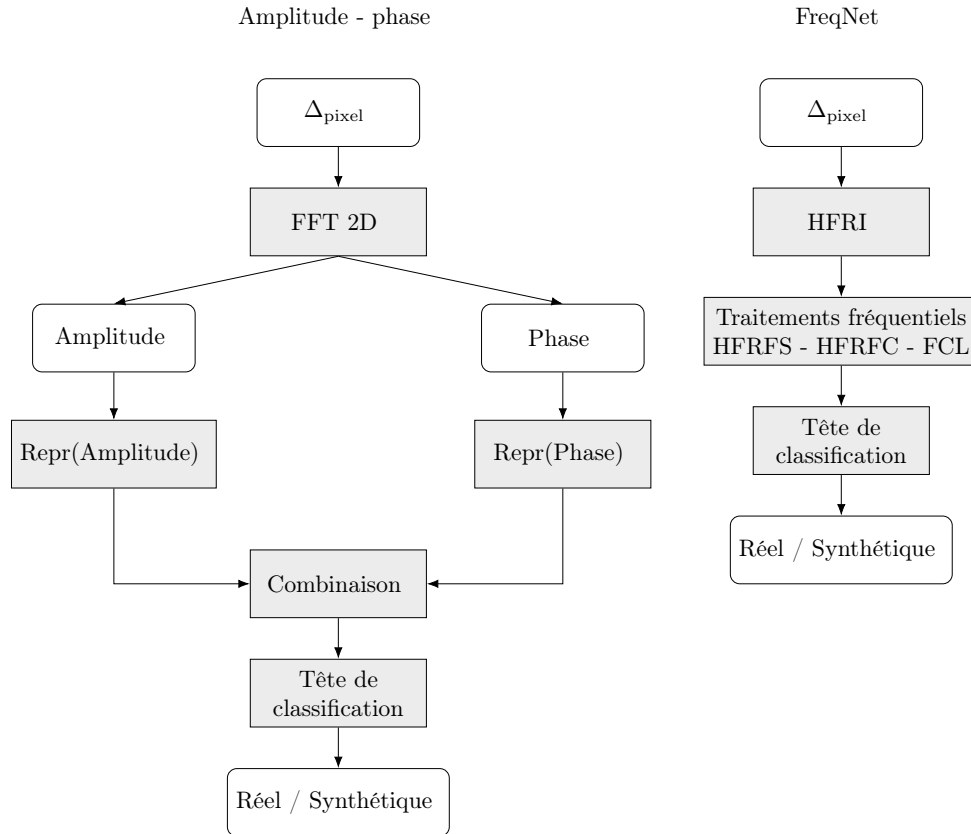


FIGURE 3.6 – Architectures étudiées pour exploiter l'information fréquentielle.

Approche basée sur la *feature* ondelettes

L'approche par ondelettes est appliquée à la différence signée Δ_{pixel} afin d'analyser les erreurs de reconstruction à plusieurs échelles spatiales et fréquentielles. Une transformée en ondelettes discrète bidimensionnelle (DWT) est appliquée indépendamment aux trois canaux et produit, à chaque niveau, les sous-bandes *LL*, *LH*, *HL* et *HH*. Les principes de la DWT et des différentes familles d'ondelettes sont détaillés en annexe C.2.

Le choix de l'ondelette et de la profondeur de décomposition n'étant pas fixé a priori, une recherche a été menée parmi les ondelettes Haar, Daubechies 2, Symlet 2 et Coiflet 1, pour des niveaux $L \in \{2, 3, 4\}$. Cette comparaison, réalisée avec un même α et une même graine, a conduit à retenir **Coiflet 1 avec un niveau de décomposition $L = 4$** , qui fournit les meilleures performances sur le jeu de validation.

La classification repose ensuite sur une architecture WaveletCNN inspirée des approches combinant décomposition en ondelettes et réseaux convolutionnels (Fujieda et al., 2018). Les informations issues des différents niveaux sont intégrées progressivement : les représentations de

plus haute résolution sont réduites par des convolutions de *stride* 2, puis concaténées avec celles du niveau suivant. Cette fusion hiérarchique permet d'exploiter conjointement les détails locaux de haute fréquence et les structures présentes à différentes échelles.

L'architecture finale utilise ainsi la décomposition **Coiflet 1 à quatre niveaux** pour construire la représentation ondelette utilisée pour la classification.

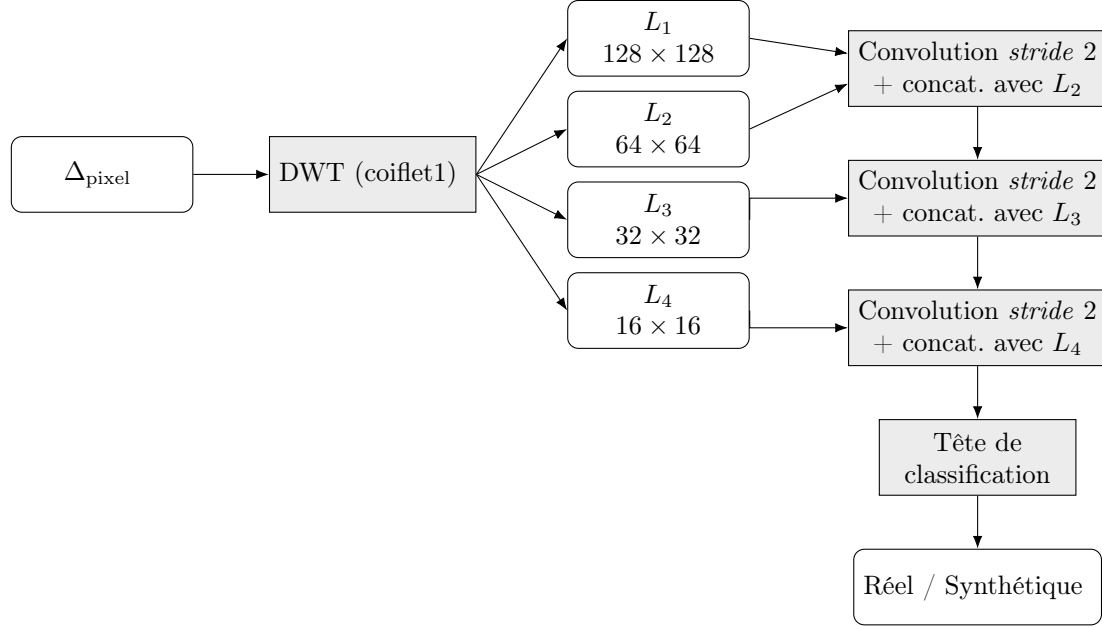


FIGURE 3.7 – Principe de l'architecture WaveletCNN.

3.4 Résultats obtenus

Les expériences finales sont réalisées selon le protocole présenté en section 3.1. Les configurations étudiées sont résumées dans le tableau 3.1.

Représentation	Architectures	Paramètres explorés	α	Graines
Pixel	CNN-D1 ; ResNet18 gelé ; ResNet18 L4 dégelé	$lr \in \{10^{-4}, 3 \times 10^{-4}\}$	0,15 à 0,95	42, 123, 3407
Latent	LatentUNetSmall ; LatentUNet-Large ; LatentSiameseCNN	$lr \in \{10^{-3}, 3 \times 10^{-3}\}$	0,15 à 0,95	42, 123, 3407
Bruit	NoiseSRMResidualCNN ; Noise-SiameseCNN	NoiseSRMResidualCNN : $lr \in \{3 \times 10^{-4}, 10^{-3}\}$ NoiseSiameseCNN : $lr \in \{10^{-4}, 3 \times 10^{-4}\}$	0,15 à 0,95	42, 123, 3407
Fréquence	AmpPhaseSiamese ; FreqNet	$lr \in \{3 \times 10^{-4}, 10^{-3}\}$ AmpPhaseSiamese : fusion par concaténation/pondérée(<i>gated</i>) FreqNet : $scale \in \{4, 8\}$	0,15 à 0,95	42, 123, 3407
Ondelette – sélection	WaveletCNN	Ondelette $\in \{\text{Haar, db2, sym2, coif1}\}$; niveau $L \in \{2, 3, 4\}$	0,55	42
Ondelette – finale	WaveletCNN, Coiflet 1, $L = 4$	$lr \in \{10^{-4}, 3 \times 10^{-4}\}$	0,15 à 0,95	42, 123, 3407

TABLE 3.1 – Grilles de recherche utilisées lors des expériences sur le jeu complet de VIGINUM.

3.4.1 Sélection des modèles

Avant de comparer les différentes *features*, une architecture représentative a été retenue pour chacune d'elles à partir des performances obtenues sur le jeu de validation. Le jeu de test n'in-

tervient donc pas dans cette étape de sélection.

Les architectures retenues pour chaque représentation sont présentées dans le tableau 3.2.

Représentation	Architecture	$\alpha = 0,15$	$\alpha = 0,35$	$\alpha = 0,55$	$\alpha = 0,75$	$\alpha = 0,95$	Moyenne
Pixel	CNN-D1	$0,877 \pm 0,019$ LR = 3×10^{-4}	$0,873 \pm 0,023$ LR = 3×10^{-4}	$0,865 \pm 0,013$ LR = 3×10^{-4}	$0,873 \pm 0,010$ LR = 3×10^{-4}	$0,864 \pm 0,006$ LR = 10^{-4}	0,870
	ResNet18 gelé	$0,903 \pm 0,007$ LR = 3×10^{-4}	$0,899 \pm 0,003$ LR = 3×10^{-4}	$0,892 \pm 0,003$ LR = 3×10^{-4}	$0,890 \pm 0,002$ LR = 3×10^{-4}	$0,891 \pm 0,002$ LR = 3×10^{-4}	
	ResNet18 L4 dégelée	$0,948 \pm 0,002$ LR = 3×10^{-4}	$0,948 \pm 0,003$ LR = 10^{-4}	$0,947 \pm 0,000$ LR = 3×10^{-4}	$0,949 \pm 0,001$ LR = 3×10^{-4}	$0,950 \pm 0,001$ LR = 3×10^{-4}	0,948
Latent	LatentUNetSmall	$0,884 \pm 0,010$ LR = 3×10^{-3}	$0,888 \pm 0,001$ LR = 10^{-3}	$0,892 \pm 0,003$ LR = 3×10^{-3}	$0,894 \pm 0,004$ LR = 3×10^{-3}	$0,893 \pm 0,002$ LR = 3×10^{-3}	0,890
	LatentUNetLarge	$0,884 \pm 0,010$ LR = 10^{-3}	$0,881 \pm 0,008$ LR = 3×10^{-3}	$0,887 \pm 0,007$ LR = 3×10^{-3}	$0,892 \pm 0,009$ LR = 10^{-3}	$0,892 \pm 0,011$ LR = 3×10^{-3}	
	LatentSiameseCNN	$0,880 \pm 0,005$ LR = 3×10^{-3}	$0,880 \pm 0,002$ LR = 3×10^{-3}	$0,882 \pm 0,005$ LR = 3×10^{-3}	$0,879 \pm 0,010$ LR = 3×10^{-3}	$0,887 \pm 0,004$ LR = 3×10^{-3}	0,882
Bruit	NoiseSRMResidualCNN	$0,875 \pm 0,006$ LR = 10^{-3}	$0,870 \pm 0,006$ LR = 10^{-3}	$0,855 \pm 0,002$ LR = 10^{-3}	$0,826 \pm 0,005$ LR = 10^{-3}	$0,813 \pm 0,007$ LR = 10^{-3}	0,848
	NoiseSiameseCNN	$0,606 \pm 0,148$ LR = 10^{-4}	$0,775 \pm 0,006$ LR = 10^{-4}	$0,785 \pm 0,004$ LR = 10^{-4}	$0,797 \pm 0,002$ LR = 10^{-4}	$0,832 \pm 0,003$ LR = 3×10^{-4}	
							0,759
Fréquence	AmpPhaseSiamese	$0,830 \pm 0,013$ LR = 10^{-3}	$0,835 \pm 0,002$ LR = 10^{-3}	$0,832 \pm 0,002$ LR = 10^{-3}	$0,830 \pm 0,004$ LR = 10^{-3}	$0,831 \pm 0,007$ LR = 10^{-3}	0,832
	FreqNet	$0,826 \pm 0,006$ LR = 10^{-3}	$0,822 \pm 0,003$ LR = 10^{-3}	$0,823 \pm 0,003$ LR = 10^{-3}	$0,823 \pm 0,005$ LR = 10^{-3}	$0,820 \pm 0,002$ LR = 3×10^{-4}	
Ondelettes	WaveletCNN – Coiflet 1, $L = 4$	$0,883 \pm 0,004$ LR = 3×10^{-4}	$0,877 \pm 0,006$ LR = 3×10^{-4}	$0,880 \pm 0,004$ LR = 3×10^{-4}	$0,875 \pm 0,002$ LR = 10^{-4}	$0,874 \pm 0,004$ LR = 3×10^{-4}	0,878

TABLE 3.2 – Balanced Accuracy obtenue sur le jeu de validation pour les différentes architectures. Pour chaque architecture et chaque valeur de α , la configuration donnant la meilleure performance moyenne sur les trois graines aléatoires est retenue. Les résultats sont présentés sous la forme moyenne $\pm$ écart-type et le taux d'apprentissage sélectionné est indiqué sous chaque valeur. Les cellules en vert clair indiquent, pour chaque représentation et chaque valeur de α , la meilleure Balanced Accuracy obtenue parmi les architectures comparées. En cas d'égalité, les deux valeurs sont mises en évidence. La dernière colonne correspond à la moyenne sur les cinq valeurs de α ; la meilleure moyenne de chaque représentation, utilisée pour sélectionner l'architecture finale, est indiquée en vert plus foncé et en gras. Pour la représentation ondelettes, une seule architecture finale est présentée ici, la sélection de la famille d'ondelettes et du niveau de décomposition étant détaillée en annexe B.4.

3.4.2 Influence du niveau de bruitage α

Après la sélection d'une architecture représentative pour chaque feature, les performances obtenues pour les différents niveaux de bruitage α sont comparées sur le jeu de test. Pour chaque valeur de α , les hyperparamètres ont préalablement été sélectionnés sur le jeu de validation. Cette analyse permet donc d'étudier la sensibilité globale des différentes approches au niveau de bruitage, sans constituer une ablation stricte où α serait le seul paramètre modifié.

Pour chaque représentation et chaque valeur de α , la Balanced Accuracy est calculée sur les trois graines aléatoires et présentée sous la forme moyenne $\pm$ écart-type. À titre de repère, la Balanced Accuracy publiée pour DIMD dans le benchmark PEReN-VIGINUM, égale à 79 %, est également indiquée comme référence externe.

La figure 3.8 compare directement les cinq représentations pour les différents niveaux de bruitage. Elle permet d'observer à la fois l'évolution des performances avec α et leur position par rapport à la baseline du benchmark.

La figure 3.8 montre que l'approche pixel présente des performances particulièrement stables, proches de 95 % pour toutes les valeurs de α . Les représentations latente, fréquentielle et ondelettes varient également relativement peu. À l'inverse, la représentation bruit est nettement plus sensible au niveau de bruitage, sa Balanced Accuracy passant d'environ 87,6 % pour $\alpha = 0,15$ à 80,9 % pour $\alpha = 0,95$.

Ces résultats suggèrent donc **une faible sensibilité au choix de α pour les approches pixel, latente, fréquentielle et ondelettes, tandis que ce paramètre joue un rôle plus important pour la représentation bruit.**

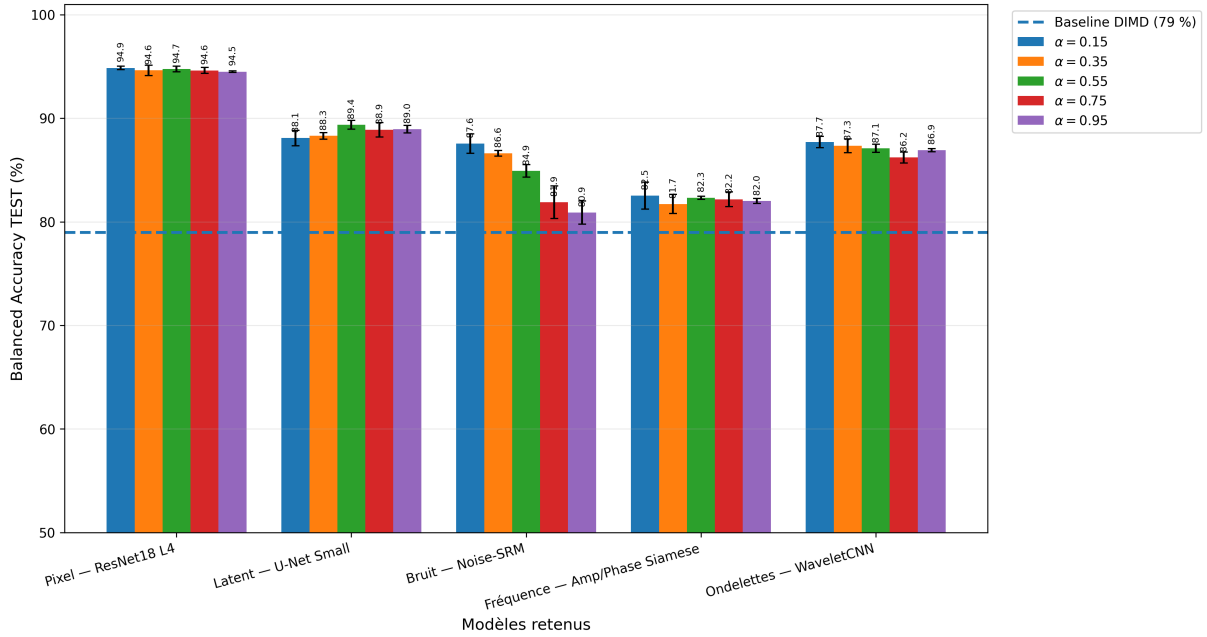


FIGURE 3.8 – Influence du niveau de bruitage α sur la Balanced Accuracy obtenue sur le jeu de test pour les cinq représentations retenues. Les groupes de barres correspondent aux modèles sélectionnés, et les différentes barres au sein de chaque groupe correspondent aux cinq valeurs de α . Les barres représentent la moyenne sur les trois graines aléatoires et les barres d'erreur l'écart-type. La ligne horizontale indique la Balanced Accuracy publiée pour DIMD (79 %) dans le benchmark PEReN-VIGINUM et sert de repère externe.

3.4.3 Comparaison globale des représentations

Les cinq représentations retenues sont comparées selon la Balanced Accuracy, l'AUC, la précision, le rappel et le F1-score. Pour nos modèles, les valeurs correspondent à la moyenne sur les cinq niveaux de α et les trois graines aléatoires, soit 15 évaluations. Les performances de CoDE et DIMD issues du benchmark Viginum sont ajoutées comme références.

La figure 3.9 montre que, dans notre protocole, la représentation pixel obtient les meilleures performances parmi les cinq représentations étudiées, avec une Balanced Accuracy de $94,7 \pm 0,3 \%$, une AUC de $98,7 \pm 0,2 \%$ et un F1-score de $94,1 \pm 0,3 \%$. Elle est suivie par la représentation latente, puis par les ondelettes, le bruit et la fréquence.

Les performances publiées de CoDE et DIMD dans le benchmark PEReN-VIGINUM sont ajoutées afin de fournir des points de référence externes. Les valeurs obtenues par nos représentations sont supérieures à ces références sur plusieurs métriques, notamment la Balanced Accuracy, l'AUC, le rappel et le F1-score. DIMD conserve toutefois une précision plus élevée, avec 97 % contre $93,0 \pm 0,8 \%$ pour notre approche pixel.

Ces écarts ne doivent cependant pas être interprétés comme une comparaison expérimentale directe entre les méthodes. CoDE et DIMD n'ont pas été réévalués dans notre protocole d'entraînement et de test. Leurs résultats permettent donc principalement de situer l'ordre de grandeur des performances obtenues par nos approches.

3.4.4 Performances selon les générateurs

Afin d'étudier la dépendance des performances au modèle génératif, le taux de détection est calculé séparément pour chaque générateur d'images synthétiques. Les résultats correspondent à la moyenne sur les cinq valeurs de α et les trois graines aléatoires.

La figure 3.10 montre que les performances dépendent fortement du générateur considéré. La

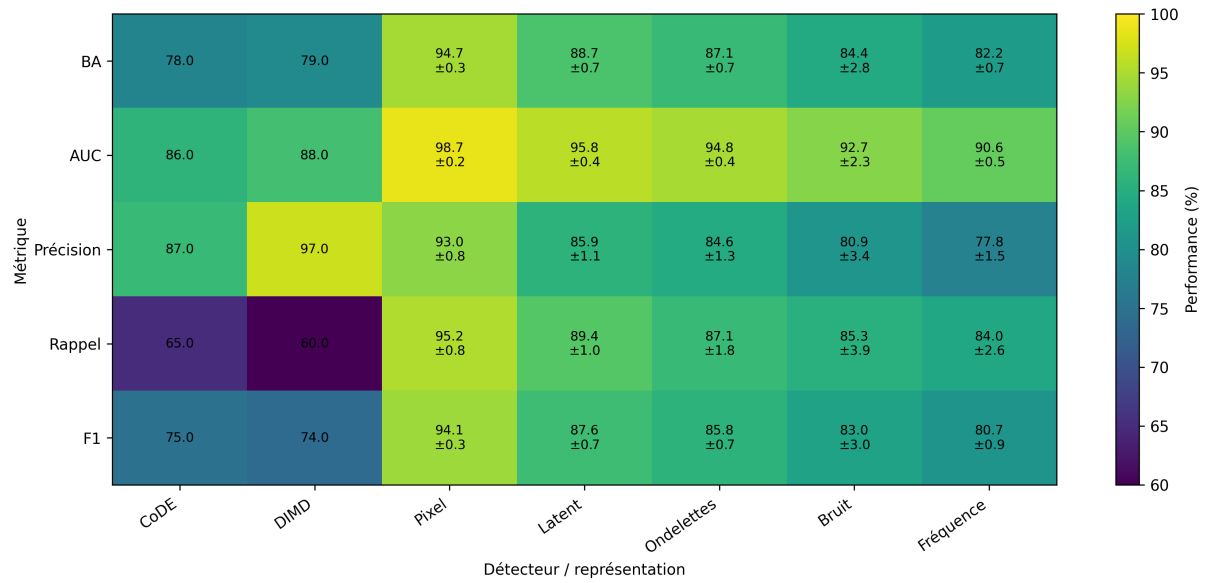


FIGURE 3.9 – Mise en perspective des performances globales des représentations proposées avec les résultats publiés de CoDE et DIMD dans le benchmark PEReN-VIGINUM. Les résultats de nos modèles correspondent à la moyenne $\pm$ écart-type sur les cinq valeurs de α et les trois graines aléatoires. Les valeurs de CoDE et DIMD sont utilisées comme références externes.

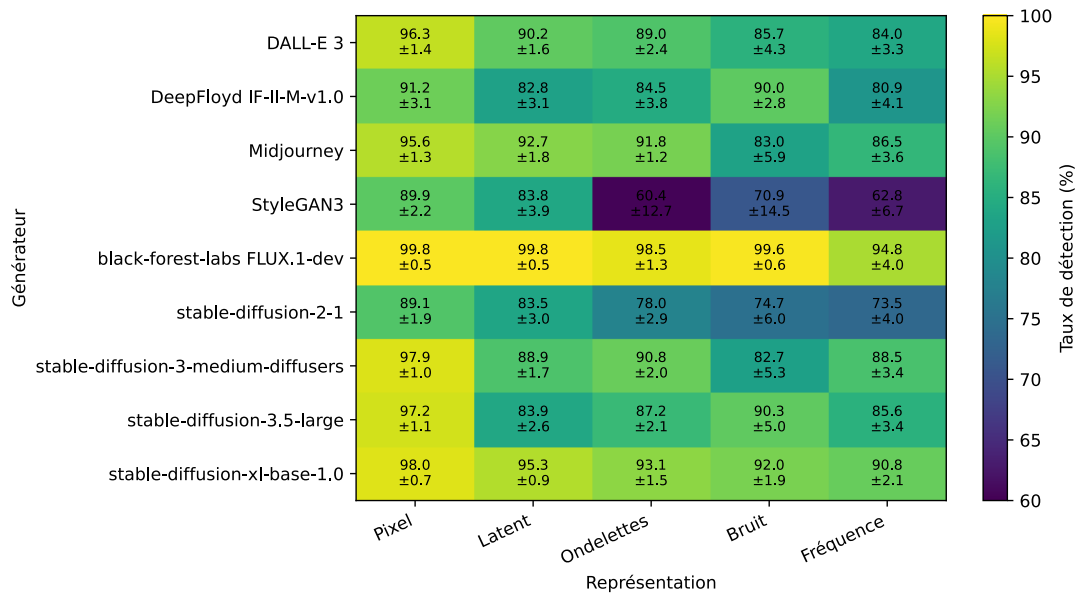


FIGURE 3.10 – Taux de détection obtenu pour chaque générateur selon la représentation utilisée. Les résultats sont présentés sous la forme moyenne $\pm$ écart-type sur les cinq niveaux de bruitage et les trois graines aléatoires.

représentation pixel reste globalement la plus performante, avec notamment 99,8 % de détection pour FLUX.1-dev, 98,0 % pour SDXL et 97,9 % pour Stable Diffusion 3 Medium.

FLUX.1-dev est particulièrement bien détecté par l'ensemble des représentations, avec des taux supérieurs à 94 %. À l'inverse, StyleGAN3 constitue le cas le plus difficile, notamment pour les ondelettes (60,4 %) et la fréquence (62,8 %).

Ces résultats montrent ainsi que la représentation pixel présente les performances les plus élevées et les plus régulières sur les différents générateurs évalués, tandis que certaines représentations sont davantage sensibles à la nature du modèle génératif.

3.4.5 Synthèse et discussion

Les expériences montrent que les erreurs de reconstruction contiennent une information discriminante exploitable dans les cinq représentations étudiées. Parmi les couples représentation–architecture évalués, l'approche pixel obtient les meilleures performances globales, avec une Balanced Accuracy de $94,7 \pm 0,3$ % et une AUC de $98,7 \pm 0,2$ %. Elle est suivie par les approches latente (88,7 % de BA) et ondelettes (87,1 %), puis par le bruit (84,4 %) et la fréquence (82,2 %).

Les résultats publiés dans le benchmark PEReN–VIGINUM permettent de situer ces performances. DIMD atteint 79 % de Balanced Accuracy et 88 % d'AUC, tandis que CoDE obtient respectivement 78 % et 86 %. Dans notre protocole, plusieurs des valeurs obtenues sont supérieures à ces références. Cette comparaison reste toutefois indicative, CoDE et DIMD n'ayant pas été réévalués dans des conditions strictement identiques aux nôtres. DIMD conserve notamment une précision de 97 %, contre $93,0 \pm 0,8$ % pour l'approche pixel, tandis que cette dernière atteint un rappel de $95,2 \pm 0,8$ %, contre 60 % pour DIMD.

L'analyse par générateur met en évidence une variation sensible des performances selon le modèle à l'origine de l'image. L'approche pixel présente les résultats les plus réguliers et atteint notamment 99,8 % de détection sur FLUX.1-dev, 98,0 % sur Stable Diffusion XL et 97,9 % sur Stable Diffusion 3 Medium. À l'inverse, **StyleGAN3 constitue un cas plus difficile, en particulier pour les représentations ondelettes, bruit et fréquence**. Une explication possible est que notre méthode repose sur une reconstruction effectuée par un modèle de diffusion, ce qui peut favoriser des signatures plus adaptées aux images issues de cette famille de générateurs. **Cette analyse met donc en évidence une dépendance au générateur, sans constituer une évaluation de la généralisation à des modèles totalement absents des données d'entraînement.**

Une limite du protocole concerne également la provenance des images. Les images réelles et synthétiques étant issues de plusieurs jeux de données, certaines différences de bas niveau peuvent provenir des conditions d'acquisition, de compression ou de prétraitement plutôt que du processus génératif lui-même. Ce point est particulièrement important pour les représentations pixel, bruit et fréquence, susceptibles d'être sensibles à ce type de traces. Les résultats ne permettent donc pas d'exclure totalement l'exploitation de biais propres aux jeux de données. Une évaluation sur de nouvelles sources permettrait de mieux caractériser la robustesse des signatures apprises.

Le niveau de bruitage α affecte par ailleurs les représentations de manière différente. Les performances des approches pixel, latente, fréquentielle et ondelettes restent relativement stables, tandis que celles de la représentation bruit diminuent progressivement lorsque α augmente, passant d'environ 87,6 % à 80,9 % de Balanced Accuracy.

Les résultats montrent également qu'une architecture plus complexe n'apporte pas nécessairement de meilleures performances. *LatentUNetSmall* est ainsi retenu devant *LatentUNetLarge* et *LatentSiameseCNN*, tandis que l'approche pixel obtient ses meilleurs résultats avec un ResNet18 dont seul le dernier bloc résiduel est dégelé. Pour les ondelettes, la famille et le niveau de décomposition ont d'abord été sélectionnés sur une configuration restreinte avant d'être fixés pour les expériences suivantes.

Il convient toutefois de noter que les différentes représentations sont associées à des architectures spécifiques. Les écarts de performance observés ne peuvent donc pas être attribués

uniquement à la représentation utilisée, mais résultent du couple représentation–architecture. Ce point est notamment important pour l’approche pixel, qui bénéficie d’un ResNet18 pré-entraîné sur ImageNet et partiellement adapté à la tâche.

Afin de conserver une présentation synthétique, seuls les modèles retenus sont analysés dans le corps du rapport. Les résultats détaillés des autres architectures, notamment CNN-D1, ResNet18 gelé, LatentUNetLarge, LatentSiameseCNN, NoiseSiameseCNN, FreqNet ainsi que les différentes configurations d’ondelettes, sont présentés en annexe [B.4](#).

Conclusion

Ce travail avait pour objectif d’étudier l’utilisation des erreurs de reconstruction produites par un modèle de diffusion pour la détection d’images générées artificiellement. Cinq représentations ont été considérées : pixel, latent, bruit, fréquence et ondelettes. Pour chacune d’elles, plusieurs architectures ont été évaluées avant de retenir le modèle le plus adapté, puis de l’étudier sur plusieurs niveaux de bruitage α , plusieurs graines aléatoires et différents générateurs.

Les résultats montrent que les erreurs de reconstruction contiennent une information discriminante exploitable. Parmi les couples représentation–architecture étudiés, l’approche pixel obtient les meilleures performances globales, avec une Balanced Accuracy moyenne de 94,7 % et une AUC de 98,7 %. Elle est suivie par les représentations latente et ondelettes, tandis que le bruit et la fréquence restent moins performants mais conservent un pouvoir discriminant.

L’étude du niveau de bruitage montre que les représentations pixel, latente, fréquentielle et ondelettes restent relativement stables lorsque α varie, alors que la représentation bruit est davantage sensible à ce paramètre. Les résultats montrent également qu’une architecture plus complexe n’apporte pas nécessairement de meilleures performances, comme l’illustrent *LatentUNetSmall* et le ResNet18 avec uniquement son dernier bloc résiduel dégelé.

L’analyse par générateur met par ailleurs en évidence une variabilité importante des performances selon le modèle ayant produit l’image. L’approche pixel présente les résultats les plus élevés et les plus réguliers sur les générateurs évalués, tandis que StyleGAN3 constitue notamment un cas plus difficile pour plusieurs représentations. Cette analyse souligne l’importance de prendre en compte la nature du générateur, sans toutefois constituer une évaluation de la généralisation à des générateurs totalement absents des données d’entraînement.

Les résultats publiés dans le benchmark PEReN–VIGINUM permettent enfin de situer les performances obtenues. Dans notre protocole, plusieurs représentations atteignent des valeurs supérieures aux résultats publiés de CoDE et DIMD sur différentes métriques, tandis que DIMD conserve notamment une précision plus élevée. Cette comparaison reste cependant indicative, les détecteurs du benchmark n’ayant pas été évalués dans des conditions strictement identiques aux nôtres.

Plusieurs perspectives restent à explorer. Il serait notamment intéressant d’étudier la robustesse aux transformations telles que la compression, le redimensionnement ou le floutage, ainsi que la combinaison de plusieurs représentations complémentaires. Une autre perspective importante consiste à évaluer la généralisation à des générateurs non vus, en entraînant les modèles sur un sous-ensemble de générateurs puis en les testant sur les autres. Ce protocole permettrait de déterminer si les signatures apprises sont réellement génériques ou restent en partie spécifiques aux générateurs rencontrés pendant l’entraînement.

Ainsi, l’analyse des erreurs de reconstruction apparaît comme une approche prometteuse pour la détection d’images synthétiques. Dans le cadre expérimental étudié, l’approche pixel offre le meilleur compromis entre performance et stabilité, tout en laissant plusieurs pistes ouvertes pour évaluer plus largement sa robustesse et sa capacité de généralisation.

Bibliographie

- Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein gan, 2017. URL <https://arxiv.org/abs/1701.07875>.
- Lorenzo Baraldi, Federico Cocchi, Marcella Cornia, Lorenzo Baraldi, Alessandro Nicolosi, and Rita Cucchiara. Contrasting deepfakes diffusion via contrastive learning and global-local similarities, 2024. URL <https://arxiv.org/abs/2407.20337>.
- Théis Bazin, Cristian Brokate, Sarah Cocher, Raphaël Couronné, Gaspard Defréville, Lisa Giordani, Christian Launay, and Tu Duyen Nguyen. Building a digital common to benchmark AI-generated image detectors. Extended abstract presented at the Journées Infox sur Seine 2025, March 2025. URL <https://infoxsurseine.org/2025/Building-a-digital-common-to-benchmark-AI-generated-image-detectors/>.
- David Beniaguev. Synthetic faces high quality – text-to-image (SFHQ-T2I) dataset. GitHub repository and Kaggle dataset, 2024. URL <https://github.com/SelfishGene/SFHQ-T2I-dataset>.
- James Betker, Gabriel Goh, Li Jing, Tim Brooks, Jianfeng Wang, Linjie Li, Long Ouyang, Juntang Zhuang, Joyce Lee, Yufei Guo, Wesam Manassra, Prafulla Dhariwal, Casey Chu, Yunxin Jiao, and Aditya Ramesh. Improving image generation with better captions. *OpenAI Technical Report*, 2023. URL <https://cdn.openai.com/papers/dall-e-3.pdf>.
- Black Forest Labs. FLUX.1 [dev]. Hugging Face model card, 2024. URL <https://huggingface.co/black-forest-labs/FLUX.1-dev>.
- Sam Bond-Taylor, Adam Leach, Yang Long, and Chris G. Willcocks. Deep generative modelling : A comparative review of vaes, gans, normalizing flows, energy-based and autoregressive models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44 (11) :7327–7347, November 2022. ISSN 1939-3539. doi : 10.1109/tpami.2021.3116668. URL <http://dx.doi.org/10.1109/TPAMI.2021.3116668>.
- C. Charles. Introduction aux ondelettes. Note technique, Université de Liège, Gembloux Agro-Bio Tech, 2010.
- Baoying Chen, Jishen Zeng, Jianquan Yang, and Rui Yang. DRCT : Diffusion reconstruction contrastive training towards universal detection of diffusion generated images, 2024. URL <https://proceedings.mlr.press/v235/chen24ay.html>. Proceedings of the 41st International Conference on Machine Learning, volume 235, pages 7621–7639.
- Beilin Chu, Xuan Xu, Xin Wang, Yufei Zhang, Weike You, and Linna Zhou. FIRE : Robust detection of diffusion-generated images via frequency-guided reconstruction error, June 2025. URL <https://doi.org/10.1109/CVPR52734.2025.01197>. 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 12830–12839.

- Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models, 2022. URL <https://arxiv.org/abs/2210.11416>.
- Riccardo Corvi, Davide Cozzolino, Giada Zingarini, Giovanni Poggi, Koki Nagano, and Luisa Verdoliva. On the detection of synthetic images generated by diffusion models, 2022. URL <https://arxiv.org/abs/2211.00680>.
- Riccardo Corvi, Davide Cozzolino, Ekta Prashnani, Shalini De Mello, Koki Nagano, and Luisa Verdoliva. Seeing what matters : Generalizable ai-generated video detection with forensic-oriented augmentation, 2025. URL <https://arxiv.org/abs/2506.16802>.
- CYANEX 1702. Midjourney images & prompt. Kaggle dataset, 2024. URL <https://www.kaggle.com/datasets/cyanex1702/midjourney-imagesprompt>. Sous-ensemble utilisé : Datasetfordream.
- Ingrid Daubechies. Orthonormal bases of compactly supported wavelets. *Communications on Pure and Applied Mathematics*, 41(7) :909–996, 1988. doi : 10.1002/cpa.3160410705.
- Ingrid Daubechies. *Ten Lectures on Wavelets*, volume 61 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. Society for Industrial and Applied Mathematics, Philadelphia, 1992. doi : 10.1137/1.9781611970104.
- Ingrid Daubechies. Orthonormal bases of compactly supported wavelets ii : Variations on a theme. *SIAM Journal on Mathematical Analysis*, 24(2) :499–519, 1993. doi : 10.1137/0524031.
- DeepFloyd. DeepFloyd IF. Hugging Face model card, 2023. URL <https://huggingface.co/DeepFloyd/IF-I-M-v1.0>.
- Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled, 2024. URL <https://arxiv.org/abs/2405.15682>.
- L. Degeorge, A. Ghosh, N. Dufour, D. Picard, and V. Kalogeiton. How far can we go with imagenet for text-to-image generation ?, 2025. URL <https://arxiv.org/abs/2502.21318>.
- Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet : A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. doi : 10.1109/CVPR.2009.5206848.
- Prafulla Dhariwal and Alex Nichol. Diffusion models beat gans on image synthesis, 2021. URL <https://arxiv.org/abs/2105.05233>.
- ehristoforu. dalle-3-images. Hugging Face Datasets, 2024a. URL <https://huggingface.co/datasets/ehristoforu/dalle-3-images>.
- ehristoforu. midjourney-images. Hugging Face Datasets, 2024b. URL <https://huggingface.co/datasets/ehristoforu/midjourney-images>.
- Abul Ehtesham, Saket Kumar, Aditi Singh, and Tala Talaei Khoei. Movie gen : Swot analysis of meta’s generative ai foundation model for transforming media generation, advertising, and entertainment industries, 2024. URL <https://arxiv.org/abs/2412.03837>.

- Patrick Esser et al. Scaling rectified flow transformers for high-resolution image synthesis. *arXiv preprint arXiv :2403.03206*, 2024. URL <https://arxiv.org/abs/2403.03206>.
- FIDLE. Supports de cours – saison 2024–2025. Cloud de l’Université Grenoble Alpes, 2025. URL <https://cloud.univ-grenoble-alpes.fr/s/xaM2R7HmpCbBHCj?dir=/Saison%202024-2025&editing=false&openfile=true>. Consulté le 24 juillet 2026.
- Joel Frank, Thorsten Eisenhofer, Lea Schönherr, Asja Fischer, Dorothea Kolossa, and Thorsten Holz. Leveraging frequency analysis for deep fake image recognition, 2020. URL <https://arxiv.org/abs/2003.08685>.
- Jessica Fridrich and Jan Kodovsky. Rich models for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 7(3) :868–882, 2012. doi : 10.1109/TIFS.2012.2190402.
- Shin Fujieda, Kohei Takayama, and Toshiya Hachisuka. Wavelet convolutional neural networks, 2018. URL <https://arxiv.org/abs/1805.08620>.
- Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014. URL <https://arxiv.org/abs/1406.2661>.
- Government of the Hong Kong Special Administrative Region. Lcq9 : Combating frauds involving deepfake. Press Releases of the Government of the Hong Kong Special Administrative Region, June 2024. URL <https://www.info.gov.hk/gia/general/202406/26/P2024062600192.htm>. Consulté le 17 juillet 2026.
- Fabrizio Guillaro, Giada Zingarini, Ben Usman, Avneesh Sud, Davide Cozzolino, and Luisa Verdoliva. A bias-free training paradigm for more general ai-generated image detection, 2025. URL <https://arxiv.org/abs/2412.17671>.
- Alfred Haar. Zur theorie der orthogonalen funktionensysteme. erste mitteilung. *Mathematische Annalen*, 69 :331–371, 1910. doi : 10.1007/BF01456326.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL <https://arxiv.org/abs/1512.03385>.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786) :504–507, 2006. doi : 10.1126/science.1127647. URL <https://www.science.org/doi/abs/10.1126/science.1127647>.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL <https://arxiv.org/abs/2006.11239>.
- Allan Jabri, David Fleet, and Ting Chen. Scalable adaptive computation for iterative generation, 2023. URL <https://arxiv.org/abs/2212.11972>.
- Dimitrios Karageorgiou, Symeon Papadopoulos, Ioannis Kompatsiaris, and Efstratios Gavves. Any-resolution ai-generated image detection by spectral learning, 2025. URL <https://arxiv.org/abs/2411.19417>.
- Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4401–4410, 2019. doi : 10.1109/CVPR.2019.00453.

- Tero Karras, Miika Aittala, Samuli Laine, Erik Härkönen, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Alias-free generative adversarial networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 852–863, 2021.
- Bahjat Kawar, Michael Elad, Stefano Ermon, and Jiaming Song. Denoising diffusion restoration models, 2022. URL <https://arxiv.org/abs/2201.11793>.
- Diederik P. Kingma and Jimmy Ba. Adam : A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.
- Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations (ICLR)*, 2014.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022. URL <https://arxiv.org/abs/1312.6114>.
- Binh M. Le and Simon S. Woo. Add : Frequency attention and multi-view based knowledge distillation to detect low-quality compressed deepfake images, 2021. URL <https://arxiv.org/abs/2112.03553>.
- Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s, 2022. URL <https://arxiv.org/abs/2201.03545>.
- Andreas Lugmayr, Martin Danelljan, Andres Romero, Fisher Yu, Radu Timofte, and Luc Van Gool. Repaint : Inpainting using denoising diffusion probabilistic models, 2022. URL <https://arxiv.org/abs/2201.09865>.
- Yunpeng Luo, Junlong Du, Ke Yan, and Shouhong Ding. Lare² : Latent reconstruction error based method for diffusion-generated image detection, 2025. URL <https://arxiv.org/abs/2403.17465>.
- Ruipeng Ma, Jinhao Duan, Fei Kong, Xiaoshuang Shi, and Kaidi Xu. Exposing the fake : Effective diffusion-generated images detection, 2023. URL <https://arxiv.org/abs/2307.06272>.
- François Malgouyres. Représentation des images et ondelettes. Cours, Département d’informatique, Université Paris 13, 2009. URL <http://www.zeus.math.univ-paris13.fr/~malgouy/>.
- Stéphane G. Mallat. A theory for multiresolution signal decomposition : The wavelet representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(7) :674–693, 1989. doi : 10.1109/34.192463.
- Philip Marcelo. FACT FOCUS : Fake image of pentagon explosion briefly sends jitters through stock market. AP News, May 2023. URL <https://apnews.com/article/pentagon-explosion-misinformation-stock-market-ai-96f534c790872fde67012ee81b5ed6a4>. Consulté le 17 juillet 2026.
- Midjourney. Version. Official Midjourney documentation, 2026. URL <https://docs.midjourney.com/hc/en-us/articles/32199405667853-Version>.
- Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks, 2018. URL <https://arxiv.org/abs/1802.05957>.
- Utkarsh Ojha, Yuheng Li, and Yong Jae Lee. Towards universal fake image detectors that generalize across generative models, 2024. URL <https://arxiv.org/abs/2302.10174>.
- Keiron O’Shea and Ryan Nash. An introduction to convolutional neural networks, 2015. URL <https://arxiv.org/abs/1511.08458>.

- Achraf Oussidi and Azeddine Elhassouny. Deep generative models : Survey. In *2018 International Conference on Intelligent Systems and Computer Vision (ISCV)*, pages 1–8, 2018. doi : 10.1109/ISACV.2018.8354080.
- Luis Perez and Jason Wang. The effectiveness of data augmentation in image classification using deep learning, 2017. URL <https://arxiv.org/abs/1712.04621>.
- Walter H. L. Pinaya, Mark S. Graham, Eric Kerfoot, Petru-Daniel Tudosiu, Jessica Dafflon, Virginia Fernandez, Pedro Sanchez, Julia Wolleb, Pedro F. da Costa, Ashay Patel, Hyungjin Chung, Can Zhao, Wei Peng, Zelong Liu, Xueyan Mei, Oeslle Lucena, Jong Chul Ye, Sotirios A. Tsaftaris, Prerna Dogra, Andrew Feng, Marc Modat, Parashkev Nachev, Sebastien Ourselin, and M. Jorge Cardoso. Generative ai for medical imaging : extending the monai framework, 2023. URL <https://arxiv.org/abs/2307.15208>.
- Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. SDXL : Improving latent diffusion models for high-resolution image synthesis. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2307.01952>.
- Pôle d’expertise de la régulation numérique (PEReN). Détecter des contenus artificiels sur les réseaux sociaux : un outil pour savoir à quels détecteurs se fier. PEReN Lab, feb 2025. URL https://www.peren.gouv.fr/fr/perenlab/2025-02-11_ai_summit/.
- Hongyuan Qi, Wenjin Hou, Hehe Fan, and Jun Xiao. Deepfake detection generalization with diffusion noise, 2026. URL <https://arxiv.org/abs/2604.14570>.
- Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks, 2016. URL <https://arxiv.org/abs/1511.06434>.
- Jonas Ricker, Simon Damm, Thorsten Holz, and Asja Fischer. Towards the detection of diffusion model deepfakes, 2024a. URL <https://arxiv.org/abs/2210.14571>.
- Jonas Ricker, Denis Lukovnikov, and Asja Fischer. Aeroblade : Training-free detection of latent diffusion images using autoencoder reconstruction error, 2024b. URL <https://arxiv.org/abs/2401.17879>.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2022. URL <https://arxiv.org/abs/2112.10752>.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net : Convolutional networks for biomedical image segmentation, 2015. URL <https://arxiv.org/abs/1505.04597>.
- Piyush Sharma, Nan Ding, Sebastian Goodman, and Radu Soricut. Conceptual captions : A cleaned hypernymed, image alt-text dataset for automatic image captioning. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1 : Long Papers)*, pages 2556–2565. Association for Computational Linguistics, 2018. doi : 10.18653/v1/P18-1238.
- Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics, 2015. URL <https://arxiv.org/abs/1503.03585>.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models, 2022. URL <https://arxiv.org/abs/2010.02502>.

- Zhihang Song, Zimin He, Xingyu Li, Qiming Ma, RuiBo Ming, Zhiqi Mao, Huaxin Pei, Lihui Peng, Jianming Hu, Danya Yao, and Yi Zhang. Synthetic datasets for autonomous driving : A survey. *IEEE Transactions on Intelligent Vehicles*, 9(1) :1847–1864, January 2024. ISSN 2379-8858. doi : 10.1109/tiv.2023.3331024. URL <http://dx.doi.org/10.1109/TIV.2023.3331024>.
- Stability AI. Stable diffusion 3.5 large. Hugging Face model card, 2024. URL <https://huggingface.co/stabilityai/stable-diffusion-3.5-large>.
- Chuangchuan Tan, Yao Zhao, Shikui Wei, Guanghua Gu, Ping Liu, and Yunchao Wei. Frequency-aware deepfake detection : Improving generalizability through frequency space domain learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(5) :5052–5060, 2024. doi : 10.1609/aaai.v38i5.28310.
- Ana Vasilcoiu, Ivona Najdenkoska, Zeno Geradts, and Marcel Worring. Latte : Latent trajectory embedding for diffusion-generated image detection, 2025. URL <https://arxiv.org/abs/2507.03054>.
- Sheng-Yu Wang, Oliver Wang, Richard Zhang, Andrew Owens, and Alexei A. Efros. Cnn-generated images are surprisingly easy to spot... for now, 2020. URL <https://arxiv.org/abs/1912.11035>.
- Xi Wang and Vicky Kalogeiton. Your Diffusion Model Is an Implicit Synthetic Image Detector, September 2024. URL <https://hal.science/hal-04713283>. Proceedings of the European Conference on Computer Vision, Milan, Italy.
- Zhendong Wang, Jianmin Bao, Wengang Zhou, Weilun Wang, Hezhen Hu, Hong Chen, and Houqiang Li. Dire for diffusion-generated image detection, 2023. URL <https://arxiv.org/abs/2303.09295>.
- Lilian Weng. From autoencoder to beta-vae. *lilianweng.github.io*, 2018. URL <https://lilianweng.github.io/posts/2018-08-12-vae/>.
- Lilian Weng. From gan to wgan, 2019.
- Lilian Weng. What are diffusion models? *lilianweng.github.io*, Jul 2021. URL <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>.
- Zhiyuan Yan, Yong Zhang, Xinhang Yuan, Siwei Lyu, and Baoyuan Wu. Deepfakebench : A comprehensive benchmark of deepfake detection, 2023. URL <https://arxiv.org/abs/2307.01426>.
- Ling Yang, Zhilong Zhang, Yang Song, Shenda Hong, Runsheng Xu, Yue Zhao, Wentao Zhang, Bin Cui, and Ming-Hsuan Yang. Diffusion models : A comprehensive survey of methods and applications, 2025. URL <https://arxiv.org/abs/2209.00796>.
- Shamim Yazdani, Akansha Singh, Nripsuta Saxena, Zichong Wang, Avash Palikhe, Deng Pan, Umapada Pal, Jie Yang, and Wenbin Zhang. Generative ai in depth : A survey of recent advances, model variants, and real-world applications, 2025. URL <https://arxiv.org/abs/2510.21887>.
- Yichi Zhang and Xiaogang Xu. Diffusion noise feature : Accurate and fast generated image detection, October 2025. ISSN 1879-8314. URL <https://doi.org/10.3233/FAIA250925>. ECAI 2025.

Annexe A

Autres modèles génératifs

A.1 Autoencodeurs variationnels

Les autoencodeurs (Hinton and Salakhutdinov, 2006) sont des réseaux de neurones qui apprennent à reconstruire une entrée. Une donnée x est transformée par un encodeur g_ϕ en une représentation latente, généralement de plus faible dimension, puis reconstruite par un décodeur $f_\theta : x' = f_\theta(g_\phi(x))$. Les paramètres (ϕ, θ) sont appris afin que x' soit aussi proche que possible de x (Weng, 2018). Cette architecture est représentée sur la figure A.1.

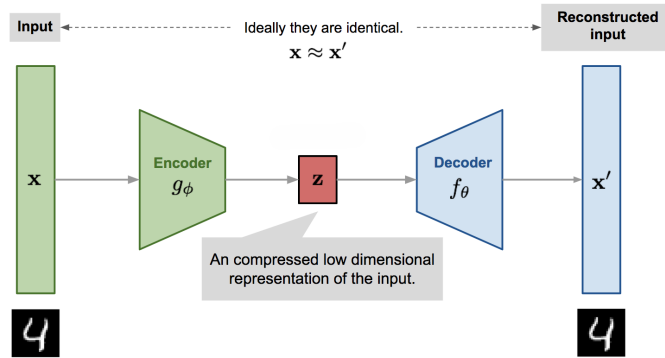


FIGURE A.1 – Architecture d’un autoencodeur Weng (2018).

Les autoencodeurs variationnels (VAE), introduits par Kingma et Welling (Kingma and Welling, 2022), constituent une extension probabiliste des autoencodeurs classiques. Au lieu d’associer chaque observation à un vecteur latent fixe, ils la représentent par une distribution de probabilité dans l’espace latent. Ils permettent ainsi de reconstruire les données, mais également d’en générer de nouvelles.

L’encodeur paramètre une distribution variationnelle $q_\phi(z|x)$, utilisée pour approximer la distribution postérieure $p_\theta(z|x)$ des variables latentes z , tandis que le décodeur paramètre la distribution $p_\theta(x|z)$ (Bond-Taylor et al., 2022; Yazdani et al., 2025).

L’objectif théorique est de maximiser la vraisemblance marginale :

$$p_\theta(x) = \int p_\theta(x|z)p(z) dz$$

Cette intégrale étant généralement intraitable (Weng, 2018), la distribution postérieure $p_\theta(z|x)$ ne peut pas être calculée directement. Elle est donc approximée par la distribution variationnelle $q_\phi(z|x)$ paramétrée par l’encodeur.

Les paramètres ϕ et θ sont appris conjointement en maximisant la borne ELBO (*Evidence Lower Bound*) :

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{\text{KL}}(q_\phi(z|x) \| p(z))$$

Cette fonction comporte deux termes (Yazdani et al., 2025) :

- un terme de reconstruction, qui favorise une reconstruction fidèle de l'entrée ;
- un terme de régularisation D_{KL} , qui permet à la distribution $q_\phi(z|x)$ de rester proche de la distribution a priori $p(z)$.

L'ELBO constitue une borne inférieure de la log-vraisemblance marginale :

$$\log p_\theta(x) = \mathcal{L}_{\text{ELBO}} + D_{\text{KL}}(q_\phi(z|x) \| p_\theta(z|x)) \geq \mathcal{L}_{\text{ELBO}}$$

L'échantillonnage de z étant stochastique, il n'est pas directement différentiable par rapport aux paramètres prédits par l'encodeur. Kingma et Welling introduisent donc la technique de reparamétrisation (Kingma and Welling, 2022), qui isole l'aléa dans une variable auxiliaire ε .

La distribution variationnelle est généralement choisie comme une loi gaussienne multivariée à covariance diagonale :

$$q_\phi(z|x) = \mathcal{N}(z; \mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$$

où $\mu_\phi(x)$ et $\sigma_\phi(x)$ sont les vecteurs des moyennes et des écarts-types prédits par l'encodeur. Nous avons alors :

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

où $\odot$ désigne le produit de Hadamard, c'est-à-dire le produit terme à terme.

L'aléa est ainsi contenu dans ε , tandis que z reste différentiable par rapport à $\mu_\phi(x)$ et $\sigma_\phi(x)$, ce qui permet la rétropropagation des gradients jusqu'aux paramètres ϕ .

L'architecture d'un VAE gaussien multivarié est représentée sur la figure A.2.

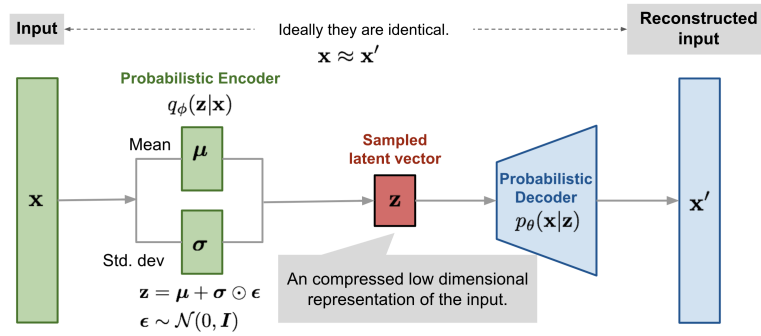


FIGURE A.2 – Architecture d'un VAE gaussien multivarié (Weng, 2018).

Une fois entraîné, le modèle peut générer de nouvelles données en échantillonnant un vecteur latent z selon la distribution a priori, puis en le décodant (Oussidi and Elhassouny, 2018). Les VAE peuvent toutefois produire des images relativement lisses ou floues, certains détails fins pouvant être perdus lors de la reconstruction (Bond-Taylor et al., 2022).

A.2 Réseaux antagonistes génératifs

Les réseaux antagonistes génératifs, ou *Generative Adversarial Networks* (GAN), ont été introduits par Goodfellow et al. (2014). Contrairement aux VAE, ils apprennent implicitement la distribution des données (Yazdani et al., 2025). Leur principe repose sur la mise en compétition de deux réseaux (Oussidi and Elhassouny, 2018) :

- le générateur $\mathcal{G}$ transforme un bruit aléatoire $z \sim p_z$ en données synthétiques et cherche à reproduire la distribution réelle p_{data} ;
- le discriminateur $\mathcal{D}$ distingue les données réelles des données générées et estime la probabilité qu'un échantillon soit réel.

Le générateur cherche ainsi à tromper le discriminateur, tandis que celui-ci apprend à mieux distinguer les deux types de données. Cette architecture est représentée sur la figure A.3.

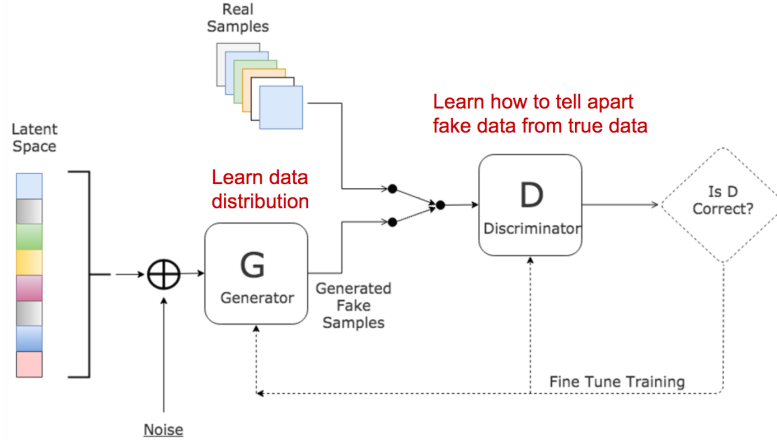


FIGURE A.3 – Architecture d'un réseau antagoniste génératif (Weng, 2019).

L'apprentissage vise théoriquement un équilibre de Nash (Yang et al., 2025), atteint lorsque la distribution générée p_g devient identique à la distribution réelle p_{data} . La fonction objectif s'écrit :

$$\min_{\mathcal{G}} \max_{\mathcal{D}} \mathbb{E}_{x \sim p_{data}(x)} [\log \mathcal{D}(x)] + \mathbb{E}_{z \sim p_z(z)} [\log (1 - \mathcal{D}(\mathcal{G}(z)))]$$

Pour un générateur $\mathcal{G}$ fixé, le discriminateur optimal est Weng (2019) :

$$\mathcal{D}^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

En remplaçant $\mathcal{D}$ par $\mathcal{D}^*$ dans la fonction objectif, Goodfellow et al. obtiennent :

$$2D_{JS}(p_{data} || p_g) - 2 \log 2$$

où D_{JS} désigne la divergence de Jensen-Shannon¹.

Lorsque le discriminateur est optimal, minimiser la fonction objectif par rapport au générateur revient donc à minimiser cette divergence. Le minimum est atteint lorsque $p_g = p_{data}$. Dans ce cas, $\mathcal{D}^*(x) = \frac{1}{2}$, le discriminateur ne peut plus distinguer les données réelles des données générées et la fonction objectif vaut $-\log 4$, ce qui correspond à l'équilibre de Nash.

L'entraînement des GAN reste toutefois instable en raison de sa nature adversariale (Weng, 2019). Si le discriminateur devient trop performant, le générateur peut recevoir des gradients très faibles, ce qui ralentit son apprentissage.

Une autre difficulté est le *mode collapse*, ou effondrement des modes : plusieurs vecteurs latents différents produisent alors des sorties identiques ou très proches. Le générateur ne représente plus qu'une partie de la distribution réelle et perd en diversité.

Plusieurs variantes ont été proposées pour améliorer la stabilité de l'entraînement, notamment DCGAN (Radford et al., 2016), WGAN (Arjovsky et al., 2017) et SN-GAN (Miyato et al., 2018), qui modifient respectivement l'architecture, la fonction objectif ou les contraintes imposées au discriminateur.

1. La divergence de Jensen-Shannon mesure la similarité entre deux distributions : $D_{JS}(P || Q) = \frac{1}{2} D_{KL}(P || M) + \frac{1}{2} D_{KL}(Q || M)$, avec $M = \frac{1}{2}(P + Q)$.

Annexe B

Métriques de classification et réseaux de neurones

B.1 Métriques d'évaluation pour la classification binaire

Dans un problème de classification binaire, chaque observation appartient à une classe positive ou négative, selon une convention propre au problème étudié. Le modèle produit généralement un score associé à la classe positive, comparé à un seuil de décision : au-dessus de ce seuil, l'observation est classée comme positive, sinon comme négative. Le choix du seuil influence plusieurs métriques d'évaluation.

La matrice de confusion compare les classes prédites aux classes réelles et comporte quatre valeurs :

	Réelle positive	Réelle négative
Prédite positive	Vrai positif (VP)	Faux positif (FP)
Prédite négative	Faux négatif (FN)	Vrai négatif (VN)

Les vrais positifs et vrais négatifs correspondent aux observations correctement classées. Les faux positifs sont des observations négatives classées à tort comme positives, tandis que les faux négatifs sont des observations positives classées comme négatives. La matrice de confusion permet ainsi d'analyser les erreurs et sert de base au calcul de plusieurs métriques.

L'*accuracy*, ou taux de bonnes classifications, mesure la proportion totale d'observations correctement classées :

$$\text{Accuracy} = \frac{\text{VP} + \text{VN}}{\text{VP} + \text{VN} + \text{FP} + \text{FN}}$$

Par exemple, une *accuracy* de 0,85 signifie que 85% des observations sont correctement classées. Cette métrique peut toutefois être trompeuse lorsque les classes sont déséquilibrées.

La *balanced accuracy* accorde la même importance aux deux classes et correspond à la moyenne des taux de vrais positifs et de vrais négatifs :

$$\text{Balanced Accuracy} = \frac{1}{2} \left(\frac{\text{VP}}{\text{VP} + \text{FN}} + \frac{\text{VN}}{\text{VN} + \text{FP}} \right)$$

La précision (*precision*) mesure la proportion d'observations réellement positives parmi celles prédites comme positives :

$$\text{Precision} = \frac{\text{VP}}{\text{VP} + \text{FP}}$$

Une précision élevée traduit donc un faible nombre de faux positifs.

Le rappel (*recall*), également appelé sensibilité ou taux de vrais positifs, mesure la proportion d'observations positives correctement détectées :

$$\text{Recall} = \frac{\text{VP}}{\text{VP} + \text{FN}}$$

Un rappel élevé traduit un faible nombre de faux négatifs. Il existe généralement un compromis entre précision et rappel : diminuer le seuil augmente souvent le rappel mais peut produire davantage de faux positifs, tandis que l'augmenter réduit généralement les faux positifs au risque de manquer davantage d'observations positives.

Le score F1 combine précision et rappel à l'aide de leur moyenne harmonique :

$$\text{F1} = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Il peut aussi s'écrire directement à partir de la matrice de confusion :

$$\text{F1} = \frac{2\text{VP}}{2\text{VP} + \text{FP} + \text{FN}}$$

Le score F1 n'est élevé que lorsque précision et rappel le sont également. Il est utile pour rechercher un compromis entre détection des observations positives et limitation des faux positifs. Il ne tient cependant pas directement compte des vrais négatifs et dépend du seuil de décision.

La courbe ROC (*Receiver Operating Characteristic*) évalue le modèle pour l'ensemble des seuils possibles en représentant le taux de vrais positifs (TPR) en fonction du taux de faux positifs (FPR). Le TPR correspond au rappel :

$$\text{TPR} = \frac{\text{VP}}{\text{VP} + \text{FN}}$$

Le FPR est défini par :

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{VN}}$$

La courbe ROC visualise ainsi le compromis entre détection des observations positives et production de faux positifs.

L'AUC (*Area Under the Curve*) correspond à l'aire sous la courbe ROC. Sa valeur est comprise entre 0 et 1. Une AUC proche de 1 indique une très bonne capacité de discrimination, tandis qu'une valeur de 0,5 correspond aux performances attendues d'un classement aléatoire. Une valeur inférieure à 0,5 indique que les scores sont globalement ordonnés dans le sens opposé à celui attendu.

Contrairement aux métriques précédentes, l'AUC ne dépend pas d'un seuil unique : elle mesure la capacité globale du modèle à ordonner les observations positives et négatives selon leurs scores.

Ces métriques sont donc complémentaires : l'*accuracy* mesure la proportion globale de prédictions correctes, la *balanced accuracy* équilibre l'importance des deux classes, la précision évalue la fiabilité des prédictions positives, le rappel la capacité à retrouver les observations positives, le score F1 combine précision et rappel, et l'AUC mesure la capacité globale de discrimination.

B.2 Réseaux de neurones et réseaux convolutifs

Cette section est en grande partie basée sur (O'Shea and Nash, 2015).

Les réseaux de neurones sont des modèles d'apprentissage automatique constitués de neurones organisés en couches successives. En apprentissage supervisé, ils apprennent une relation entre des données d'entrée et une sortie attendue, par exemple une classe.

Un neurone reçoit des valeurs $x_1, \dots, x_n$, calcule leur combinaison linéaire puis applique une fonction d'activation non linéaire :

$$y = \phi \left(\sum_{i=1}^n w_i x_i + b \right)$$

où $w_1, \dots, w_n$ sont les poids, b le biais et ϕ la fonction d'activation. Les poids et biais sont appris pendant l'entraînement.

La fonction d'activation ReLU (*Rectified Linear Unit*) est fréquemment utilisée :

$$\text{ReLU}(x) = \max(0, x)$$

Les fonctions d'activation non linéaires permettent au réseau de modéliser des relations plus complexes que de simples transformations linéaires.

Dans une couche entièrement connectée, ou couche linéaire, chaque neurone est relié à l'ensemble des valeurs de la couche précédente. Plusieurs couches linéaires successives, généralement séparées par des fonctions d'activation, forment un perceptron multicouche ou MLP (*Multi-Layer Perceptron*).

Lors de l'entraînement supervisé, une propagation vers l'avant produit une prédiction, comparée à la sortie attendue à l'aide d'une fonction de perte. L'erreur est ensuite rétropropagée afin de calculer les gradients et de mettre à jour les paramètres. Le processus est répété sur plusieurs époques jusqu'à obtenir un modèle capable de généraliser à de nouvelles données.

Les données sont généralement divisées en trois sous-ensembles. Le jeu d'entraînement sert à apprendre les paramètres du réseau. Le jeu de validation permet de sélectionner les hyperparamètres, de comparer les configurations et de surveiller le surapprentissage. Le jeu de test est réservé à l'évaluation finale, une fois l'architecture, les hyperparamètres et le seuil de décision fixés.

Plusieurs techniques permettent de limiter le surapprentissage. L'augmentation de données diversifie artificiellement les exemples d'entraînement, tandis que le *dropout* désactive aléatoirement certains neurones. La régularisation des poids et l'arrêt anticipé (*early stopping*) peuvent également améliorer la généralisation.

Les réseaux de neurones convolutifs, ou *Convolutional Neural Networks* (CNN), sont particulièrement adaptés au traitement des images. Contrairement aux couches entièrement connectées, les neurones d'une couche convolutionnelle ne traitent qu'une région locale de l'entrée, appelée champ réceptif. Cette propriété exploite la structure spatiale des images tout en limitant le nombre de paramètres.

Une couche convolutionnelle comporte plusieurs filtres, ou noyaux, généralement de taille spatiale 3×3 ou 5×5 . Chaque filtre est appliqué à différentes positions spatiales de l'entrée et calcule une combinaison pondérée des valeurs de son champ réceptif. Ses coefficients étant réutilisés à toutes les positions, ce partage des paramètres réduit fortement le nombre de paramètres à apprendre.

Chaque filtre produit une carte de caractéristiques. Ainsi, une couche comportant K filtres produit K cartes de caractéristiques, qui constituent les K canaux de sortie de la couche. Les premières couches détectent généralement des motifs locaux simples, comme des contours ou des textures, tandis que les couches plus profondes les combinent pour représenter des structures plus complexes.

Le déplacement des filtres est contrôlé par le *stride*. Un *stride* de 1 déplace le filtre d'une position, tandis qu'une valeur plus élevée réduit les dimensions spatiales de la sortie. Le *padding* ajoute des valeurs autour de l'entrée afin de contrôler les dimensions des cartes et de mieux prendre en compte les bords.

Les dimensions spatiales peuvent également être réduites par une opération de *pooling*. Le *max-pooling*, par exemple, conserve la valeur maximale de chaque région locale, tandis que le *average pooling* en calcule la valeur moyenne.

Un CNN est généralement constitué d’une succession de convolutions, de fonctions d’activation et d’opérations de réduction des dimensions. L’ensemble des cartes de caractéristiques peut ensuite être transformé en un vecteur par une opération de *flatten*, qui réorganise un tenseur de dimensions $C \times H \times W$ sans modifier ses valeurs. Des couches entièrement connectées (aussi appelées couches denses) utilisent ensuite ce vecteur pour produire les scores des différentes classes.

Dans une architecture de classification, la partie chargée d’extraire les cartes de caractéristiques est souvent appelée *backbone*. Elle est suivie d’une tête de classification produisant la prédiction finale.

Un exemple d’architecture classique de CNN est présenté sur la figure B.1.

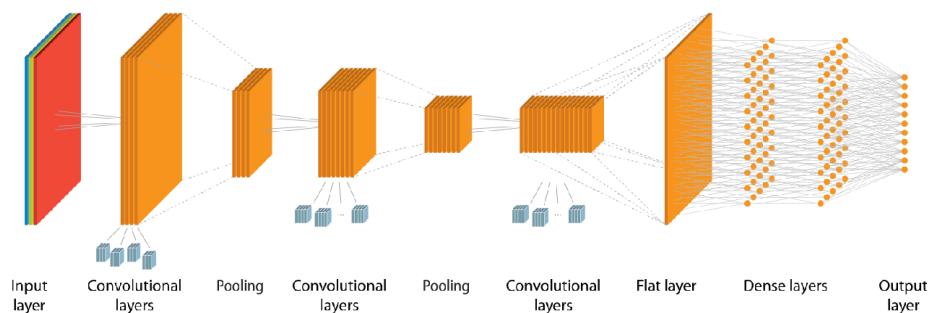


FIGURE B.1 – Architecture d’un CNN (FIDDLE, 2025).

L’entraînement d’un CNN repose sur les mêmes mécanismes que celui des autres réseaux de neurones : propagation avant, calcul de la fonction de perte, rétropropagation du gradient et mise à jour des paramètres. Pour une classification binaire, l’entropie croisée binaire est fréquemment utilisée comme fonction de perte.

B.3 Détail des architectures utilisées dans le rapport

B.3.1 Architectures associées à la *feature* pixel

Pour la *feature* pixel, l’entrée des modèles est la différence signée $\Delta_{\text{pixel}} \in \mathbb{R}^{3 \times 256 \times 256}$. Trois configurations ont été étudiées : un réseau convolutionnel entraîné de zéro (CNN-D1), un ResNet18 dont le *backbone* est entièrement gelé, et un ResNet18 dont seul le dernier bloc résiduel (*layer4*) est dégelé. Les figures B.2, B.3 et B.4 résument ces trois architectures.

CNN-D1

L’architecture CNN-D1 est un réseau convolutionnel simple conçu pour apprendre directement à partir des cartes Δ_{pixel} , sans transfert d’apprentissage. Sa structure est illustrée sur la figure B.2.

Elle prend en entrée un tenseur de taille $3 \times 256 \times 256$ puis applique quatre blocs convolutifs successifs, séparés par trois opérations de *pooling*. Les dimensions des représentations intermédiaires sont les suivantes :

$$(3, 256, 256) \rightarrow (32, 256, 256) \rightarrow (32, 128, 128) \rightarrow (64, 128, 128) \\ \rightarrow (64, 64, 64) \rightarrow (128, 64, 64) \rightarrow (128, 32, 32) \rightarrow (256, 32, 32).$$

Les cartes finales sont ensuite agrégées par un *global average pooling*, ce qui produit un vecteur de dimension 256. La tête de classification est un perceptron multicouche de dimensions : $256 \rightarrow 64 \rightarrow 1$.

la dernière sortie correspond à la prédiction binaire réel / synthétique.

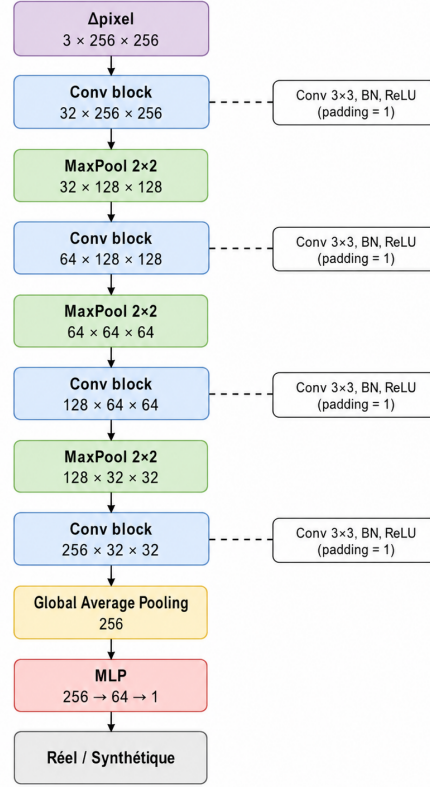


FIGURE B.2 – Architecture CNN-D1 utilisée pour la *feature* pixel.

ResNet18 avec *backbone* gelé

La deuxième configuration repose sur un ResNet18 pré-entraîné sur ImageNet. L'objectif est ici d'évaluer dans quelle mesure des représentations visuelles pré-apprises peuvent être réutilisées pour analyser les erreurs de reconstruction pixel, sans réentraîner le *backbone*. La figure B.3 présente cette configuration.

Après l'entrée $\Delta_{\text{pixel}} \in \mathbb{R}^{3 \times 256 \times 256}$, le réseau suit l'enchaînement standard de ResNet18 :

$$(3, 256, 256) \rightarrow (64, 128, 128) \rightarrow (64, 64, 64) \rightarrow (64, 64, 64) \rightarrow (128, 32, 32) \rightarrow (256, 16, 16) \rightarrow (512, 8, 8)$$

Plus précisément :

- **Conv1** : convolution 7×7 , *stride* 2 ;
- **MaxPool** : pooling 3×3 , *stride* 2 ;
- **Layer1 à Layer4** : quatre groupes de deux *BasicBlocks*.

Les cartes finales sont agrégées par un *global average pooling*, ce qui fournit un vecteur de dimension 512. La tête de classification remplaçant la tête originale de ResNet18 est de dimension : $512 \rightarrow 64 \rightarrow 1$.

Dans cette configuration, l'ensemble du *backbone* convolutionnel (*Conv1*, *Layer1* à *Layer4*) est gelé, et seule la tête dense est entraînée.

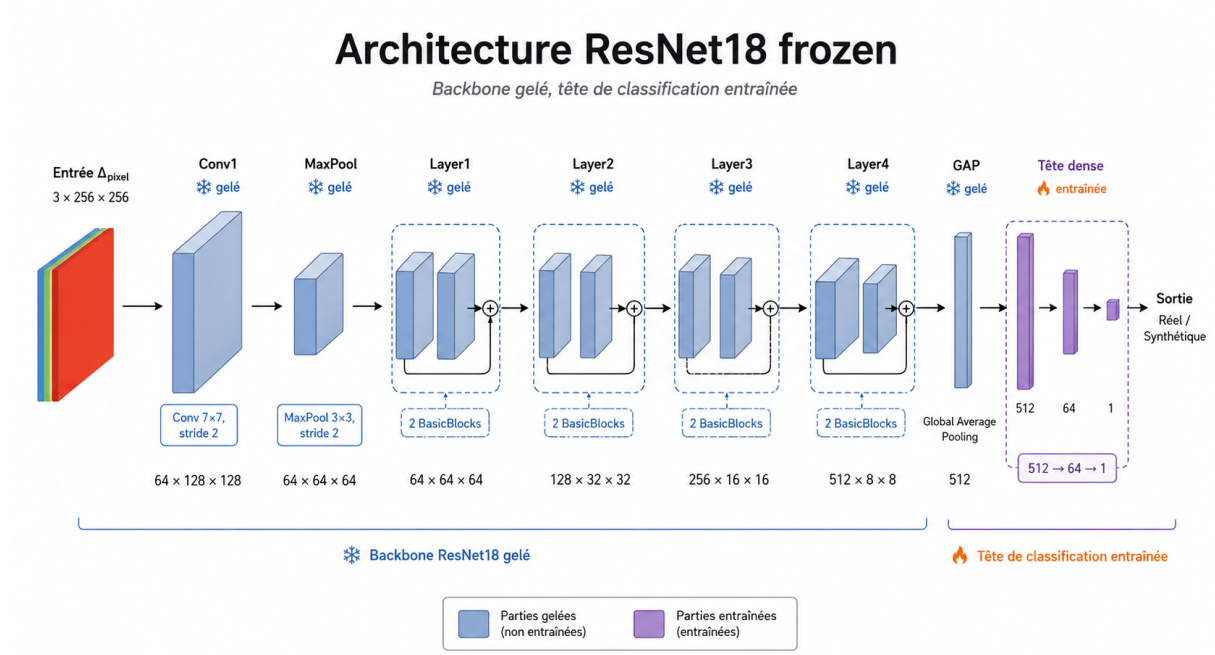


FIGURE B.3 – Architecture ResNet18 avec *backbone* entièrement gelé.

ResNet18 avec *layer4* dégelée

La troisième configuration reprend le même ResNet18 pré-entraîné, mais autorise cette fois l'adaptation du dernier bloc résiduel. L'idée est de conserver les représentations génériques apprises dans les premières couches tout en laissant les caractéristiques de plus haut niveau s'ajuster à la tâche étudiée. Cette configuration est illustrée sur la figure B.4.

Le chemin des dimensions intermédiaires reste identique à celui du ResNet18 précédent :

$$(3, 256, 256) \rightarrow (64, 128, 128) \rightarrow (64, 64, 64) \rightarrow (64, 64, 64) \\ \rightarrow (128, 32, 32) \rightarrow (256, 16, 16) \rightarrow (512, 8, 8) \rightarrow (512)$$

La tête de classification est également inchangée : $512 \rightarrow 64 \rightarrow 1$

La différence porte uniquement sur les paramètres entraînés :

- **gelés** : *Conv1*, *Layer1*, *Layer2* et *Layer3* ;
- **entraînés** : *Layer4* et la tête de classification.

Cette configuration correspond donc à un *fine-tuning* partiel du modèle.

B.3.2 Architectures associées à la *feature* latente

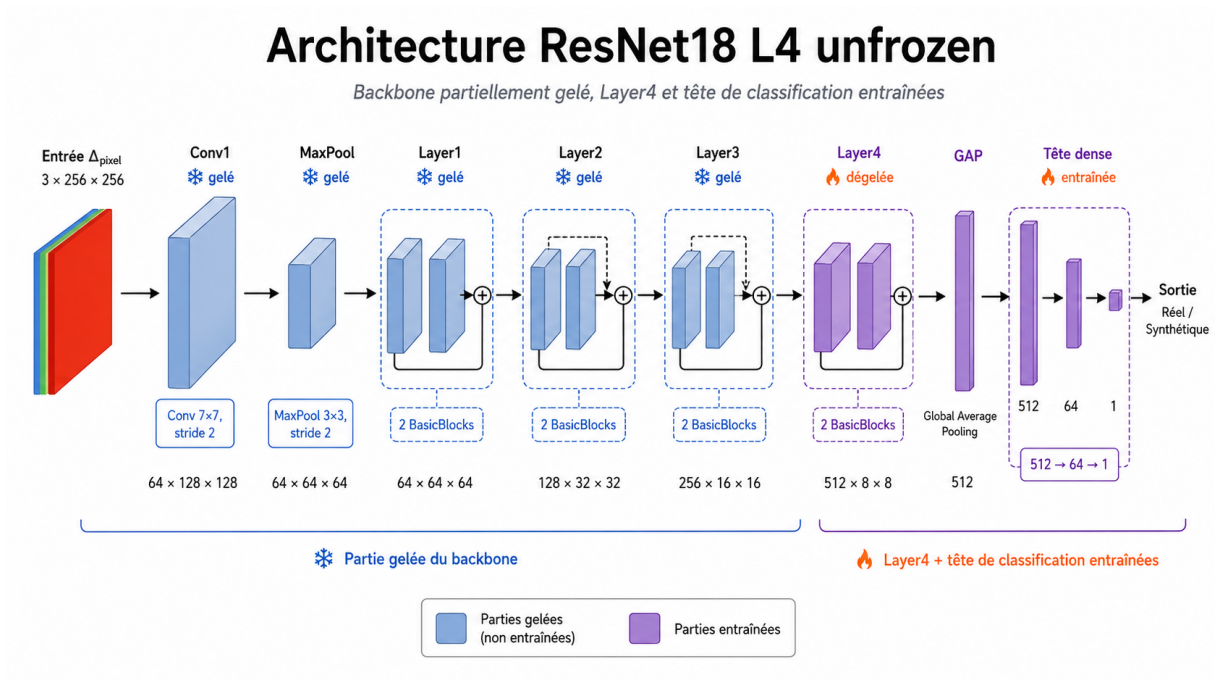
Trois architectures ont été étudiées pour exploiter les représentations latentes : *LatentUNetSmall*, *LatentUNetLarge* et *LatentSiameseCNN*. Les deux premières utilisent une concaténation de z , $\hat{z}_\alpha$ et de leur différence absolue, tandis que la troisième traite z et $\hat{z}_\alpha$ dans deux branches partageant les mêmes paramètres.

LatentUNetSmall

LatentUNetSmall utilise comme entrée

$$X_{\text{latent}} = \text{concat}(z, \hat{z}_\alpha, |z - \hat{z}_\alpha|)$$

soit un tenseur de 12 canaux. Pour des latents de taille $4 \times 32 \times 32$, l'entrée possède donc la dimension $12 \times 32 \times 32$.

FIGURE B.4 – Architecture ResNet18 avec $layer_4$ dégelée et tête de classification entraînée.

L'architecture correspond à un encodeur compact composé de trois blocs convolutionnels suivis d'un *bottleneck*. Les dimensions successives sont :

$$12 \times 32 \times 32 \rightarrow 16 \times 32 \times 32 \rightarrow 32 \times 16 \times 16 \rightarrow 64 \times 8 \times 8 \rightarrow 128 \times 4 \times 4$$

Chaque bloc convolutionnel est constitué de deux convolutions 3×3 , chacune suivie d'une *Group Normalization* et d'une activation GELU. Un *Dropout2d* est appliqué à la fin du bloc. La réduction spatiale entre deux blocs est réalisée par un *MaxPool* 2×2 .

Au niveau du *bottleneck*, un *global average pooling* et un *global max pooling* produisent deux vecteurs de dimension 128, qui sont concaténés : $128 + 128 = 256$.

La tête de classification suit alors : $256 \rightarrow 128 \rightarrow 64 \rightarrow 1$.

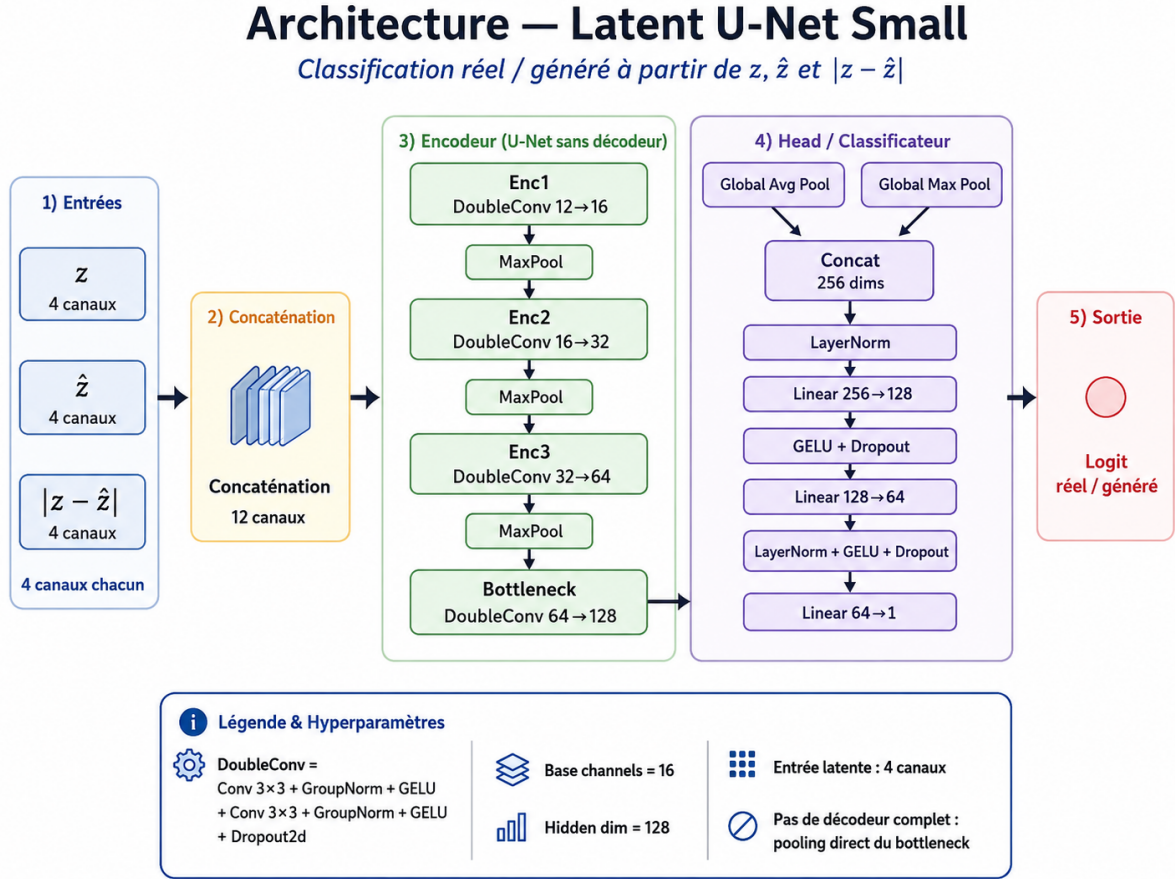


FIGURE B.5 – Architecture détaillée de LatentUNetSmall.

LatentUNetLarge

LatentUNetLarge utilise le même tenseur d'entrée $X_{\text{latent}} \in \mathbb{R}^{12 \times 32 \times 32}$, mais conserve une architecture encodeur-décodeur complète avec connexions de saut.

La partie encodeur produit les représentations suivantes :

$$12 \times 32 \times 32 \rightarrow 16 \times 32 \times 32 \rightarrow 32 \times 16 \times 16 \rightarrow 64 \times 8 \times 8 \rightarrow 128 \times 4 \times 4 \rightarrow 256 \times 2 \times 2$$

Le décodeur restaure ensuite progressivement la résolution spatiale. Chaque étape commence par une convolution transposée 2×2 de *stride* 2, suivie d'une concaténation avec la représentation correspondante de l'encodeur :

$$256 \times 2 \times 2 \rightarrow 128 \times 4 \times 4 \rightarrow 64 \times 8 \times 8 \rightarrow 32 \times 16 \times 16 \rightarrow 16 \times 32 \times 32$$

Les connexions de saut relient respectivement les représentations de mêmes résolutions de l'encodeur et du décodeur.

La sortie finale comporte 16 canaux. Le *global average pooling* et le *global max pooling* produisent donc un vecteur concaténé de dimension $16 + 16 = 32$.

La tête de classification est alors : $32 \rightarrow 128 \rightarrow 64 \rightarrow 1$.

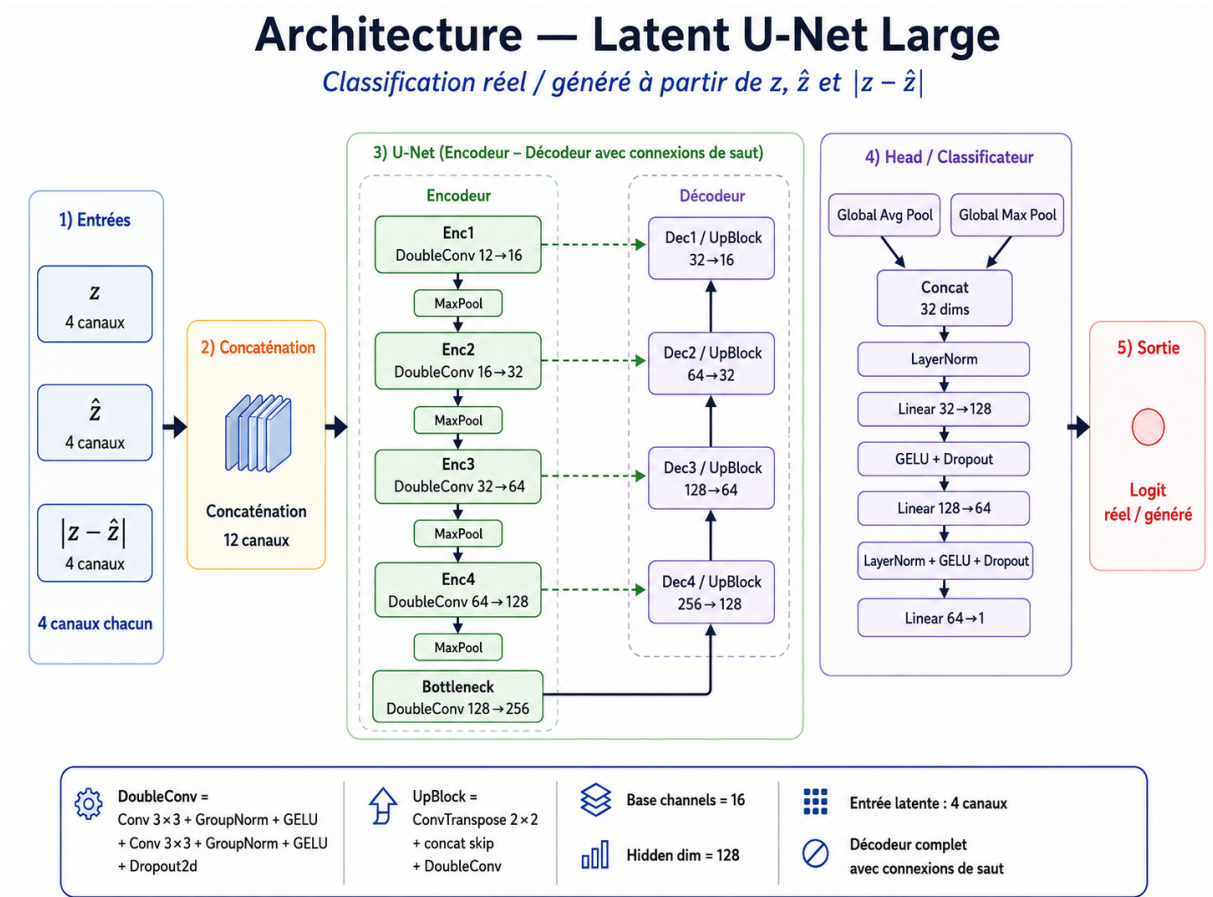


FIGURE B.6 – Architecture détaillée de LatentUNetLarge. Les connexions de saut relient les niveaux correspondants de l’encodeur et du décodeur.

LatentSiameseCNN

LatentSiameseCNN traite séparément le latent initial z et le latent reconstruit $\hat{z}_\alpha$. Chaque entrée possède la dimension $4 \times 32 \times 32$.

Les deux entrées traversent le même encodeur convolutionnel r_θ , les paramètres étant partagés entre les deux branches.

Pour chacune des branches, les dimensions évoluent selon :

$$4 \times 32 \times 32 \rightarrow 16 \times 32 \times 32 \rightarrow 32 \times 16 \times 16 \rightarrow 64 \times 8 \times 8 \rightarrow 128 \times 4 \times 4$$

Comme pour les architectures précédentes, chaque étage repose sur un double bloc convolutionnel et les changements de résolution sont obtenus par *MaxPool* 2×2 .

À la sortie de chaque branche, le *global average pooling* et le *global max pooling* donnent $128 + 128 = 256$ caractéristiques. Une projection linéaire les ramène à un vecteur d’*embedding* de dimension 128. On obtient ainsi deux représentations

$$e_z, \quad e_{\hat{z}} \in \mathbb{R}^{128}$$

Quatre informations sont ensuite concaténées :

$$e_z, \quad e_{\hat{z}}, \quad |e_z - e_{\hat{z}}|, \quad e_z \odot e_{\hat{z}}$$

Le vecteur de comparaison possède donc une dimension $4 \times 128 = 512$.

La tête de classification suit finalement $512 \rightarrow 128 \rightarrow 64 \rightarrow 1$.

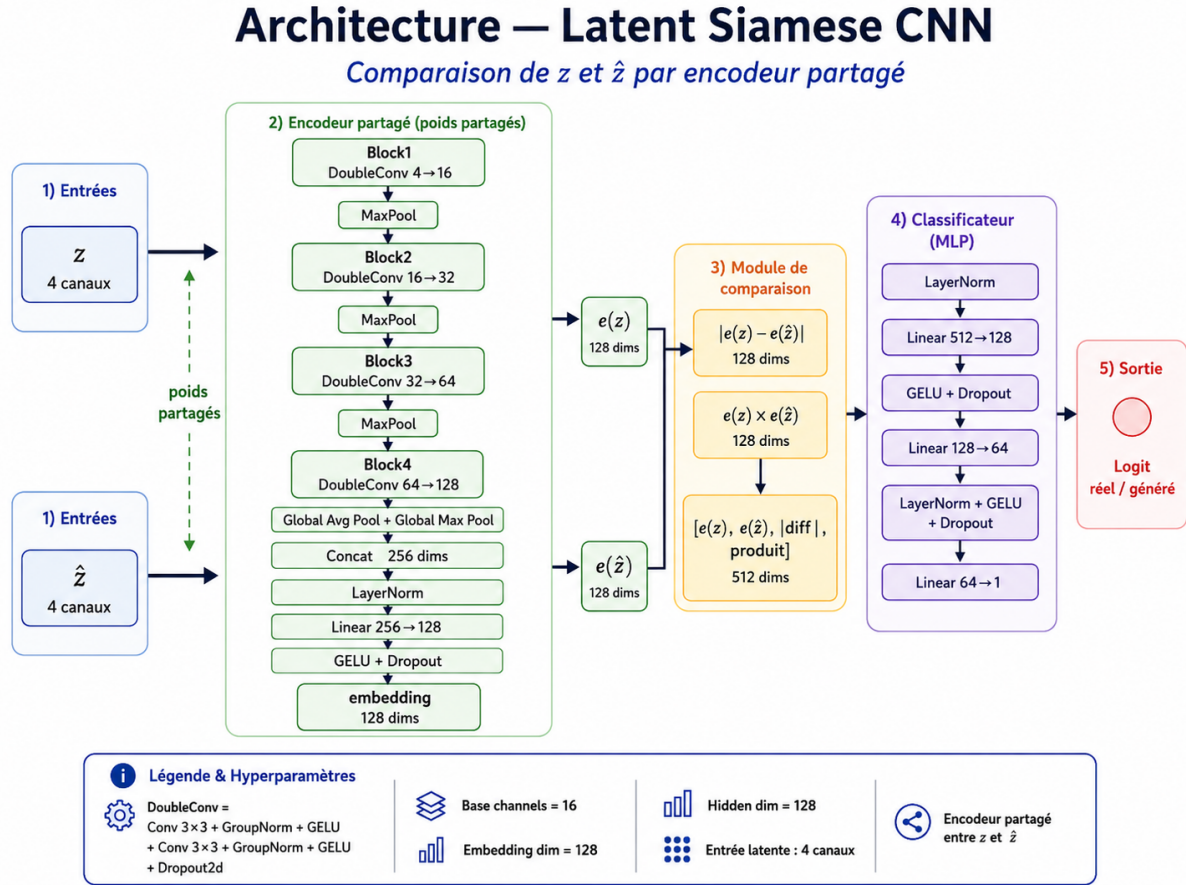


FIGURE B.7 – Architecture détaillée de LatentSiameseCNN. Les deux branches convolutionnelles partagent les mêmes paramètres.

B.3.3 Architectures associées à la *feature* bruit

Deux architectures ont été étudiées pour exploiter les représentations de bruit : *NoiseSRM-ResidualCNN* et *NoiseSiameseCNN*. La première travaille directement sur la différence $\Delta_{\text{bruit}} = \varepsilon - \hat{\varepsilon}_\alpha$, tandis que la seconde compare séparément le bruit injecté ε et le bruit prédit $\hat{\varepsilon}_\alpha$.

NoiseSRMResidualCNN

NoiseSRMResidualCNN prend en entrée

$$\Delta_{\text{bruit}} = \varepsilon - \hat{\varepsilon}_\alpha \in \mathbb{R}^{4 \times 32 \times 32}$$

Une première branche applique trois filtres SRM fixes de taille 5×5 indépendamment à chacun des quatre canaux. Elle produit donc $3 \times 4 = 12$ cartes supplémentaires. Celles-ci sont concaténées aux quatre canaux de la différence originale :

$$4 \times 32 \times 32 \longrightarrow 16 \times 32 \times 32$$

Le tenseur obtenu est ensuite projeté vers 32 canaux par le *stem* convolutionnel. Celui-ci contient deux convolutions 3×3 , la seconde étant appliquée avec un *stride* de 2 :

$$16 \times 32 \times 32 \rightarrow 32 \times 32 \times 32 \rightarrow 32 \times 16 \times 16$$

La représentation traverse ensuite quatre blocs résiduels :

$$32 \times 16 \times 16 \rightarrow 32 \times 16 \times 16 \rightarrow 64 \times 8 \times 8 \rightarrow 128 \times 4 \times 4 \rightarrow 256 \times 2 \times 2$$

Chaque bloc contient deux convolutions 3×3 avec normalisation *GroupNorm* et activation SiLU. Un *Dropout2d* de 0.10 est appliqué après la première convolution. Lorsque la résolution ou le nombre de canaux change, la connexion résiduelle utilise une convolution 1×1 .

À la sortie du dernier bloc, un *global average pooling* et un *global max pooling* sont appliqués en parallèle : $256 + 256 = 512$

Les deux vecteurs sont concaténés puis transmis à la tête de classification : $512 \rightarrow 1$ avec un *dropout* de 0.30 avant la couche linéaire.

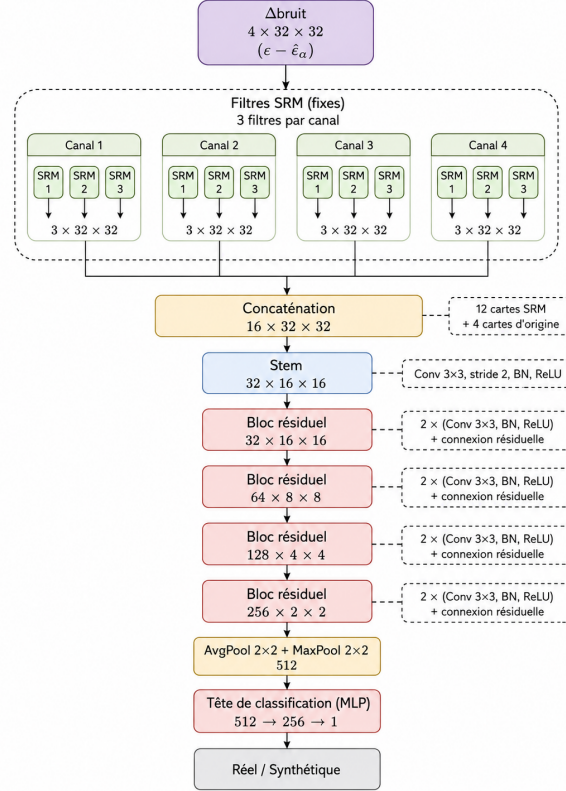


FIGURE B.8 – Architecture détaillée de NoiseSRMResidualCNN. Les réponses des trois filtres SRM appliqués aux quatre canaux sont concaténées à la différence de bruit avant les blocs résiduels.

NoiseSiameseCNN

L'architecture NoiseSiameseCNN reprend le même principe que LatentSiameseCNN présenté dans la section B.3.2. Les deux entrées sont ici le bruit injecté ε et le bruit prédit $\hat{\varepsilon}_\alpha$, tous deux de dimension $4 \times 32 \times 32$. Ils sont traités séparément par deux branches convolutionnelles partageant les mêmes paramètres, puis leurs représentations sont combinées avant la classification.

B.3.4 Architectures associées à la *feature* fréquence

Deux architectures ont été étudiées pour exploiter l'information fréquentielle contenue dans la différence pixel Δ_{pixel} : *AmpPhaseSiamese* et une adaptation de *FreqNet* (Tan et al., 2024). La première décompose explicitement le spectre en amplitude et phase, tandis que la seconde intègre des traitements fréquentiels directement dans les différentes étapes du réseau.

AmpPhaseSiamese

L'architecture AmpPhaseSiamese prend en entrée

$$\Delta_{\text{pixel}} \in \mathbb{R}^{3 \times 256 \times 256}$$

Une transformée de Fourier bidimensionnelle est tout d'abord appliquée indépendamment à chacun des trois canaux :

$$F = \mathcal{F}_{2D}(\Delta_{\text{pixel}})$$

Le spectre complexe obtenu est séparé en deux représentations :

$$A = |F|, \quad \phi = \arg(F)$$

correspondant respectivement à l'amplitude et à la phase.

L'amplitude est compressée logarithmiquement afin de limiter la dynamique des coefficients :

$$A_{\log} = \log(1 + A)$$

La phase étant périodique, elle est représentée par son sinus et son cosinus :

$$P = \text{concat}(\sin(\phi), \cos(\phi))$$

Cette représentation est également pondérée en fonction de l'amplitude afin de réduire l'influence de la phase lorsque l'énergie du coefficient fréquentiel est très faible.

Les représentations d'amplitude et de phase ne possédant pas le même nombre de canaux, chacune traverse d'abord une couche d'adaptation :

$$A_{\log} \longrightarrow A', \quad P \longrightarrow P'$$

Les deux tenseurs sont alors envoyés dans deux branches convolutionnelles partageant les mêmes paramètres s_{θ} :

$$e_A = s_{\theta}(A'), \quad e_P = s_{\theta}(P')$$

Les deux vecteurs obtenus sont ensuite fusionnés avant d'être transmis à la tête de classification. Deux mécanismes de fusion ont été considérés lors des expérimentations : une concaténation directe et une fusion pondérée (*gated fusion*).

Le fonctionnement général peut ainsi être résumé par :

$$\Delta_{\text{pixel}} \rightarrow \text{FFT} \rightarrow \begin{cases} \text{Amplitude} \rightarrow s_{\theta} \rightarrow e_A \\ \text{Phase} \rightarrow s_{\theta} \rightarrow e_P \end{cases} \rightarrow \text{Fusion} \rightarrow \text{Classification}$$

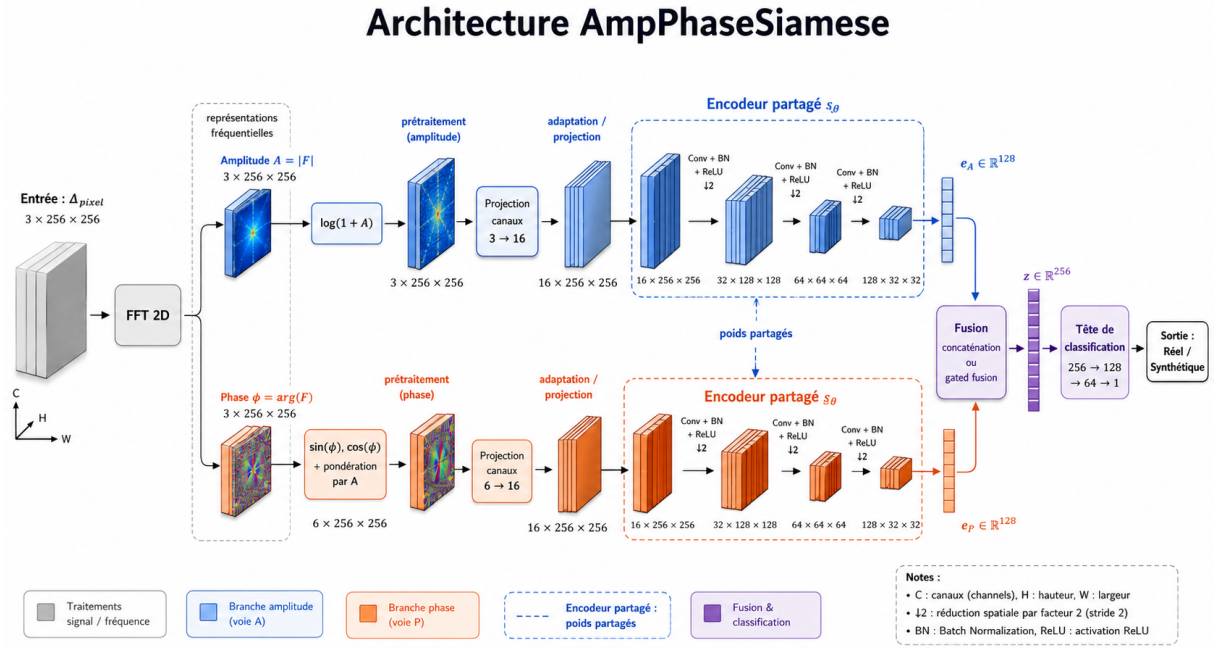


FIGURE B.9 – Architecture détaillée de AmpPhaseSiamese. Les représentations d’amplitude et de phase sont traitées par deux branches convolutionnelles partageant les mêmes paramètres avant leur fusion.

FreqNet

La seconde architecture est adaptée de FreqNet (Tan et al., 2024). Elle reçoit également

$$\Delta_{\text{pixel}} \in \mathbb{R}^{3 \times 256 \times 256}$$

mais, contrairement à AmpPhaseSiamese, elle ne sépare pas explicitement l’amplitude et la phase en deux branches. L’information fréquentielle est introduite directement dans les différentes étapes d’extraction des caractéristiques.

L’architecture repose principalement sur quatre types de modules : *HFRI*, *HFRFS*, *HFRFC* et *FCL*.

Module HFRI. Le module HFRI (*High-Frequency Representation of Image*) intervient au niveau de l’entrée. Il vise à mettre explicitement en évidence les composantes de haute fréquence de Δ_{pixel} avant leur traitement par les blocs suivants :

$$\Delta_{\text{pixel}} \longrightarrow \text{HFRI} \longrightarrow X_{\text{HF}}$$

Module HFRFS. Le module HFRFS (*High-Frequency Representation of Feature across Spatial dimensions*) agit sur les cartes de caractéristiques intermédiaires. Il extrait les composantes fréquentielles selon les deux dimensions spatiales afin de conserver les détails de haute fréquence au fur et à mesure de la profondeur du réseau.

Module HFRFC. Le module HFRFC (*High-Frequency Representation of Feature across Channel dimension*) complète HFRFS en exploitant les relations fréquentielles selon la dimension des canaux. Les deux modules permettent ainsi d’analyser les caractéristiques à la fois spatialement et entre canaux.

Frequency Convolution Layer. Les couches FCL (*Frequency Convolution Layer*) réalisent directement une partie du traitement dans le domaine fréquentiel. Pour une carte de caractéristiques X , le principe général est :

$$X \xrightarrow{\mathcal{F}} \mathcal{F}(X) \xrightarrow{\text{opérations apprises}} \tilde{F} \xrightarrow{\mathcal{F}^{-1}} \tilde{X}$$

La transformée de Fourier permet donc de travailler explicitement sur les composantes fréquentielles, avant de revenir dans le domaine spatial pour poursuivre l'extraction des caractéristiques.

L'architecture peut ainsi être résumée par :

$$\Delta_{\text{pixel}} \rightarrow \text{HFRI} \rightarrow [\text{HFRFS}, \text{HFRFC}, \text{FCL}] \rightarrow \dots \rightarrow \text{Global Average Pooling} \rightarrow \text{Classification}$$

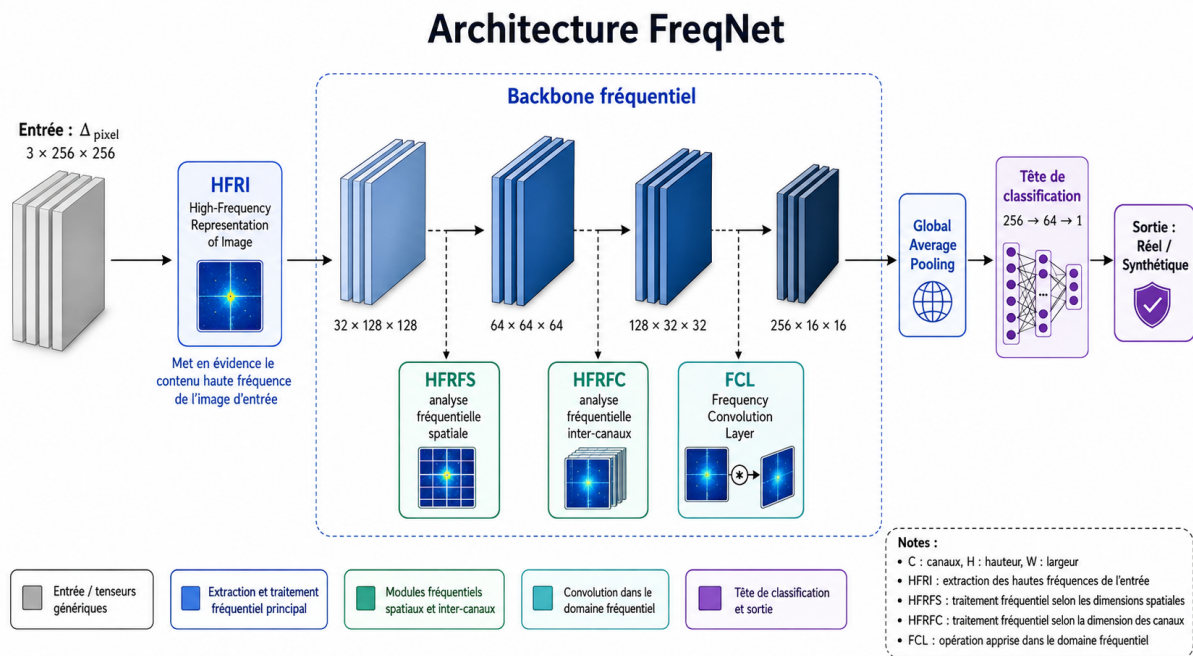


FIGURE B.10 – Architecture détaillée de FreqNet. Les modules HFRI, HFRFS, HFRFC et FCL introduisent progressivement des traitements fréquentiels dans les représentations intermédiaires du réseau.

B.3.5 Architecture associée à la *feature* ondelettes

Une seule architecture de classification, appelée *WaveletCNN*, est utilisée pour exploiter les représentations en ondelettes. Elle prend en entrée la différence pixel signée

$$\Delta_{\text{pixel}} \in \mathbb{R}^{3 \times 256 \times 256}$$

Avant l'entraînement final, plusieurs familles d'ondelettes et plusieurs profondeurs de décomposition ont été comparées. Les ondelettes Haar, Daubechies 2, Symlet 2 et Coiflet 1 ont été évaluées pour $L \in \{2, 3, 4\}$.

La configuration retenue sur le jeu de validation est **Coiflet 1 avec quatre niveaux de décomposition**.

Décomposition en ondelettes

Une DWT bidimensionnelle est appliquée indépendamment à chacun des trois canaux de Δ_{pixel} . À chaque niveau, elle produit quatre sous-bandes : LL , LH , HL , HH .

Pour les trois canaux, les sous-bandes sont concaténées, ce qui produit une représentation de $3 \times 4 = 12$ canaux à chaque niveau.

Seule la composante LL est à nouveau décomposée pour construire le niveau suivant. Pour une entrée de taille $3 \times 256 \times 256$, les quatre représentations utilisées par WaveletCNN ont ainsi les dimensions :

$$L_1 : 12 \times 128 \times 128$$

$$L_2 : 12 \times 64 \times 64$$

$$L_3 : 12 \times 32 \times 32$$

$$L_4 : 12 \times 16 \times 16$$

La décomposition peut donc être résumée par

$$3 \times 256 \times 256 \xrightarrow{\text{DWT Coiflet1}} \begin{cases} L_1 : 12 \times 128 \times 128 \\ L_2 : 12 \times 64 \times 64 \\ L_3 : 12 \times 32 \times 32 \\ L_4 : 12 \times 16 \times 16 \end{cases}$$

WaveletCNN

WaveletCNN fusionne progressivement les informations contenues aux différentes échelles. Le niveau L_1 est d'abord traité par une convolution de *stride* 2 afin de ramener sa résolution spatiale de 128×128 à 64×64 :

$$L_1 \rightarrow F_1^{(2)} \in \mathbb{R}^{C_1 \times 64 \times 64}$$

Cette représentation est concaténée avec L_2 :

$$F_2 = \text{concat} \left(F_1^{(2)}, L_2 \right)$$

Le même principe est ensuite répété pour les niveaux suivants :

$$F_2 \xrightarrow{\text{Conv}, s=2} F_2^{(2)} \rightarrow \text{concat}(F_2^{(2)}, L_3)$$

puis

$$F_3 \xrightarrow{\text{Conv}, s=2} F_3^{(2)} \rightarrow \text{concat}(F_3^{(2)}, L_4)$$

La progression spatiale de l'architecture est donc :

$$128 \times 128 \rightarrow 64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16$$

Contrairement à une concaténation directe de toutes les sous-bandes, cette construction aligne progressivement les différentes résolutions avant leur fusion.

Après l'intégration du niveau L_4 , une dernière convolution produit 128 cartes de caractéristiques :

$$F_{\text{final}} \in \mathbb{R}^{128 \times 16 \times 16}.$$

Un *global average pooling* réduit ensuite cette représentation à un vecteur :

$$128 \times 16 \times 16 \rightarrow 128$$

Ce vecteur est finalement transmis au perceptron multicouche chargé de la classification binaire :

$$128 \rightarrow \text{MLP} \rightarrow 1$$

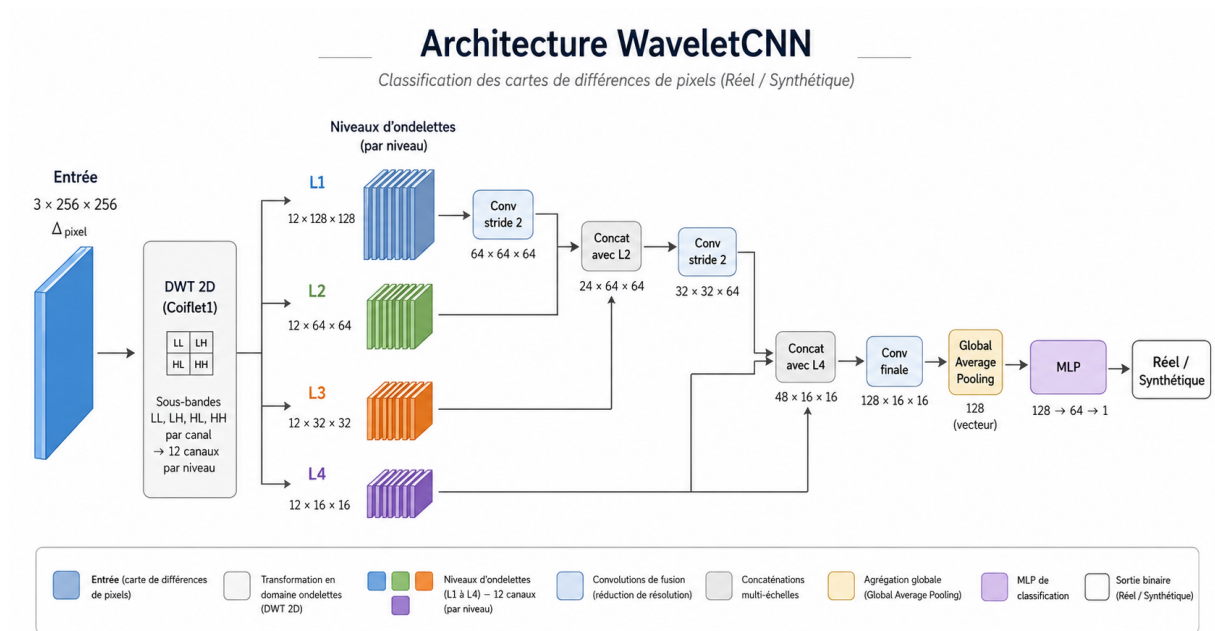


FIGURE B.11 – Architecture détaillée de WaveletCNN avec une décomposition Coiflet 1 à quatre niveaux. Les représentations issues des différentes échelles sont progressivement alignées spatialement puis concaténées avant la classification.

B.4 Comparaison détaillée des architectures

Cette section présente les performances des différentes architectures évaluées lors de la phase de sélection. Pour chaque modèle, les barres correspondent aux cinq valeurs de α et représentent la Balanced Accuracy moyenne sur les trois graines aléatoires, avec l'écart-type associé.

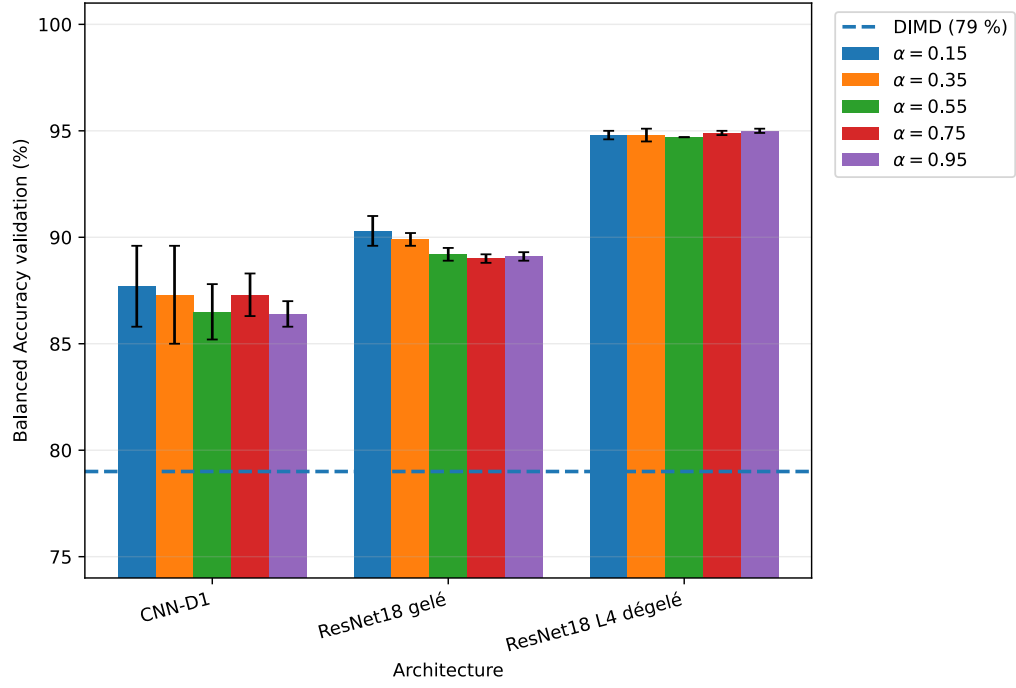


FIGURE B.12 – Comparaison des architectures étudiées pour la représentation pixel.

Représentation pixel.

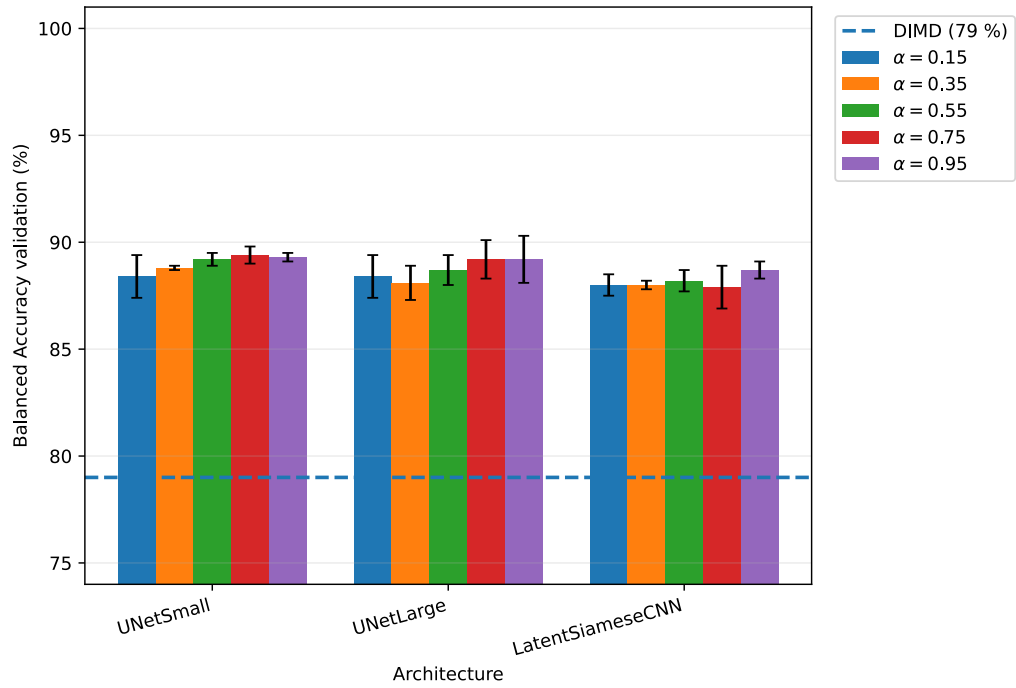


FIGURE B.13 – Comparaison des architectures étudiées pour la représentation latente.

Représentation latente.

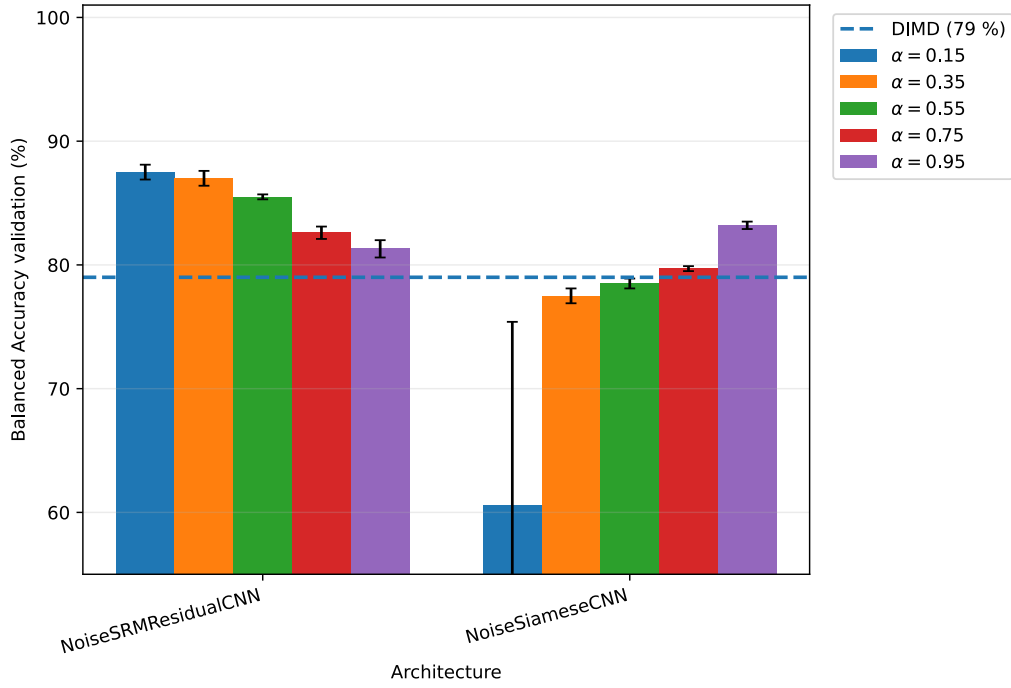


FIGURE B.14 – Comparaison des architectures étudiées pour la représentation bruit.

Représentation bruit.

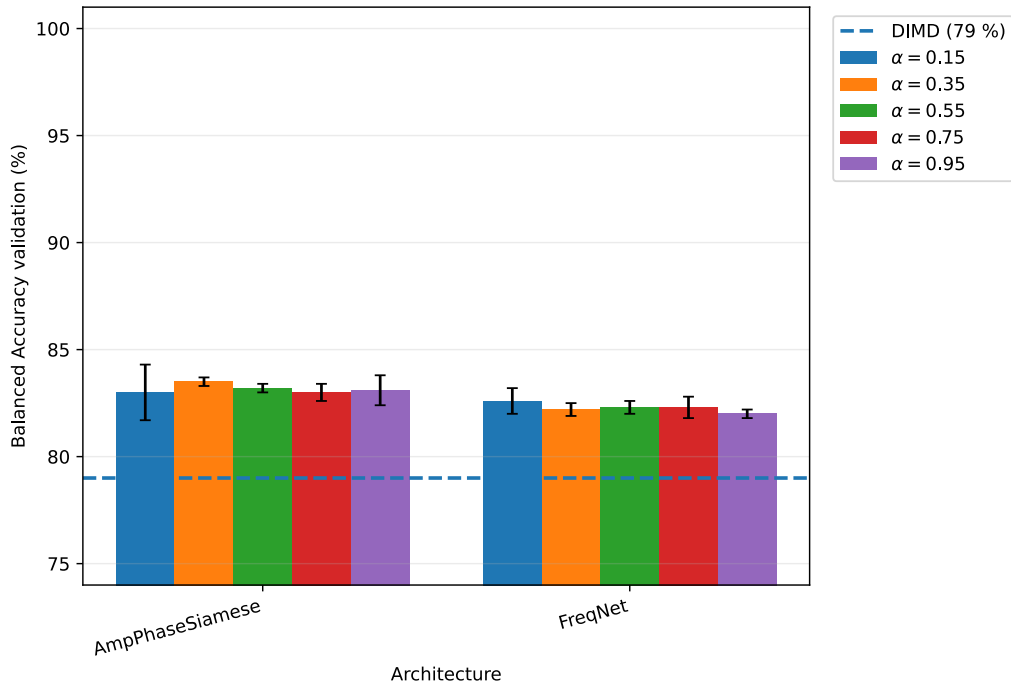


FIGURE B.15 – Comparaison des architectures étudiées pour la représentation fréquentielle.

Représentation fréquence.

Représentation ondelettes. Pour la représentation ondelettes, la comparaison porte d'abord sur le choix de la famille d'ondelettes et du niveau de décomposition. Les ondelettes Haar,

Daubechies 2 (*db2*), Symlet 2 (*sym2*) et Coiflet 1 (*coif1*) ont été évaluées pour des niveaux $L \in \{2, 3, 4\}$.

Afin de limiter le coût de cette recherche, cette première sélection a été réalisée pour $\alpha = 0,55$ et avec la graine 42. La configuration Coiflet 1 avec un niveau $L = 4$ obtient la meilleure performance sur le jeu de validation et est donc retenue pour les expériences finales sur les cinq valeurs de α et les trois graines aléatoires.

La figure B.16 présente les performances obtenues pour les différentes configurations testées.

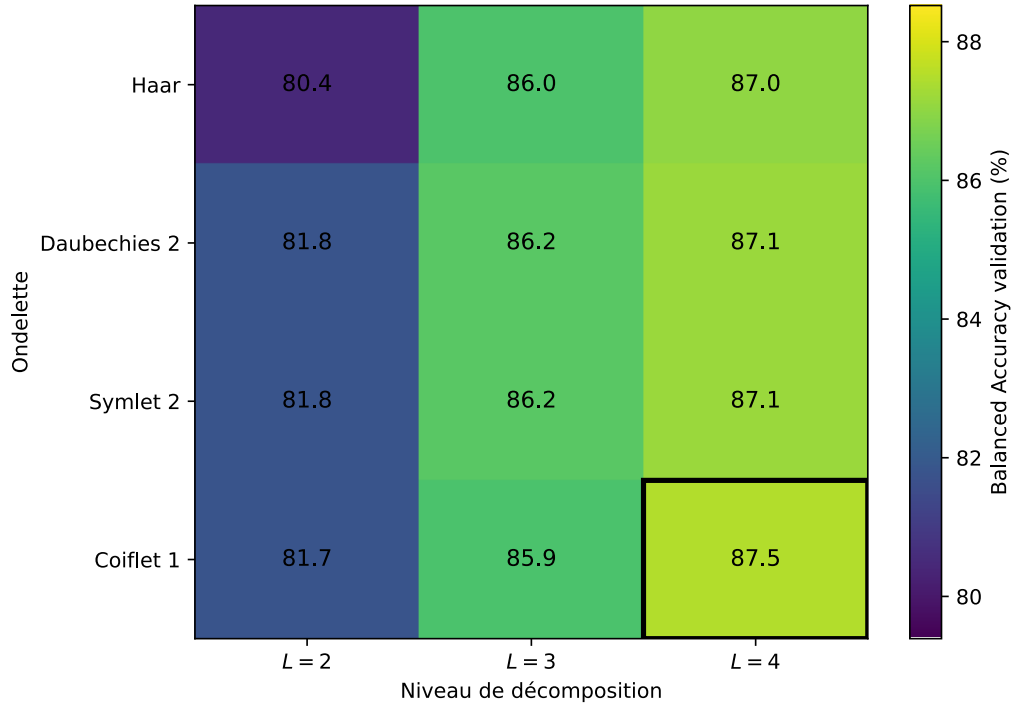


FIGURE B.16 – Balanced Accuracy obtenue sur le jeu de validation pour les différentes familles d'ondelettes et niveaux de décomposition testés, avec $\alpha = 0,55$ et la graine 42.

Annexe C

Analyse fréquentielle et décomposition en ondelettes

C.1 Analyse dans le domaine fréquentiel

Une image numérique est définie sur une grille bidimensionnelle de pixels. Une image en niveaux de gris associe une valeur à chaque pixel, tandis qu'une image en couleurs comporte généralement trois composantes : rouge, verte et bleue [Malgouyres \(2009\)](#).

Une image en couleurs de dimensions $M \times N$ peut ainsi être représentée par les valeurs $x_{m,n}^{(c)}$, où $m \in \{0, \dots, M-1\}$ et $n \in \{0, \dots, N-1\}$ désignent les indices de ligne et de colonne, et $c \in \{R, G, B\}$ le canal considéré. Les transformations fréquentielles peuvent être appliquées indépendamment à chaque canal.

La transformée de Fourier fournit une représentation de l'image dans une base de fonctions oscillatoires associées à différentes fréquences [Malgouyres \(2009\)](#). Pour chaque canal c , la transformée de Fourier discrète bidimensionnelle est définie par :

$$\hat{x}_{k,l}^{(c)} = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x_{m,n}^{(c)} \exp \left(-2i\pi \left(\frac{km}{M} + \frac{ln}{N} \right) \right)$$

où m et n repèrent les pixels, tandis que k et l correspondent aux fréquences spatiales discrètes selon les axes vertical et horizontal.

La transformée est inversible. Le canal initial est reconstruit par :

$$x_{m,n}^{(c)} = \frac{1}{MN} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} \hat{x}_{k,l}^{(c)} \exp \left(2i\pi \left(\frac{km}{M} + \frac{ln}{N} \right) \right)$$

La transformée de Fourier constitue une représentation globale : chaque coefficient dépend de l'ensemble des pixels du canal. Elle permet donc d'identifier les fréquences présentes dans l'image, mais pas de localiser précisément les régions où elles apparaissent [Malgouyres \(2009\)](#).

En pratique, elle est généralement calculée à l'aide de la transformée de Fourier rapide, ou *Fast Fourier Transform* (FFT). Pour une image carrée de dimensions $N \times N$, sa complexité est de l'ordre de $\mathcal{O}(N^2 \log N)$ [Malgouyres \(2009\)](#).

Pour une image en couleurs, la représentation fréquentielle complète peut être notée :

$$\hat{x} = \left(\hat{x}^{(R)}, \hat{x}^{(G)}, \hat{x}^{(B)} \right)$$

Les coefficients de Fourier sont complexes et peuvent être écrits sous forme polaire :

$$\hat{x}_{k,l}^{(c)} = A_{k,l}^{(c)} \exp \left(i\theta_{k,l}^{(c)} \right)$$

où $A_{k,l}^{(c)}$ représente l'amplitude et $\theta_{k,l}^{(c)}$ la phase.

L'amplitude, ou module du coefficient de Fourier, est définie par :

$$A_{k,l}^{(c)} = |\hat{x}_{k,l}^{(c)}| = \sqrt{\operatorname{Re} \left(\hat{x}_{k,l}^{(c)} \right)^2 + \operatorname{Im} \left(\hat{x}_{k,l}^{(c)} \right)^2}$$

Elle mesure l'importance de la composante fréquentielle associée aux indices (k, l) : une amplitude élevée indique que cette fréquence est fortement présente dans le canal considéré.

La phase est définie par :

$$\theta_{k,l}^{(c)} = \arg \left(\hat{x}_{k,l}^{(c)} \right)$$

L'amplitude renseigne ainsi sur l'importance des fréquences, tandis que la phase contient des informations sur l'organisation spatiale des structures et l'alignement des composantes fréquentielles.

C.2 Décomposition en ondelettes

Contrairement à la transformée de Fourier, qui fournit une représentation fréquentielle globale, la transformée en ondelettes permet une analyse à la fois spatiale et fréquentielle, ainsi qu'à plusieurs niveaux de détail [Charles \(2010\)](#).

Une ondelette est une fonction oscillante et localisée. À partir d'une ondelette mère, des fonctions d'analyse sont obtenues par translation, pour étudier différentes positions, et par changement d'échelle, pour analyser des structures de tailles différentes.

Dans le cas discret, la transformée repose sur une banque de filtres : un filtre passe-bas h , qui conserve principalement les variations lentes, et un filtre passe-haut g , qui extrait les variations rapides. Les résultats sont ensuite sous-échantillonnés d'un facteur deux [Charles \(2010\)](#). Pour un signal discret a_j au niveau j :

$$a_{j+1}[k] = \sum_n h[n - 2k] a_j[n], \quad d_{j+1}[k] = \sum_n g[n - 2k] a_j[n]$$

où a_{j+1} désigne les coefficients d'approximation et d_{j+1} les coefficients de détail. Le terme $2k$ traduit le sous-échantillonnage d'un facteur deux.

Pour une image bidimensionnelle, la décomposition est généralement réalisée de manière séparable sur chaque canal. Les filtres sont d'abord appliqués selon une première dimension :

$$L_c[p, n] = \sum_m h[m - 2p] x_{m,n}^{(c)} \quad H_c[p, n] = \sum_m g[m - 2p] x_{m,n}^{(c)}$$

où L_c contient les composantes de basse fréquence et H_c celles de haute fréquence selon cette première dimension.

Ces deux images sont ensuite filtrées selon la seconde dimension, produisant quatre sous-bandes :

$$\begin{aligned} LL_c[p, q] &= \sum_n h[n - 2q] L_c[p, n] & LH_c[p, q] &= \sum_n g[n - 2q] L_c[p, n] \\ HL_c[p, q] &= \sum_n h[n - 2q] H_c[p, n] & HH_c[p, q] &= \sum_n g[n - 2q] H_c[p, n] \end{aligned}$$

Les indices p et q repèrent les positions dans les sous-bandes, tandis que les termes $2p$ et $2q$ correspondent au sous-échantillonnage selon les deux dimensions.

La première lettre indique le filtrage appliqué selon la première dimension et la seconde celui appliqué selon la seconde. Ainsi, LL résulte de deux filtrages passe-bas, LH et HL combinent un

filtrage passe-bas et un filtrage passe-haut, et HH résulte de deux filtrages passe-haut [Malgouyres \(2009\)](#).

Avec le mode de périodisation utilisé dans ce travail, pour une image de dimensions paires $M \times N$, chaque sous-bande possède des dimensions $\frac{M}{2} \times \frac{N}{2}$. Pour une image en couleurs, chacune conserve trois canaux : Pour une image en couleurs, chacune conserve trois canaux :

$$LL = (LL_R, LL_G, LL_B)$$

et de même pour LH , HL et HH . Une décomposition à un niveau produit donc quatre tenseurs à trois canaux, soit douze cartes bidimensionnelles considérées séparément.

Les images étant de taille finie, les filtres peuvent dépasser leurs limites au voisinage des bords. Un mode de prolongement doit alors être choisi, par exemple symétrique, périodique ou par réflexion. Ce choix influence principalement les coefficients proches des bords, notamment pour les filtres à support étendu.

La décomposition peut être répétée afin d'obtenir une représentation multi-résolution. Dans une décomposition classique, seule la sous-bande LL du niveau précédent est à nouveau décomposée, tandis que LH , HL et HH sont conservées [Malgouyres \(2009\)](#). Le niveau j est ainsi obtenu par :

$$LL_{j-1} \longrightarrow \{LL_j, LH_j, HL_j, HH_j\} \quad 1 \leq j \leq J$$

Après J niveaux, la représentation est constituée de la dernière approximation LL_J et des trois sous-bandes de détail de chaque niveau :

$$\mathcal{W}(x) = \left\{ LL_J, (LH_j, HL_j, HH_j)_{j=1}^J \right\}$$

Elle comporte donc $3J + 1$ sous-bandes : une approximation finale et trois sous-bandes de détail par niveau [Charles \(2010\)](#).

Avec le mode de périodisation utilisé, pour une image de dimensions $M \times N$ divisibles par 2^J , les sous-bandes du niveau j ont pour dimensions $\frac{M}{2^j} \times \frac{N}{2^j}$.

Chaque famille d'ondelettes possède des filtres différents. Leur choix dépend notamment du nombre de moments nuls, qui traduit la capacité à représenter efficacement les régions régulières de l'image, et de la taille du support, liée à la localisation spatiale. Il existe ainsi un compromis entre la représentation des zones régulières et la localisation des bords [Malgouyres \(2009\)](#).

Ondelette de Haar

L'ondelette de Haar est l'ondelette discrète orthogonale la plus simple. Elle possède un moment nul et un support très court. Ses filtres passe-bas et passe-haut sont :

$$h_{\text{Haar}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} \quad g_{\text{Haar}} = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 & 1 \end{bmatrix}$$

Le filtre passe-bas combine deux valeurs voisines pour représenter leur composante de basse fréquence, tandis que le filtre passe-haut représente leur différence [Charles \(2010\)](#). Grâce à son support très court, Haar possède une bonne localisation spatiale et permet de mettre en évidence les changements brusques, tels que les contours. Sa forme discontinue la rend cependant moins adaptée à la représentation des variations progressives et des régions très régulières.

Ondelette de Daubechies d'ordre 2

Les ondelettes de Daubechies sont des ondelettes orthogonales à support compact [Daubechies \(1988\)](#). L'ondelette de Daubechies d'ordre 2 possède deux moments nuls et un support plus étendu que celui de Haar. Elle permet ainsi de mieux représenter les régions régulières, au prix d'une localisation spatiale moins fine.

Ondelette Symlet d'ordre 2

Les Symlets sont des variantes des ondelettes de Daubechies conçues pour réduire leur asymétrie tout en conservant une base orthogonale et un support compact Daubechies (1993). Elles présentent généralement des propriétés proches des ondelettes de Daubechies de même ordre. À l'ordre 2, la Symlet possède deux moments nuls et coïncide avec l'ondelette de Daubechies d'ordre 2 dans les implémentations usuelles.

Ondelette Coiflet d'ordre 1

Les Coiflets sont des ondelettes orthogonales à support compact Daubechies (1993). Leur construction impose des propriétés de moments nuls à la fois à l'ondelette et à la fonction d'échelle. La Coiflet d'ordre 1 possède un support plus étendu que les ondelettes précédentes, ce qui réduit sa localisation spatiale tout en favorisant la représentation des régions régulières.